Chương 1

Nền tảng của học máy

> *Mục tiêu của học máy là phát triển các phương pháp có thể tự động phát hiện các khuôn mẫu trong dữ liệu, rồi dùng những khuôn mẫu đã phát hiện được để dự đoán dữ liệu tương lai hoặc những kết quả khác mà ta quan tâm.*
>
> – Murphy [2012]

> *Học máy có thể được định nghĩa một cách khái quát là các phương pháp tính toán sử dụng kinh nghiệm để cải thiện hiệu năng hoặc để đưa ra dự đoán chính xác.*
>
> – Mohri và cộng sự [2018]

NLP dựa trên dữ liệu phù hợp với các định nghĩa trên[1]. Nó dạy máy tính học *kinh nghiệm* ngôn ngữ từ các kho ngữ liệu (*corpora*), rồi hiểu và sử dụng ngôn ngữ dựa trên *kinh nghiệm* đó. Việc kết nối học máy (*machine learning, ML*) với xử lý ngôn ngữ tự nhiên (*natural language processing, NLP*) không chỉ đơn thuần là một cách giúp máy tính bắt chước trí thông minh ngôn ngữ của con người từ dữ liệu. Nó đang dẫn dắt một cuộc cách mạng trong cả hai lĩnh vực: xử lý ngôn ngữ tự nhiên tiến bộ nhờ có được một công cụ mạnh mẽ để rút ra ý nghĩa từ kho ngữ liệu, còn học máy tiến bộ nhờ giải quyết các thách thức của NLP và được kiểm nghiệm trên dữ liệu thực tế.

[1] Từ đây trở đi, chúng tôi bỏ cụm từ *dựa trên dữ liệu* và mặc định rằng mọi mô hình NLP trong phần còn lại của sách đều dựa trên dữ liệu.

Trong chương này, chúng tôi trình bày một số khái niệm và mô hình cơ bản của học máy. Không có phần nào thật sự khó, chỉ là một số kiến thức chuẩn bị cho các chương sau. Ở đây, chúng tôi tập trung vào cách áp dụng học máy cho các bài toán NLP, đặc biệt là cách phát biểu một bài toán NLP thành một bài toán học thống kê. Để làm điều này, chúng tôi bắt đầu với phân loại (*classification*) — một trong những ví dụ được dùng rộng rãi nhất trong hầu hết các sách nhập môn. Sau đó, chúng tôi trình bày một số vấn đề nền tảng của học máy. Tiếp theo là phần thảo luận về các bài toán NLP dưới góc nhìn học máy.

Chú giải của người dịch: Kho ngữ liệu là gì?

Kho ngữ liệu (*corpus*, số nhiều là *corpora*) là một tập hợp rất lớn các văn bản (hoặc bản ghi lời nói) được thu thập để máy tính "đọc" và học. Ví dụ: toàn bộ bài báo của một tờ báo điện tử trong 10 năm, hay hàng triệu trang Wikipedia. "Học kinh nghiệm ngôn ngữ" ở đây có nghĩa là máy tính tự rút ra các quy luật thống kê (từ nào hay đi với từ nào, câu nào thường mang nghĩa tích cực…) từ kho ngữ liệu đó, thay vì được con người viết sẵn từng quy tắc.

1.1 Kiến thức toán cơ bản

Trong phần còn lại của chương này và các chương sau, chúng ta sẽ bàn về các bài toán học máy bằng công cụ toán học ứng dụng. Dưới đây là phần kiến thức toán cơ bản. Nếu thấy các chi tiết này quá đơn giản, bạn có thể chuyển thẳng tới Mục 1.2.

1.1.1 Đại số tuyến tính

1. Vectơ và ma trận

Số vô hướng (*scalar*) có thể không phải là khái niệm đơn giản nhất trong đại số tuyến tính, nhưng chắc chắn là khái niệm phổ biến nhất mà ai cũng học ở trung học hay đại học. Một số vô hướng chính là một con số. Nó là một đại lượng có độ lớn nhưng không có hướng. Chẳng hạn, chiều cao, cân nặng, khoảng cách, nhiệt độ đều là những ví dụ về đại lượng vô hướng. Ở đây, ta dùng chữ in nghiêng để ký hiệu một số vô hướng, ví dụ a, b, x, A, v.v.

Vectơ (*vector*) và **ma trận** (*matrix*) được định nghĩa dựa trên số vô hướng. Một vectơ là một dãy các số vô hướng, hay đơn giản là một danh sách số. Một ma trận là một bảng hình chữ nhật gồm các số vô hướng. Trong sách này, chúng tôi theo quy ước dùng chữ in đậm để ký hiệu vectơ và ma trận. Ví dụ, một vectơ n chiều (*dimension*) có thể được viết là

$$\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}, \tag{1.1}$$

trong đó $\{a_1, a_2, \cdots, a_n\}$ là các phần tử (*elements, entries*) của vectơ. Mỗi phần tử ứng với một chiều. Để tiện ký hiệu, đôi khi ta viết a_i thành $a(i)$. Một vectơ được gọi là vectơ thực chỉ khi mọi phần tử của nó đều là số thực (tức là $a_i \in \mathbb{R}$ với mọi i), ký hiệu là $\mathbf{a} \in \mathbb{R}^n$. Tương tự, ta có thể viết một ma trận $m \times n$ là

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix}, \tag{1.2}$$

trong đó m là số hàng và n là số cột. A_{ij} là phần tử (i, j) của ma trận. Một ma trận thực được ký hiệu là $\mathbf{A} \in \mathbb{R}^{m \times n}$. Đôi khi, ta dùng $\mathbf{A}_{m \times n}$ để nhấn mạnh rằng ma trận có kích thước $m \times n$.

Có một vài ma trận đặc biệt. Chẳng hạn, ma trận có mọi phần tử đều bằng 0 là **ma trận không** (*zero matrix*), ký hiệu là $\mathbf{0}$. Một ví dụ khác là **ma trận đơn vị** (*identity matrix*), ký hiệu là $\mathbf{I}$. Đó là một ma trận vuông có các phần tử trên đường chéo chính đều bằng 1, còn các phần tử khác bằng 0. Vectơ cũng có thể được xem là một loại ma trận đặc biệt. Chẳng hạn, vectơ trong Phương trình (1.1) là một ma trận chỉ có một hàng.

Chú giải của người dịch: So sánh với vectơ ở lớp 10

Ở lớp 10, bạn đã gặp vectơ $\vec{u} = (3; 4)$ trong mặt phẳng tọa độ: một mũi tên có hướng và độ dài. Trong học máy, ta dùng cùng ý tưởng nhưng với *rất nhiều* tọa độ: vectơ $\mathbf{a} = [0.2 \quad -1.5 \quad \cdots \quad 3.1]$ có thể có hàng trăm, hàng nghìn phần tử. Ta không "vẽ" được nó nữa, nhưng các phép toán (cộng, nhân với số, tích vô hướng) vẫn y hệt. *Ma trận* đơn giản là một bảng số hình chữ nhật, giống như bảng điểm của một lớp: mỗi hàng là một học sinh, mỗi cột là một môn. Ví dụ, ma trận $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ là ma trận đơn vị cỡ 2×2; với ma trận

A ở Phương trình (1.3) dưới đây, $A_{12} = 0$ (hàng 1, cột 2) và $A_{23} = 7$ (hàng 2, cột 3). Ký hiệu $\mathbb{R}^{m \times n}$ đọc là "tập các ma trận thực có m hàng, n cột".

2. Chuyển vị ma trận

Chuyển vị (*transpose*) của một ma trận $A_{m \times n}$ là một ma trận $B_{n \times m}$ thỏa mãn $A_{ij} = B_{ji}$ với mọi cặp i và j. Chuyển vị của **A** thường được ký hiệu là $\mathbf{A}^\top$. Ví dụ, với ma trận

$$\mathbf{A} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix}, \tag{1.3}$$

ma trận chuyển vị là

$$\mathbf{A}^\top = \begin{bmatrix} 8 & 2 \\ 0 & 9 \\ 0 & 7 \end{bmatrix}. \tag{1.4}$$

Ta cũng có thể chuyển vị một vectơ. Với vectơ

$$\mathbf{a} = \begin{bmatrix} 1 & 9 & 7 & 3 \end{bmatrix} \tag{1.5}$$

thì vectơ chuyển vị là

$$\mathbf{a}^\top = \begin{bmatrix} 1 \\ 9 \\ 7 \\ 3 \end{bmatrix} \tag{1.6}$$

Nói chung, vectơ chỉ có một hàng được gọi là **vectơ hàng** (*row vector*) (như trong Phương trình (1.5)), còn vectơ chỉ có một cột được gọi là **vectơ cột** (*column vector*) (như trong Phương trình (1.6)). Trong sách này, mặc định mọi vectơ đều là vectơ hàng.

Chú giải của người dịch: Chuyển vị = "lật" ma trận qua đường chéo

Chuyển vị đơn giản là biến hàng thành cột: hàng thứ nhất $[8 \;\; 0 \;\; 0]$ của **A** trở thành cột thứ nhất của $\mathbf{A}^\top$, hàng thứ hai $[2 \;\; 9 \;\; 7]$ trở thành cột thứ hai. Vì thế một ma trận 2×3 sau khi chuyển vị sẽ có kích thước 3×2. Chuyển vị hai lần thì quay về ma trận ban đầu: $(\mathbf{A}^\top)^\top = \mathbf{A}$.

3. Các phép toán theo từng phần tử trên ma trận

Giả sử $\mathbf{A}$ và $\mathbf{B}$ là hai ma trận có cùng kích thước, chẳng hạn $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$. **Phép cộng ma trận** (*matrix addition*) của $\mathbf{A}$ và $\mathbf{B}$ được viết là $\mathbf{A}+\mathbf{B}$. $\mathbf{A}+\mathbf{B}$ là một ma trận trong $\mathbb{R}^{m \times n}$ mà mỗi phần tử bằng tổng của các phần tử tương ứng của $\mathbf{A}$ và $\mathbf{B}$. Dưới đây là một ví dụ.

$$\begin{aligned}\mathbf{A}+\mathbf{B} &= \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 4 \end{bmatrix} \\ &= \begin{bmatrix} 9 & 1 & 1 \\ 3 & 9 & 11 \end{bmatrix}. \end{aligned} \tag{1.7}$$

Theo cách tương tự, ta có thể định nghĩa phép trừ ($\mathbf{A}-\mathbf{B}$), phép nhân ($\mathbf{A} \odot \mathbf{B}$), phép chia ($\mathbf{A} \oslash \mathbf{B}$) theo từng phần tử (*element-wise*) và các phép toán khác. Một trường hợp đặc biệt của phép toán theo từng phần tử là **phép nhân với số vô hướng** (*scalar multiplication*), trong đó ta nhân một ma trận $\mathbf{A}$ với một số vô hướng k, ký hiệu là $k \times \mathbf{A}$ hoặc $k\mathbf{A}$. Xem ví dụ dưới đây với $k = 2$ và $\mathbf{A} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix}$.

$$\begin{aligned} k\mathbf{A} &= 2 \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} \\ &= \begin{bmatrix} 16 & 0 & 0 \\ 4 & 18 & 14 \end{bmatrix}. \end{aligned} \tag{1.8}$$

Chú giải của người dịch: Phép nhân theo từng phần tử

Ký hiệu $\odot$ (đôi khi gọi là tích Hadamard) nghĩa là nhân từng cặp phần tử ở cùng vị trí, giống hệt phép cộng ở trên. Ví dụ, với hai ma trận ở Phương trình (1.7): $\begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} \odot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 4 \end{bmatrix} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 0 & 28 \end{bmatrix}$. Cần phân biệt phép nhân này với *phép nhân ma trận* (mục 5 dưới đây), vốn được tính theo một quy tắc hoàn toàn khác.

Cho $\mathbf{A}$, $\mathbf{B}$ và $\mathbf{C}$ là các ma trận trong $\mathbb{R}^{m \times n}$, k và l là các số vô hướng trong $\mathbb{R}$. Các phép toán ma trận có một số tính chất sau:

- Tính chất của ma trận không:

$$\mathbf{A} = \mathbf{A} + \mathbf{0}$$

$$= \mathbf{0} + \mathbf{A}. \tag{1.9}$$

- **Tính giao hoán**:

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A} \tag{1.10}$$
$$kl\mathbf{A} = lk\mathbf{A}. \tag{1.11}$$

- **Tính kết hợp**:

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) \tag{1.12}$$
$$(kl)\mathbf{A} = l(k\mathbf{A}). \tag{1.13}$$

- **Tính phân phối**:

$$k(\mathbf{A} + \mathbf{B}) = k\mathbf{A} + k\mathbf{B} \tag{1.14}$$
$$(k + l)\mathbf{A} = k\mathbf{A} + l\mathbf{A}. \tag{1.15}$$

4. Tích vô hướng

Tích vô hướng (*dot product*) của hai vectơ cùng số chiều $\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$ và $\mathbf{b} = \begin{bmatrix} b_1 & b_2 & \cdots & b_n \end{bmatrix}$ được định nghĩa là:

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= \sum_{i=1}^{n} a_i b_i \\ &= a_1 b_1 + a_2 b_2 + \cdots + a_n b_n. \end{aligned} \tag{1.16}$$

Về mặt hình học, một vectơ thực **a** có thể được xem như một đối tượng hình học có cả độ lớn (ký hiệu là $|\mathbf{a}|$) và hướng. Tích vô hướng của **a** và **b** cũng có thể được định nghĩa là:

$$\mathbf{a} \cdot \mathbf{b} \;=\; |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta), \tag{1.17}$$

trong đó θ là góc giữa **a** và **b**.

Chú giải của người dịch: Ký hiệu tổng $\sum$ và ý nghĩa của tích vô hướng

Ký hiệu $\sum_{i=1}^{n} a_i b_i$ đọc là "tổng của $a_i b_i$ khi i chạy từ 1 đến n". Ví dụ, với $\mathbf{a} = [1 \;\; 2 \;\; 3]$ và $\mathbf{b} = [4 \;\; 0 \;\; -1]$: $\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot 0 + 3 \cdot (-1) = 1$. Đây chính là công thức tích vô hướng $\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2$ bạn đã học ở lớp 10, chỉ mở rộng ra n tọa độ; Phương trình (1.17) cũng là công thức quen thuộc $\vec{u} \cdot \vec{v} = |\vec{u}|\,|\vec{v}| \cos(\vec{u}, \vec{v})$. Trong NLP, tích vô hướng thường được dùng để đo mức độ "giống nhau" của hai vectơ: hai vectơ càng cùng hướng thì $\cos\theta$ càng gần 1 và tích vô hướng càng lớn.

5. Tích ma trận

Tích ma trận (*matrix product*) (hay **tích ma trận–ma trận** (*matrix-matrix product*)) tác động lên hai ma trận. Cho một ma trận $\mathbf{A} \in \mathbb{R}^{m \times p}$ và một ma trận $\mathbf{B} \in \mathbb{R}^{p \times n}$, tích ma trận của $\mathbf{A}$ và $\mathbf{B}$ tạo ra một ma trận $\mathbf{C} \in \mathbb{R}^{m \times n}$ có các phần tử được định nghĩa là:

$$\begin{aligned} C_{ij} &= \sum_{k=1}^{p} A_{ik} \times B_{kj} \\ &= A_{i1} \times B_{1j} + A_{i2} \times B_{2j} + \cdots + A_{ip} \times B_{pj}. \end{aligned} \tag{1.18}$$

Tích ma trận đòi hỏi số cột của $\mathbf{A}$ phải bằng đúng số hàng của $\mathbf{B}$. Trong sách này, ta dùng $\mathbf{AB}$ để ký hiệu tích ma trận của $\mathbf{A}$ và $\mathbf{B}$. Dưới đây là một vài tính chất của tích ma trận.

- **Tính phân phối**. Với $\mathbf{A} \in \mathbb{R}^{m \times p}$ và $\mathbf{B}, \mathbf{C} \in \mathbb{R}^{p \times n}$, tính phân phối bên trái được định nghĩa là:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) \;=\; \mathbf{AB} + \mathbf{AC}. \tag{1.19}$$

  Với $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times p}$ và $\mathbf{C} \in \mathbb{R}^{p \times n}$, tính phân phối bên phải được định nghĩa là:

$$(\mathbf{A} + \mathbf{B})\mathbf{C} \;=\; \mathbf{AC} + \mathbf{BC}. \tag{1.20}$$

- **Tính kết hợp**. Với $\mathbf{A} \in \mathbb{R}^{m \times p}$, $\mathbf{B} \in \mathbb{R}^{p \times q}$ và $\mathbf{C} \in \mathbb{R}^{q \times n}$, tính kết hợp được phát biểu là

$$(\mathbf{AB})\mathbf{C} \;=\; \mathbf{A}(\mathbf{BC}). \tag{1.21}$$

- **Chuyển vị**. Với $\mathbf{A} \in \mathbb{R}^{m\times p}$ và $\mathbf{B} \in \mathbb{R}^{p\times n}$, ta có

$$(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top. \tag{1.22}$$

Tích ma trận không có tính giao hoán, tức là ta không có $\mathbf{AB} = \mathbf{BA}$ với mọi $\mathbf{A}$ và $\mathbf{B}$, ngay cả khi $\mathbf{A}$ và $\mathbf{B}$ là các ma trận vuông cùng kích thước. Dựa trên tích ma trận–ma trận, ta có thể định nghĩa tương ứng tích vectơ–ma trận và tích ma trận–vectơ. Việc này rất đơn giản, vì tất cả những gì ta cần làm là coi vectơ như một ma trận khi nhân.

Chú giải của người dịch: Nhân ma trận: "hàng nhân cột"

Phần tử C_{ij} của tích $\mathbf{AB}$ chính là *tích vô hướng* của hàng thứ i của $\mathbf{A}$ với cột thứ j của $\mathbf{B}$. Ví dụ:

$$\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix} = \begin{bmatrix}1\cdot 5+2\cdot 7 & 1\cdot 6+2\cdot 8\\ 3\cdot 5+4\cdot 7 & 3\cdot 6+4\cdot 8\end{bmatrix} = \begin{bmatrix}19 & 22\\ 43 & 50\end{bmatrix}.$$

Đổi thứ tự: $\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix}\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix} = \begin{bmatrix}23 & 34\\ 31 & 46\end{bmatrix}$, khác kết quả trên — đó là lý do tích ma trận không giao hoán. Quy tắc kích thước dễ nhớ: $(m\times p)\cdot(p\times n) \to (m\times n)$; hai số "ở giữa" phải bằng nhau và "triệt tiêu" nhau. Chẳng hạn, vectơ hàng 1×3 nhân với ma trận 3×2 cho vectơ hàng 1×2. Phép nhân ma trận là phép toán cơ bản nhất trong mạng nơ-ron: phần lớn thời gian tính toán của một mô hình ngôn ngữ lớn là để nhân các ma trận khổng lồ.

6. Chuẩn

Trong một không gian vectơ, **chuẩn** (*norm*) là một hàm dùng để đo "độ dài" của vectơ. Với vectơ $\mathbf{a} \in \mathbb{R}^n$, chuẩn p (*p-norm*) (hay chuẩn l_p) được định nghĩa là:

$$\|\mathbf{a}\|_p = \Big(\sum_{i=1}^{n} |a_i|^p\Big)^{1/p}. \tag{1.23}$$

Các chuẩn p được dùng phổ biến nhất là với $p = 1$, 2 và ∞:

$$\|\mathbf{a}\|_1 = \sum_{i=1}^{n} |a_i| \tag{1.24}$$

$$\|\mathbf{a}\|_2 = \sqrt{\sum_{i=1}^{n} |a_i|^2} \tag{1.25}$$

$$\|\mathbf{a}\|_\infty = \max\{|a_1|, |a_2|, \cdots, |a_n|\}. \tag{1.26}$$

Chuẩn cũng có thể dùng để đo khoảng cách giữa hai điểm. Cho $\mathbf{b} \in \mathbb{R}^n$ là một vectơ khác. Khoảng cách theo chuẩn p giữa $\mathbf{a}$ và $\mathbf{b}$ là $\|\mathbf{a} - \mathbf{b}\|_p$.

Chú giải của người dịch: Tính thử các chuẩn

Với $\mathbf{a} = [3 \quad -4]$: $\|\mathbf{a}\|_1 = |3| + |-4| = 7$; $\|\mathbf{a}\|_2 = \sqrt{3^2 + 4^2} = 5$ (chính là độ dài vectơ quen thuộc ở lớp 10, theo định lý Py-ta-go); $\|\mathbf{a}\|_\infty = \max\{3, 4\} = 4$. Hình dung trong thành phố có đường phố kẻ ô bàn cờ: chuẩn 1 là quãng đường phải đi dọc theo các phố ("khoảng cách Manhattan"), chuẩn 2 là đường chim bay, còn chuẩn ∞ chỉ quan tâm tới tọa độ chênh lệch lớn nhất. Với $\mathbf{b} = [0 \;\; 0]$, khoảng cách $\|\mathbf{a} - \mathbf{b}\|_2 = 5$.

1.1.2 Xác suất và thống kê

1. Xác suất là gì

Xác suất (*probability*) là một thước đo của sự không chắc chắn. Nó là một đại lượng mô tả khả năng một sự kiện xảy ra. Ví dụ, nếu sự kiện chắc chắn xảy ra, ta nói xác suất bằng 1; nếu sự kiện không bao giờ xảy ra, ta nói xác suất bằng 0.

Một sự kiện (*event*) là kết quả của một phép thử ngẫu nhiên. Ví dụ, trong phép thử tung đồng xu, một sự kiện có thể là "đồng xu ra mặt ngửa". Một tập hợp các sự kiện liên quan được mô tả bởi một **biến ngẫu nhiên** (*random variable*), gọi tắt là **biến** (*variable*). Ví dụ, ta có thể định nghĩa kết quả của việc tung một đồng xu là một biến x. Vì có hai kết quả (ngửa hoặc sấp), ta có hai lựa chọn cho giá trị của x. Ta có thể quy định $x = 1$ khi đồng xu ra mặt ngửa, và $x = 0$ trong trường hợp ngược lại. Do đó, x là một **biến nhị phân** (*binary variable*), hay chính xác hơn là một biến 0-1. Việc một biến ngẫu nhiên nhận một giá trị có nghĩa là một sự kiện xảy ra. Ví dụ, $x = 1$ nghĩa là sự kiện đồng xu ra mặt ngửa.

Trong toán học, xác suất được định nghĩa là một **độ đo xác suất** (*probability measure*) trên một không gian mẫu (*sample space*). Độ đo này phải thỏa mãn một số tiên đề nhất định, chẳng hạn tính cộng tính đếm được. Ở đây chúng tôi bỏ qua các chi tiết của lý thuyết độ đo, và coi xác suất là một hàm ánh xạ các sự kiện tới các số thực trong đoạn $[0, 1]$.

Thông thường, một độ đo xác suất được ký hiệu là một hàm $\Pr(\cdot)$, gọi là **hàm xác suất** (*probability function*). Khi đầu vào của $\Pr(\cdot)$ được định nghĩa trên một tập rời rạc các sự kiện, đầu ra của hàm là xác suất để một sự kiện xảy ra. Ví dụ, $\Pr(x = 1)$ là xác suất để x bằng 1. Lưu ý rằng $\Pr(x)$ là một hàm có đầu ra thay đổi khi ta chọn các giá trị khác nhau của x. Giả sử x_1 là một giá trị mà x có thể nhận. Khi ta viết $\Pr(x_1)$, điều đó có nghĩa là $\Pr(x = x_1)$. Vì xác suất trên tất cả các sự kiện phải bằng 1, mọi hàm xác suất đều phải thỏa mãn:

$$\sum_{x_i \in X} \Pr(x_i) \;=\; 1, \tag{1.27}$$

trong đó X là tập hợp mọi sự kiện. Một hàm xác suất có thể được định nghĩa trên hai biến trở lên. Dưới đây là một vài trường hợp được dùng rộng rãi.

- **Xác suất đồng thời** (*joint probability*). Đây là xác suất để hai sự kiện x và y cùng xảy ra, ký hiệu là $\Pr(x, y)$. Nếu x và y độc lập (*independent*), xác suất đồng thời của chúng bằng tích các xác suất riêng lẻ

$$\Pr(x, y) = \Pr(x)\Pr(y). \tag{1.28}$$

- **Xác suất có điều kiện** (*conditional probability*). Đây là xác suất để x xảy ra khi đã biết y xảy ra, ký hiệu là $\Pr(x|y)$. $\Pr(x|y)$ có thể được định nghĩa là:

$$\Pr(x|y) \;=\; \frac{\Pr(x, y)}{\Pr(y)}. \tag{1.29}$$

- **Xác suất biên** (*marginal probability*). Đây là một cách khác để định nghĩa xác suất của một biến đơn lẻ. Khi biết xác suất đồng thời trên hai biến, xác suất biên được định nghĩa là

$$\Pr(x) \;=\; \sum_{y_j \in Y} \Pr(x, y_j), \tag{1.30}$$

  trong đó Y là không gian sự kiện của y_j. Phương trình (1.30) nói rằng $\Pr(x)$ được lấy biên (*marginalized*) trên Y.

Thêm một lưu ý về xác suất đồng thời. Trong một số trường hợp, người ta muốn dùng xác suất có điều kiện để biểu diễn xác suất đồng thời. Để

làm vậy, ta có thể viết lại xác suất đồng thời theo **quy tắc nhân** (*product rule*) hay **quy tắc chuỗi** (*chain rule*) như sau

$$\Pr(x, y) = \Pr(x|y)\Pr(y). \tag{1.31}$$

Chú giải của người dịch: Ví dụ tính tay: đồng thời, có điều kiện, biên

Một lớp có 40 học sinh. Gọi x là "học sinh thích Toán", y là "học sinh là nữ". Giả sử có 20 bạn nữ, trong đó 12 bạn thích Toán; và 20 bạn nam, trong đó 10 bạn thích Toán. Chọn ngẫu nhiên một học sinh:

- Xác suất đồng thời: $\Pr(x, y) = \Pr(\text{thích Toán và là nữ}) = 12/40 = 0.3$.
- Xác suất có điều kiện: $\Pr(x|y)$ là xác suất thích Toán *khi đã biết* đó là bạn nữ. Ta chỉ xét 20 bạn nữ: $\Pr(x|y) = 12/20 = 0.6$. Theo Phương trình (1.29): $\Pr(x, y)/\Pr(y) = 0.3/0.5 = 0.6$ — khớp.
- Xác suất biên: $\Pr(x) = \Pr(x, \text{nữ}) + \Pr(x, \text{nam}) = 12/40 + 10/40 = 0.55$ — ta "cộng dồn" theo mọi giá trị của y.

Ký hiệu $\Pr(x|y)$ đọc là "xác suất của x biết y". Lưu ý $\Pr(x|y)$ thường khác $\Pr(y|x)$: ở đây $\Pr(y|x) = \Pr(\text{nữ} \mid \text{thích Toán}) = 12/22 \approx 0.545$. Phương trình (1.31) chỉ là Phương trình (1.29) nhân chéo: $0.3 = 0.6 \times 0.5$. Vì $\Pr(x) = 0.55$, $\Pr(y) = 0.5$ mà $\Pr(x)\Pr(y) = 0.275 \neq 0.3$, nên hai sự kiện này *không* độc lập.

Với biến liên tục, xác suất để biến nhận chính xác một giá trị điểm nào đó bằng 0. Thay vào đó, ta có một mật độ tại điểm đó. Chặt chẽ hơn, cho một biến liên tục x, **mật độ xác suất** (*probability density*) của x được viết là $\Pr(x)$. Giả sử $x \in \mathbb{R}$. Xác suất để x nằm trong đoạn $[a, b]$ được định nghĩa qua một tích phân:

$$\Pr(x \in [a, b]) = \int_a^b \Pr(x)dx. \tag{1.32}$$

Hiển nhiên, ta có

$$\int_{-\infty}^{+\infty} \Pr(x)dx = 1. \tag{1.33}$$

Đối với các tính chất khác, như xác suất đồng thời và xác suất có điều kiện, dạng công thức cho biến liên tục gần như giống hệt với biến rời rạc. Ta chỉ cần thay các tổng trong công thức bằng tích phân.

Chú giải của người dịch: Vì sao xác suất tại một điểm bằng 0?

Hãy nghĩ tới chiều cao của một người: xác suất để ai đó cao *đúng* $1.700000\ldots$ m (vô hạn chữ số) gần như bằng 0, nhưng xác suất cao trong khoảng $[1.65; 1.75]$ m thì rõ ràng là dương. Mật độ $\Pr(x)$ giống như "mật độ dân cư": nó cho biết xác suất tập trung dày hay thưa quanh x, và muốn có xác suất thật ta phải lấy tích phân (tức "diện tích dưới đồ thị") trên một đoạn, như ở Phương trình (1.32). Phương trình (1.33) nói rằng tổng diện tích dưới toàn bộ đồ thị mật độ bằng 1.

2. Phân phối và kỳ vọng

Một **phân phối xác suất** (*probability distribution*) (gọi tắt là **phân phối**) mô tả cách các xác suất được gán cho các giá trị có thể có của một biến ngẫu nhiên. Nó được xác định bởi một **hàm khối xác suất** (*probability mass function, PMF*) đối với biến rời rạc, hoặc một **hàm mật độ xác suất** (*probability density function, PDF*) đối với biến liên tục. Ví dụ, một phân phối đều (*uniform distribution*) trên một biến rời rạc x nhận giá trị trong $\{x_1, x_2\}$ có thể được mô tả là $\Pr(x_i) = 1/2$, vì $\Pr(x_1) = \Pr(x_2)$ và $\Pr(x_1) + \Pr(x_2) = 1$; một phân phối đều trên một biến liên tục trong $[-2, 2]$ có thể được mô tả là $\Pr(x) = 1/4$, vì hàm mật độ xác suất $\Pr(x)$ là hằng số với mọi $x \in [-2, 2]$ và $\int_{-2}^{2} \Pr(x)dx = 1$. Các nhà thống kê đã xây dựng nhiều phân phối để mô tả thế giới chúng ta đang sống, chẳng hạn phân phối nhị thức (*binomial distribution*), phân phối Bernoulli (*Bernoulli distribution*) và phân phối Gauss/chuẩn (*Gaussian/normal distribution*). Có thể tìm thấy chi tiết về các phân phối này trong hầu hết các giáo trình thống kê [McClave và Sincich, 2006; Freedman và cộng sự, 2007].

Để mô tả các tính chất của một biến, một cách phổ biến là tính **giá trị kỳ vọng** (*expected value*) hay **kỳ vọng** (*expectation*) của biến đó. Cho x là một biến rời rạc nhận giá trị trong $\{x_1, \cdots, x_n\}$, và $\Pr(x)$ là một phân phối trên x. Giá trị kỳ vọng của x được định nghĩa là:

$$\mathbb{E}_{x\sim\Pr(x)}(x) \;=\; \sum_{i=1}^{n} x_i \cdot \Pr(x_i), \tag{1.34}$$

trong đó chỉ số dưới $x \sim \Pr(x)$ cho biết x tuân theo phân phối $\Pr(x)$. Trong nhiều trường hợp, ta có thể bỏ chỉ số dưới và viết gọn là $\mathbb{E}(x)$. Về bản chất, $\mathbb{E}(x)$ là giá trị trung bình có trọng số của x dưới phân phối

$\Pr(x)$. Nó là một thước đo xu hướng trung tâm, và đôi khi được gọi là **giá trị trung bình** (*mean*) của một biến (ký hiệu là μ).

Tiếp theo, ta có thể định nghĩa **phương sai** (*variance*) của một biến là kỳ vọng của bình phương độ lệch so với giá trị trung bình

$$\text{Var}(x) \;=\; \mathbb{E}[(x - \mathbb{E}(x))^2]. \tag{1.35}$$

Nói một cách không chính thức, nó mô tả các giá trị nằm cách giá trị trung bình xa đến mức nào. $\text{Var}(x)$ thường được viết là σ^2, trong đó σ được gọi là **độ lệch chuẩn** (*standard deviation*).

Với một biến liên tục $x \in \mathbb{R}$, giá trị kỳ vọng được định nghĩa là:

$$\mathbb{E}(x) \;=\; \int_{-\infty}^{+\infty} x \cdot \Pr(x) dx, \tag{1.36}$$

trong đó $\Pr(x)$ là một hàm mật độ xác suất. Để tính phương sai của x, ta chỉ cần dùng lại Phương trình (1.35).

Chú giải của người dịch: Kỳ vọng và phương sai của một con xúc xắc

Gieo một con xúc xắc cân đối, $x \in \{1, 2, \ldots, 6\}$, mỗi mặt có xác suất $1/6$. Theo Phương trình (1.34): $\mathbb{E}(x) = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \cdots + 6 \cdot \frac{1}{6} = 3.5$. Không mặt nào ghi 3.5, nhưng nếu gieo rất nhiều lần thì trung bình cộng các kết quả sẽ tiến gần tới 3.5. Phương sai: $\text{Var}(x) = \frac{1}{6}\big[(1-3.5)^2 + (2-3.5)^2 + \cdots + (6-3.5)^2\big] = \frac{17.5}{6} \approx 2.92$, nên $\sigma \approx 1.71$. Đây chính là số trung bình và phương sai trong thống kê lớp 10, chỉ khác là "tần suất" được thay bằng "xác suất". Ký hiệu $\mathbb{E}[\cdot]$ đọc là "kỳ vọng của…".

3. Entropy

Entropy (*entropy*) là một trong những công cụ quan trọng nhất để mô tả các biến ngẫu nhiên và các quá trình ngẫu nhiên [Shannon, 1948b]. Nó là thước đo của **độ bất ngờ kỳ vọng** (*expected surprisal*). Kết quả càng mang tính tất định thì càng ít bất ngờ và càng chứa ít thông tin. Để đơn giản, ở đây ta chỉ bàn về biến rời rạc[2]. Cho x là một biến và $\Pr(x)$ là một phân phối trên x. Entropy được viết là:

$$H(x) \;=\; -\sum_{i=1}^{n} \Pr(x_i) \cdot \log_b \Pr(x_i), \tag{1.37}$$

[2] Với biến liên tục, ta có các phép tính tương tự.

trong đó b là cơ số của hàm logarit. Giá trị của b thường được chọn là 2, 10 hoặc e.

Chú giải của người dịch: Entropy: đo mức "khó đoán"

Với $b = 2$, entropy đo bằng *bit*. Đồng xu cân đối (Pr $= 0.5$ cho mỗi mặt): $H = -(0.5\log_2 0.5 + 0.5\log_2 0.5) = -(0.5 \cdot (-1) + 0.5 \cdot (-1)) = 1$ bit — khó đoán nhất. Đồng xu "méo" ra ngửa với xác suất 0.9: $H = -(0.9\log_2 0.9 + 0.1\log_2 0.1) \approx 0.469$ bit — dễ đoán hơn nên entropy nhỏ hơn. Đồng xu hai mặt đều là ngửa: $H = 0$ (quy ước $0 \cdot \log 0 = 0$), không có gì bất ngờ. Dấu trừ đứng trước vì log của một số trong $(0, 1]$ luôn ≤ 0, nhờ đó entropy luôn không âm.

Ngoài việc đo độ bất định của một phân phối, ta còn có thể định lượng mức độ khác biệt giữa hai phân phối. Giả sử $p(x)$ và $q(x)$ là các phân phối trên x. **Entropy tương đối** (*relative entropy*) của p so với q được định nghĩa là:

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{i=1}^{n} p(x_i) \cdot \log_b \frac{p(x_i)}{q(x_i)}. \tag{1.38}$$

Ta có thể coi $p(x_i)$ là trọng số cho tỉ số hợp lý (*likelihood ratio*) dạng logarit $\log_b \frac{p(x_i)}{q(x_i)}$. Do đó, $D_{\mathrm{KL}}(p \,\|\, q)$ là tổng có trọng số của các tỉ số hợp lý trên mọi giá trị có thể có. Giá trị $D_{\mathrm{KL}}(p \,\|\, q)$ càng nhỏ cho thấy hai phân phối p và q càng gần nhau. Ví dụ, p và q sẽ trùng nhau nếu $D_{\mathrm{KL}}(p \,\|\, q) = 0$. Entropy tương đối còn được gọi là **độ phân kỳ Kullback-Leibler (KL)** (*Kullback-Leibler (KL) divergence*). Lưu ý rằng entropy tương đối không đối xứng, tức là ta không thể bảo đảm $D_{\mathrm{KL}}(p \,\|\, q) = D_{\mathrm{KL}}(q \,\|\, p)$.

Một khái niệm khác rất phổ biến trong học máy là **entropy chéo** (*cross-entropy*). Nó là thước đo lượng thông tin (tính bằng tổng số bit) cần thiết để truyền các sự kiện từ một nguồn tuân theo một phân phối khi dùng một phân phối khác. Chặt chẽ hơn, ta viết entropy chéo của phân phối p với phân phối q là $H_{\mathrm{cross}}(p, q)$. Nó được định nghĩa là:

$$H_{\mathrm{cross}}(p, q) \;=\; -\sum_{i=1}^{n} p(x_i) \cdot \log_2 q(x_i). \tag{1.39}$$

Giống như entropy tương đối, entropy chéo cũng không đối xứng. Trong bối cảnh tối ưu hóa, chẳng hạn khi huấn luyện các hệ thống NLP, cả hai đều thường được dùng làm hàm mục tiêu (*objective function*). Mối quan

hệ giữa chúng được cho bởi $H(p,q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$. Trong khi entropy tương đối đo số bit **tăng thêm** cần dùng do sử dụng q thay cho p, entropy chéo đo tổng số bit cần dùng.

Chú giải của người dịch: Ví dụ số về độ phân kỳ KL và entropy chéo

Lấy $p = (0.5;\ 0.5)$ (đồng xu cân đối, phân phối "thật") và $q = (0.9;\ 0.1)$ (phân phối mô hình "đoán"), dùng $\log_2$:

- $H_{\text{cross}}(p,q) = -\big(0.5\log_2 0.9 + 0.5\log_2 0.1\big) \approx -(0.5\cdot(-0.152) + 0.5\cdot(-3.322)) \approx 1.737$ bit.
- $H(p) = 1$ bit, nên $D_{\mathrm{KL}}(p \,\|\, q) = H_{\text{cross}}(p,q) - H(p) \approx 0.737$ bit (đúng hệ thức ở trên; ở đây $H(p,q)$ chính là $H_{\text{cross}}(p,q)$).

Nếu mô hình đoán đúng $q = p$ thì $D_{\mathrm{KL}} = 0$ và entropy chéo đạt giá trị nhỏ nhất bằng $H(p)$. Vì vậy, khi huấn luyện mô hình, ta thường *cực tiểu hóa entropy chéo* giữa phân phối của dữ liệu thật p và phân phối dự đoán của mô hình q: làm vậy tương đương với kéo q lại gần p. Bạn sẽ gặp lại ý tưởng này ở hầu hết các chương sau.

1.2 Thiết kế một bộ phân loại văn bản

Phân loại là một trong những bài toán phổ biến nhất trong học máy. Nó nhằm tự động xếp các đối tượng vào một tập các lớp. Các lớp này được gọi là **nhãn** (*labels*), hay **thẻ** (*tags*), hay **hạng mục** (*categories*). Nói chung, một chương trình thực hiện việc phân loại được gọi là một **bộ phân loại** (*classifier*) hay **hệ thống phân loại** (*classification system*). Bộ phân loại có vô số ứng dụng thực tế. Một ví dụ đơn giản là lọc thư rác, trong đó ta cần gán cho mỗi email nhãn "thư rác" hoặc "không phải thư rác". Những ví dụ khó hơn gồm: phân loại ảnh chụp cắt lớp vi tính (CT) các cơ quan nội tạng thành "bình thường" hoặc "bất thường", xác định một đoạn văn bản tiếng Trung có phải do người bản ngữ viết hay không, gán cho một đơn đăng ký sáng chế tập các mã IPC[3] mà nó thuộc về, v.v.

Nhiều lý thuyết và thuật toán học máy được mô hình hóa và kiểm nghiệm trên các bài toán phân loại. Theo thông lệ này, chúng tôi lấy phân loại văn bản (*text classification*) làm ví dụ mở đầu. Giả sử ta có một kho

[3] **[ND]** IPC (*International Patent Classification*) là hệ thống Phân loại sáng chế quốc tế.

ngữ liệu như sau.

Văn bản	Nhãn
Trận đấu thật tuyệt vời.	Không phải đồ ăn
Tôi đã cố hết sức để làm lại món đó trong bếp nhà mình. Vị của nó tuyệt hảo.	Đồ ăn
Hàng thế kỷ nay, rong biển được coi là thức ăn của người bình dân.	Đồ ăn
Hôm nay bạn đã làm xong việc lập trình chưa?	Không phải đồ ăn
Tôi đang thắc mắc sao bạn lại có thể lỡ chuyến xe buýt.	Không phải đồ ăn
Tôi thích trái cây vì nó tốt cho sức khỏe.	Đồ ăn
Nghiên cứu xử lý ngôn ngữ tự nhiên thật thú vị.	Không phải đồ ăn
…	…

Mỗi dòng của kho ngữ liệu là một bộ gồm một đoạn văn bản (từ đây gọi là một tài liệu (*document*)) và một nhãn cho biết văn bản đó có nói về đồ ăn hay không. Ta gọi các bộ như vậy là **mẫu** (*samples*), hay chính xác hơn là **mẫu có nhãn** (*labeled samples*). Về bản chất, mẫu có nhãn là các cặp câu hỏi–câu trả lời, dù chúng không nhất thiết tuân theo dạng chuẩn của câu hỏi và câu trả lời. Chẳng hạn, trong các mẫu ở trên, ta có thể coi một tài liệu là câu hỏi và nhãn của nó là câu trả lời. Trong vài chương tới, chúng tôi sẽ chỉ ra rằng cách phát biểu này là tổng quát và phù hợp với hầu hết các bài toán trong NLP.

Tiếp theo, giả sử ta có một bộ phân loại học cách gán nhãn cho tài liệu từ các mẫu này. Sau đó, bộ phân loại được dùng để gán nhãn "Đồ ăn" hoặc "Không phải đồ ăn" cho những tài liệu mới, chưa từng gặp. Ví dụ, với văn bản

Trái cây không phải món tôi thích nhất, nhưng tôi vẫn có thể ăn ngon lành.

bộ phân loại sẽ xếp nó vào lớp "Đồ ăn".

Tuy nhiên, phân loại văn bản, dù bề ngoài có vẻ đơn giản, không chỉ đơn thuần là sắp xếp các mẫu chưa có nhãn vào các lớp. Nó đặt ra rất nhiều vấn đề, nhất là khi xét đến sự nhập nhằng và phong phú của ngôn ngữ. Các bộ phân loại hiện đại không phải là những hệ thống gồm một tập quy tắc do con người viết tay. Thay vào đó, chúng mô hình hóa bài toán

phân loại theo cách xác suất, nhờ đó có thể học cách phân loại từ dữ liệu có nhãn quy mô lớn.

1.2.1 Phát biểu bài toán

Cho x là một tài liệu và c là một nhãn. Ở đây ta giả định có một bộ phân loại xác suất, ước lượng khả năng ta chọn c làm nhãn của x, ký hiệu là $\Pr(c|x)$. Nói chung, $\Pr(c|x)$ là một **mô hình phân loại** (*classification model*). Nó mô tả một phân phối trên tập mọi nhãn có thể có, thỏa mãn

$$\sum_{c \in \mathcal{C}} \Pr(c|x) \;=\; 1, \tag{1.40}$$

trong đó $\mathcal{C}$ là tập nhãn. Với mọi tài liệu, ta chọn nhãn có xác suất cao nhất làm đầu ra thông qua mô hình phân loại, như sau

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \Pr(c|x), \tag{1.41}$$

trong đó $\hat{c}$ là nhãn "tốt nhất" do mô hình dự đoán. arg max là viết tắt của *arguments of the maxima* (các đối số làm hàm đạt cực đại). Nó trả về giá trị của đối số làm cho một hàm nào đó đạt giá trị lớn nhất.

Chú giải của người dịch: Phân biệt max **và** arg max

max trả về *giá trị lớn nhất*, còn arg max trả về *cái làm cho giá trị lớn nhất*. Ví dụ, giả sử với một tài liệu x, mô hình cho $\Pr(\text{Đồ ăn}|x) = 0.8$ và $\Pr(\text{Không phải đồ ăn}|x) = 0.2$ (tổng bằng 1, đúng Phương trình (1.40)). Khi đó $\max_c \Pr(c|x) = 0.8$, còn $\arg\max_c \Pr(c|x) = \text{Đồ ăn}$. Dấu mũ trong $\hat{c}$ (đọc là "c mũ") là quy ước thường dùng để chỉ một giá trị *dự đoán/ước lượng*, phân biệt với giá trị đúng c.

Phương trình (1.41) là phương trình nền tảng của phân loại. Nó kéo theo ba bài toán

- **Bài toán mô hình hóa**. $\Pr(c|x)$ là một thách thức về mặt tính toán vì không hiển nhiên là làm thế nào để có được giá trị của $\Pr(c|x)$ cho mỗi cặp x và c. Để xây dựng một mô hình phù hợp, ta có thể cần biểu diễn x và c theo một cách tính toán được và định nghĩa một quan hệ toán học kết nối chúng.

- **Bài toán học**. Từ góc nhìn học thống kê, dạng tổng quát của $\Pr(c|x)$ đại diện cho một họ các mô hình được cấu hình với các biến hay tham số (*parameters*) khác nhau. Các mô hình này về bản chất có cùng một dạng, nhưng sẽ hoạt động khác nhau nếu ta chọn các giá trị khác nhau cho các tham số đó. Vì vậy, ta cần chọn một mô hình "tốt" trong số chúng. Việc này thường được giải quyết bằng cách tối ưu các tham số trên dữ liệu có nhãn theo một tiêu chí nào đó.

- **Bài toán dự đoán**. Ở đây ta đang giải quyết một bài toán **phân loại nhị phân** (*binary classification*). Do đó, việc dự đoán lớp của tài liệu rất đơn giản, vì ta chỉ cần xác định lớp nào có xác suất cao hơn lớp còn lại. Tuy nhiên, khó mà hình dung được bài toán dự đoán trong thế giới thực khó đến mức nào, đặc biệt khi phải dự đoán các cấu trúc dạng cây hay đồ thị và các cấu trúc phi tuyến khác[4]. Với nhiều bài toán NLP, việc dự đoán cần đến các thuật toán tìm kiếm hiệu quả và nhanh.

Các bài toán này có tính tổng quát và bao trùm nhiều tác vụ học máy cũng như xử lý ngôn ngữ tự nhiên. Phân loại nhị phân, dù là một trong những trường hợp đơn giản nhất, cũng đủ để minh họa các khái niệm nền tảng của học máy. Phân loại còn có một số biến thể, chẳng hạn:

- **Phân loại đa lớp** (*multi-class classification*). Đây là dạng tổng quát hóa của phân loại nhị phân. Trong phân loại đa lớp, ta cần xếp mẫu vào một trong ba lớp trở lên.

- **Phân loại đa nhãn** (*multi-label classification*). Điều này có thể gây nhầm lẫn vì phân loại đa lớp và phân loại đa nhãn có vẻ là cùng một thứ. Theo cách dùng thuật ngữ thông thường, phân loại đa nhãn là việc gán nhiều nhãn cho một mẫu. Ngược lại, bài toán trình bày trong Phương trình (1.41) là một bài toán **phân loại đơn nhãn** (*single-label classification*).

[4]Dự đoán cây hay các cấu trúc khác không được coi là một bài toán con tiêu chuẩn của phân loại. Nó thường được gọi là **dự đoán cấu trúc** (*structure prediction*). Ở các mục sau, chúng tôi sẽ chỉ ra rằng cả phân loại và dự đoán cấu trúc đều có thể dùng chung một khuôn mẫu học máy tương tự nhau.

Chú giải của người dịch: Đa lớp hay đa nhãn?

Phân loại một bài báo vào *đúng một* trong các chuyên mục "Thể thao", "Kinh tế", "Giáo dục" là phân loại *đa lớp* (có từ ba lớp trở lên, nhưng mỗi mẫu chỉ có một nhãn). Gắn cho một bài báo *nhiều* thẻ cùng lúc, chẳng hạn vừa "Thể thao" vừa "Kinh tế" (bài viết về chuyển nhượng cầu thủ), là phân loại *đa nhãn*.

Phân loại sẽ thú vị và thách thức hơn nếu ta mở rộng nó sang trường hợp xử lý dữ liệu có cấu trúc phân cấp. Ví dụ, với dữ liệu sinh học và dữ liệu sáng chế, một số lớp có thể được gộp vào một lớp cha. Điều này tạo nên một hệ phân cấp các lớp và đòi hỏi một lược đồ phân loại phân cấp (*hierarchical classification*).

1.2.2 Biểu diễn tài liệu bằng vectơ đặc trưng

Vấn đề đầu tiên ta gặp phải khi thiết kế các mô hình phân loại văn bản là làm thế nào để biểu diễn một tài liệu. Coi x đơn thuần là một chuỗi ký tự không phải là giải pháp tốt. Người ta có thể muốn một cách biểu diễn (*representation*) mà qua đó con người hiểu được văn bản. Chẳng hạn, ta có thể phân tích mỗi câu trong tài liệu thành một cây cú pháp và dùng các cây làm biểu diễn văn bản. Tuy nhiên, cách này đòi hỏi công sức phát triển thêm các công cụ NLP khác (như bộ phân tích cú pháp).

Dĩ nhiên, biểu diễn là một vấn đề nền tảng trong NLP. Ở đây chúng tôi bỏ qua các mô hình phức tạp, tiên tiến nhất (*state-of-the-art*), để trình bày một cách tiếp cận đơn giản nhưng hiệu quả — **mô hình túi từ (BOW)** (*bag-of-words model*). Mô hình túi từ là một mô hình biểu diễn tài liệu dựa trên đặc trưng. Trong học máy, một **đặc trưng** (*feature*) là một thuộc tính của mẫu. Nó có thể được định nghĩa không chỉ là một thuộc tính phân loại (ví dụ tên hoặc giới tính) mà còn là một giá trị số, chẳng hạn một số thực.

Trong mô hình túi từ, một đặc trưng tương ứng với tần số (hay số lần xuất hiện) của một từ. Cho V là một bộ từ vựng (*vocabulary*). Một tài liệu có thể được biểu diễn bằng một vectơ đặc trưng (*feature vector*) $|V|$ chiều. Mỗi chiều tương ứng với một đặc trưng đếm từ, biểu diễn số lần từ thứ i trong V xuất hiện trong tài liệu. Chặt chẽ hơn, cho $\mathbf{x}$ là một vectơ

đặc trưng. Phần tử thứ i của $\mathbf{x}$ được định nghĩa là:

$$x_i \;=\; \text{count}(V_i), \tag{1.42}$$

trong đó V_i là từ thứ i trong V và $\text{count}(\cdot)$ là một hàm đếm. Hãy xét ví dụ các dòng văn bản sau[5]. [6]

As I went to Bonner
I met a pig
Without a wig,
Upon my word and honor.

Khi đó ta có một bộ từ vựng được trích ra từ kho ngữ liệu[7], như sau

$$\begin{aligned} V \;=\; \{&\text{“a”, “and”, “as”, “Bonner”, “honor”,} \\ &\text{“I”, “met”, “word”, “my”, “pig”,} \\ &\text{“to”, “upon”, “went”, “wig”, “without”,} \\ &\text{“,”, “.”}\}. \end{aligned}$$

Mỗi dòng của văn bản có thể được xem là một tài liệu và được biểu diễn bằng một vectơ đặc trưng. Dưới đây là các vectơ đặc trưng sinh ra bằng mô hình túi từ.

	a	and	as	Bonner	honor	I	met	word	my	pig	to	upon	went	wig	without	,	.
As I went to Bonner	[0	0	1	1	0	1	0	0	0	0	1	0	1	0	0	0	0]
I met a pig	[1	0	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0]
Without a wig,	[1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0]
Upon my word and honor.	[0	1	0	0	1	0	0	1	1	0	0	1	0	0	0	0	1]

[5] Đoạn văn bản lấy từ tập đồng dao *Mother Goose rhymes*.

[6] **[ND]** Ví dụ này minh họa việc đếm từ tiếng Anh nên được giữ nguyên. Nghĩa đại ý: “Khi tôi đi đến Bonner / Tôi gặp một chú lợn / Không đội tóc giả, / Thề có lời danh dự của tôi.”

[7] Chúng tôi chuyển mọi từ thành chữ thường ở đầu mỗi dòng.

Chú giải của người dịch: Đọc bảng vectơ túi từ

Mỗi hàng là vectơ đặc trưng của một dòng thơ; mỗi cột ứng với một từ trong V (kể cả dấu phẩy và dấu chấm). Ví dụ, dòng "I met a pig" có các từ "I", "met", "a", "pig", mỗi từ xuất hiện 1 lần, nên vectơ có số 1 ở đúng 4 cột đó và số 0 ở 13 cột còn lại; $|V| = 17$ nên vectơ có 17 chiều. Tích vô hướng của vectơ dòng 2 và dòng 3 bằng 1 (chung đúng một từ "a"), còn của dòng 1 và dòng 3 bằng 0 (không chung từ nào). Chú ý rằng mô hình này bỏ qua hoàn toàn *thứ tự* từ: "I met a pig" và "a pig met I" cho cùng một vectơ — giống như đổ các từ vào một cái "túi" rồi chỉ đếm số lượng, đó là nguồn gốc của tên gọi.

Mô hình túi từ định nghĩa một không gian vectơ (*vector space*)[8]. Trong không gian này, độ tương đồng giữa hai vectơ được đo bằng các độ đo như tích vô hướng. Điều này giúp thiết lập quan hệ giữa các tài liệu: hai tài liệu có nhiều từ trùng nhau hơn được coi là giống nhau hơn. Khái niệm trực giác này là nền tảng của nhiều hệ thống phân loại văn bản.

Vẻ đẹp của mô hình túi từ đến từ dạng đơn giản của nó, ở chỗ mỗi từ độc lập với các từ khác. Giả định độc lập này cho phép mã hóa một tài liệu với vô số quan hệ giữa các từ thành một vectơ đếm được, có kích thước khả thi. Mặt khác, biểu diễn một tài liệu bằng vectơ đặc trưng gồm số lần xuất hiện của từ không phải là lựa chọn duy nhất. Để cải tiến, ta có thể tính thêm nhiều ngữ cảnh hơn, và/hoặc thiết kế các đặc trưng mạnh hơn. Điều này dẫn tới một hướng nghiên cứu sôi động về các phương pháp biểu diễn văn bản, từ các phương pháp dựa trên kinh nghiệm (heuristic) đến các phương pháp học biểu diễn (*representation learning*). Chúng ta sẽ thấy một vài phương pháp trong số đó ở các chương sau.

1.2.3 Bộ phân loại tuyến tính

Bộ phân loại tuyến tính (*linear classifier*) là một trong những mô hình nền tảng nhất của học máy. Cho $\mathbf{x} \in \mathbb{R}^n$ là một vectơ đặc trưng đầu vào, và $\mathbf{w} \in \mathbb{R}^n$, $b \in \mathbb{R}$ lần lượt là vectơ trọng số (*weight*) và tham số độ lệch (bias) (*bias*). Một hàm tính điểm tuyến tính được định nghĩa là:

$$s(\mathbf{x}, \mathbf{w}, b) = \mathbf{w} \cdot \mathbf{x} + b$$

[8] Một không gian vectơ phải đóng đối với phép cộng vectơ và phép nhân với số vô hướng.

$$= \sum_{i=1}^{n} w_i x_i + b. \tag{1.43}$$

Để đơn giản hóa ký hiệu, ta thường dùng **mẹo gộp độ lệch** (*bias trick*), định nghĩa một vectơ đầu vào mở rộng $\mathbf{x}' = [\mathbf{x}; 1]$ và vectơ trọng số $\mathbf{w}' = [\mathbf{w}; b]$. Điều này cho phép viết Phương trình (1.43) gọn lại dưới dạng một tích vô hướng:

$$s(\mathbf{x}', \mathbf{w}') \;=\; \mathbf{w}' \cdot \mathbf{x}'. \tag{1.44}$$

Từ đây trở đi, chúng tôi sẽ bỏ số hạng độ lệch tường minh và gọi chung các tham số là $\mathbf{w}$.

Chú giải của người dịch: Hàm tính điểm tuyến tính và mẹo gộp độ lệch

Hàm $s = \mathbf{w}\cdot\mathbf{x}+b$ là phiên bản nhiều chiều của hàm bậc nhất $y = ax+b$ quen thuộc. Mỗi trọng số w_i cho biết đặc trưng x_i "đóng góp" bao nhiêu vào điểm số: w_i dương lớn thì từ thứ i càng ủng hộ lớp đang xét, w_i âm thì ngược lại. Ví dụ: $\mathbf{x} = [2 \;\; 1]$, $\mathbf{w} = [0.5 \;\; -1]$, $b = 3$ cho $s = 0.5\cdot 2 + (-1)\cdot 1 + 3 = 3$. Mẹo gộp độ lệch: nối thêm số 1 vào cuối $\mathbf{x}$ và b vào cuối $\mathbf{w}$: $\mathbf{x}' = [2 \;\; 1 \;\; 1]$, $\mathbf{w}' = [0.5 \;\; -1 \;\; 3]$, khi đó $\mathbf{w}' \cdot \mathbf{x}' = 1 - 1 + 3 = 3$ — cùng kết quả, nhưng công thức chỉ còn một tích vô hướng. Ký hiệu $[\mathbf{x}; 1]$ nghĩa là "ghép thêm phần tử 1 vào sau $\mathbf{x}$".

Trong bối cảnh phân loại, hàm tuyến tính định lượng mức độ "phù hợp" giữa một đầu vào và một lớp. Với phân loại nhị phân có các lớp $\mathcal{C} = \{c_a, c_b\}$, ta gắn với mỗi lớp một vectơ trọng số, ký hiệu là $\mathbf{w}_a$ và $\mathbf{w}_b$.

Để thực hiện dự đoán, ta ánh xạ các điểm số liên tục thành các nhãn lớp rời rạc bằng một hàm quyết định (*decision function*) (thường được gọi là **hàm kích hoạt** (*activation function*) hay **hàm ngưỡng** (*threshold function*)), $\psi(\cdot)$. Với phân loại nhị phân, quy tắc quyết định thường là:

$$\begin{aligned} \hat{c} &= \psi\big(s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b)\big) \\ &= \begin{cases} c_a & s(\mathbf{x}, \mathbf{w}_a) > s(\mathbf{x}, \mathbf{w}_b) \\ c_b & \text{trường hợp ngược lại} \end{cases} \end{aligned} \tag{1.45}$$

Do tính tuyến tính, $s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$. Biểu thức $f(\mathbf{x}) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$ đóng vai trò là **hàm phân biệt tuyến tính** (*linear discriminant function*).

Dù các hàm phân biệt dự đoán trực tiếp nhãn lớp, bản thân chúng không cho ra các ước lượng xác suất. Để có đầu ra dạng xác suất (tức là $\Pr(c|x)$), ta phải ánh xạ các điểm số thành một phân phối xác suất hợp lệ bằng một hàm chuẩn hóa $\psi(\cdot)$, chẳng hạn hàm Softmax:

$$\begin{bmatrix}\Pr(c_a|\mathbf{x}) & \Pr(c_b|\mathbf{x})\end{bmatrix} = \begin{bmatrix}\frac{\exp(s(\mathbf{x},\mathbf{w}_a))}{\exp(s(\mathbf{x},\mathbf{w}_a))+\exp(s(\mathbf{x},\mathbf{w}_b))} & \frac{\exp(s(\mathbf{x},\mathbf{w}_b))}{\exp(s(\mathbf{x},\mathbf{w}_a))+\exp(s(\mathbf{x},\mathbf{w}_b))}\end{bmatrix} \tag{1.46}$$

Theo cách phát biểu này, dự đoán lớp có xác suất cao nhất vẫn tương đương với dự đoán lớp có điểm số cao nhất, nhất quán với Phương trình (1.45).

Chú giải của người dịch: Hàm Softmax biến điểm số thành xác suất

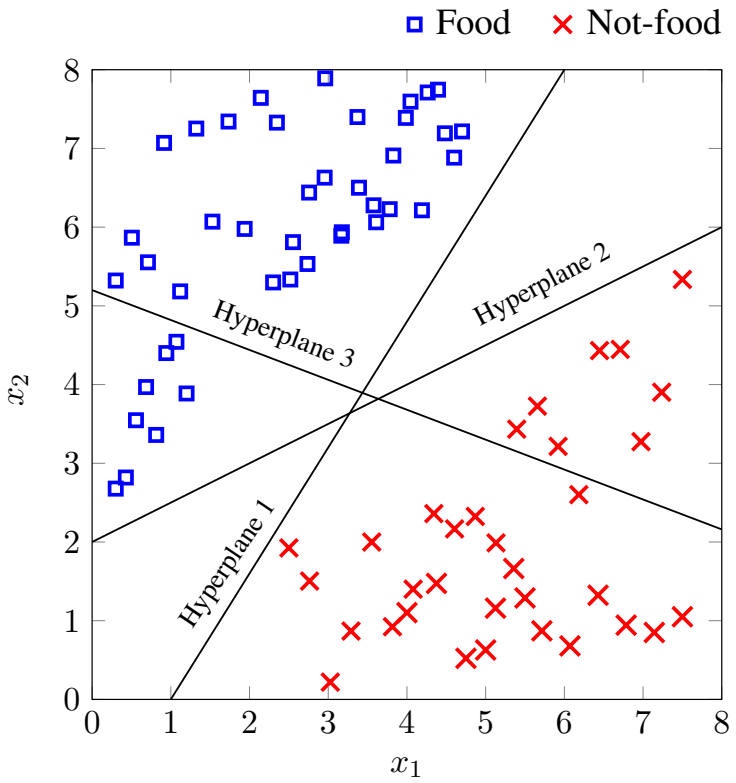
$\exp(s)$ nghĩa là e^s với $e \approx 2.718$. Vì $e^s > 0$ với mọi s, và mỗi phần tử được chia cho tổng tất cả, nên các kết quả đều dương và cộng lại bằng 1 — tức là một phân phối xác suất hợp lệ. Ví dụ, điểm số $s_a = 2$, $s_b = 0$: $\Pr(c_a|\mathbf{x}) = \frac{e^2}{e^2+e^0} = \frac{7.389}{8.389} \approx 0.881$ và $\Pr(c_b|\mathbf{x}) \approx 0.119$. Vì e^s là hàm đồng biến, lớp có điểm cao hơn luôn có xác suất cao hơn, nên quyết định không thay đổi so với Phương trình (1.45).

Về mặt hình học, một bộ phân loại tuyến tính xác định một **siêu phẳng phân tách** (*separating hyperplane*) (hay **biên quyết định** (*decision boundary*)) trong không gian đặc trưng. Hình 1.1 minh họa điều này trong không gian hai chiều. Mục tiêu của việc học là tìm ra một siêu phẳng (như Siêu phẳng 1 và 2) tách đúng các điểm dữ liệu theo lớp, tránh các lỗi mà Siêu phẳng 3 mắc phải.

Cần lưu ý rằng tính tuyến tính là nền tảng của nhiều bộ phân loại, mặc dù phần lớn chúng không có cùng dạng với Phương trình (1.43–1.44). Mô hình tuyến tính là lý tưởng cho các bài toán **tách được tuyến tính** (*linearly separable*). Với dữ liệu không tách được tuyến tính, mô hình tuyến tính thường được kết hợp với các thành phần phi tuyến, chẳng hạn các phép biến đổi đặc trưng phi tuyến hoặc các hàm kích hoạt trong mạng nơ-ron (*neural network*), để nắm bắt những biên quyết định phức tạp hơn.

Chú giải của người dịch: Siêu phẳng là gì?

Trong mặt phẳng hai chiều, tập các điểm thỏa mãn $w_1x_1 + w_2x_2 + b = 0$ là một *đường thẳng* (bạn đã học phương trình tổng quát $ax + by + c = 0$ ở lớp 10). Trong không gian ba chiều, đó là một *mặt phẳng*. Trong không gian



Hình 1.1: Các điểm dữ liệu và các siêu phẳng phân tách trong không gian hai chiều. Có hai lớp điểm dữ liệu (đồ ăn và không phải đồ ăn). Cả siêu phẳng 1 và 2 đều chia không gian thành hai không gian con, trong đó hai lớp điểm dữ liệu được tách riêng. Theo nghĩa này, bài toán ở đây là tách được tuyến tính. Ngược lại, siêu phẳng 3 không tách được hai lớp, tức là các điểm dữ liệu trong cùng một lớp bị phân vào hai lớp khác nhau. (Chú giải nhãn trong hình: *Food* = Đồ ăn, *Not-food* = Không phải đồ ăn, *Hyperplane* = Siêu phẳng.)

n chiều, ta gọi chung là *siêu phẳng*. Đường thẳng này chia mặt phẳng thành hai nửa: điểm nào làm $\mathbf{w} \cdot \mathbf{x} + b > 0$ thì thuộc một lớp, < 0 thì thuộc lớp kia. "Học" một bộ phân loại tuyến tính chính là tìm các hệ số $\mathbf{w}, b$ sao cho đường phân chia này đặt đúng chỗ, như Siêu phẳng 1 hay 2 trong Hình 1.1.

1.2.4 Mô hình sinh và mô hình phân biệt

Các cách tiếp cận phân loại thường được chia thành hai trường phái: **mô hình sinh** (*generative models*) và **mô hình phân biệt** (*discriminative models*). Dù cả hai cách tiếp cận thường dẫn tới các biên quyết định tuyến tính, chúng khác nhau căn bản ở cách mô hình hóa phân phối dữ liệu bên dưới.

1. Mô hình sinh

Một mục tiêu của phân loại là học $\Pr(c|\mathbf{x})$. Mô hình sinh không mô hình hóa trực tiếp xác suất có điều kiện này. Thay vào đó, chúng mô hình hóa xác suất đồng thời $\Pr(\mathbf{x}, c)$, rồi dùng quy tắc Bayes (*Bayes' rule*) để tính $\Pr(c|\mathbf{x})$. Điều này được cho bởi phương trình sau.

$$\begin{aligned}\Pr(c|\mathbf{x}) &= \frac{\Pr(\mathbf{x}, c)}{\Pr(\mathbf{x})} \\ &= \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})},\end{aligned} \tag{1.47}$$

trong đó $\Pr(\mathbf{x}, c)$ được viết lại thành $\Pr(\mathbf{x}|c)\Pr(c)$.

Chú giải của người dịch: Định lý Bayes: "lật ngược" xác suất có điều kiện

Phương trình (1.47) có được bằng cách áp dụng Phương trình (1.31) cho tử số: $\Pr(\mathbf{x}, c) = \Pr(\mathbf{x}|c)\Pr(c)$. Ý nghĩa: ta muốn biết $\Pr(c|\mathbf{x})$ ("biết văn bản, đoán nhãn"), nhưng thường dễ ước lượng chiều ngược lại $\Pr(\mathbf{x}|c)$ ("biết nhãn, văn bản trông thế nào"). Ví dụ: 30% email là thư rác, $\Pr(\text{rác}) = 0.3$. Từ "khuyến mãi" xuất hiện trong 60% thư rác và 5% thư thường. Một email có chữ "khuyến mãi", xác suất nó là thư rác là $\frac{0.3 \times 0.6}{0.3 \times 0.6 + 0.7 \times 0.05} = \frac{0.18}{0.215} \approx 0.84$. Mẫu số $\Pr(\mathbf{x}) = 0.215$ được tính bằng xác suất biên, Phương trình (1.30). $\Pr(c)$ gọi là xác suất *tiên nghiệm* (biết trước khi nhìn dữ liệu), còn $\Pr(c|\mathbf{x})$ là xác suất *hậu nghiệm* (sau khi đã thấy $\mathbf{x}$).

Để tìm lớp tối ưu $\hat{c}$, ta chọn lớp c làm cực đại $\Pr(c|\mathbf{x})$ (xem Phương trình (1.41)). Vì mẫu số $\Pr(\mathbf{x})$ là hằng số đối với lớp c, ta chỉ cần cực đại hóa tử số. Khi đó, ta viết lại Phương trình (1.41) dưới dạng

$$\begin{aligned}\hat{c} &= \arg\max_{c\in\mathcal{C}} \Pr(c|\mathbf{x}) \\ &= \arg\max_{c\in\mathcal{C}} \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})} \\ &= \arg\max_{c\in\mathcal{C}} \Pr(c)\Pr(\mathbf{x}|c), \end{aligned} \tag{1.48}$$

trong đó $\Pr(c)$ là xác suất tiên nghiệm (*prior*) của c, và $\Pr(\mathbf{x}|c)$ là xác suất có điều kiện của vectơ tài liệu đầu vào $\mathbf{x}$ khi biết c. Tính $\Pr(c)$ khá đơn giản. Ví dụ, **ước lượng hợp lý cực đại (MLE)** (*maximum likelihood estimation*) định nghĩa $\Pr(c)$ là một tần suất tương đối:

$$\Pr(c) \;=\; \frac{\text{count}(c)}{\sum_{c'\in\mathcal{C}} \text{count}(c')}, \tag{1.49}$$

trong đó $\text{count}(c)$ đếm số lần xuất hiện của c trong một kho ngữ liệu.

Chú giải của người dịch: Ước lượng hợp lý cực đại (MLE) bằng "đếm"

Ý tưởng của MLE: chọn các giá trị tham số làm cho dữ liệu ta đã quan sát được có xác suất xảy ra *lớn nhất*. Với các phân phối rời rạc như ở đây, MLE cho ra đúng điều trực giác mách bảo: xác suất = tần suất. Ví dụ, nếu kho ngữ liệu có 100 tài liệu, trong đó 40 có nhãn "Đồ ăn" và 60 có nhãn "Không phải đồ ăn", thì $\Pr(\text{Đồ ăn}) = 40/(40+60) = 0.4$. (Có thể kiểm chứng: nếu tung đồng xu 10 lần được 7 ngửa, hàm $p^7(1-p)^3$ đạt cực đại tại $p = 0.7$ — tính bằng đạo hàm như ở lớp 12.)

Tuy nhiên, ước lượng $\Pr(\mathbf{x}|c)$ là việc khó khăn, vì tính thưa của dữ liệu (*data sparsity*) khiến ta không thể ước lượng chính xác với các vectơ nhiều chiều. Nhắc lại rằng mô hình túi từ định nghĩa x_i là tần số của từ V_i trong tài liệu. Ở đây ta giả định rằng vectơ đặc trưng $\mathbf{x} = [x_1, \cdots, x_n]$ được sinh ra bởi một phân phối đa thức (*multinomial distribution*) với các tham số $(p_1(c), \cdots, p_n(c))$, trong đó $p_i(c)$ là xác suất từ V_i xuất hiện trong lớp c. Dựa trên MLE, ta ước lượng $p_i(c)$ là:

$$p_i(c) \;=\; \frac{\text{count}(V_i, c)}{\sum_{1\le i'\le n} \text{count}(V_{i'}, c)}, \tag{1.50}$$

trong đó $\text{count}(V_i, c)$ là số lần V_i xuất hiện trong tất cả các tài liệu có nhãn c. Khi đó, $\Pr(\mathbf{x}|c)$ được cho bởi

$$\Pr(\mathbf{x}|c) \;=\; \frac{\left(\sum_{i=1}^{n} x_i\right)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!} \cdot \prod_{i=1}^{n} p_i(c)^{x_i}. \tag{1.51}$$

Thay Phương trình (1.51) vào Phương trình (1.48), ta có

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \; \Pr(c) \cdot \frac{\left(\sum_{i=1}^{n} x_i\right)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!} \cdot \prod_{i=1}^{n} p_i(c)^{x_i}. \tag{1.52}$$

Chú giải của người dịch: Giải nghĩa Phương trình (1.51)

Ký hiệu $\prod_{i=1}^{n}$ là "tích" (giống $\sum$ nhưng nhân thay vì cộng). Hãy hình dung một tài liệu được "sinh ra" bằng cách rút lần lượt từng từ, mỗi lần rút từ V_i với xác suất $p_i(c)$ (như gieo một con xúc xắc n mặt không cân đối). Xác suất rút ra một *thứ tự* cụ thể gồm x_1 lần từ V_1, …, x_n lần từ V_n là $\prod_i p_i(c)^{x_i}$. Vì mô hình túi từ không quan tâm thứ tự, ta nhân thêm số cách sắp xếp, chính là số *hoán vị lặp* đã học ở THPT: $\frac{(x_1+\cdots+x_n)!}{x_1!\cdots x_n!}$. Ví dụ nhỏ: chỉ có 2 từ với $p_1 = 0.6$, $p_2 = 0.4$; tài liệu gồm 2 lần từ 1 và 1 lần từ 2 thì $\Pr = \frac{3!}{2!\,1!} \cdot 0.6^2 \cdot 0.4 = 3 \cdot 0.144 = 0.432$. Đây là mở rộng của công thức Bernoulli (nhị thức) ra nhiều kết quả.

Lưu ý rằng $\frac{(\sum_{i=1}^{n} x_i)!}{x_1! \cdot x_2! \cdot \cdots \cdot x_n!}$ là hằng số đối với biến lớp c. Ta bỏ nó đi trong arg max, và viết lại vế phải của phương trình theo thang logarit:

$$\hat{c} \;=\; \arg\max_{c \in \mathcal{C}} \; \log(\Pr(c)) + \sum_{i=1}^{n} x_i \cdot \log(p_i(c)). \tag{1.53}$$

Rõ ràng, đây là một mô hình tuyến tính. Nó định nghĩa vectơ đặc trưng và vectơ trọng số như sau

$$\mathbf{x} = \begin{bmatrix} 1 & x_1 & \cdots & x_n \end{bmatrix} \tag{1.54}$$

$$\mathbf{w} = \begin{bmatrix} \log(\Pr(c)) & \log(p_1(c)) & \cdots & \log(p_n(c)) \end{bmatrix}. \tag{1.55}$$

Dạng như vậy đôi khi được gọi là mô hình **log-tuyến tính** (*log-linear*), vì tính tuyến tính có được nhờ biến đổi bài toán ban đầu bằng một phép biến đổi logarit.

Chú giải của người dịch: Vì sao lấy logarit lại không đổi kết quả?

Hàm log đồng biến, nên số nào lớn hơn thì logarit của nó cũng lớn hơn: $\arg\max$ của f và của $\log f$ là như nhau. Logarit biến tích thành tổng ($\log(ab) = \log a + \log b$) và lũy thừa thành phép nhân ($\log p^x = x \log p$), nên tích ở Phương trình (1.52) trở thành tổng ở Phương trình (1.53). Tổng này đúng bằng tích vô hướng $\mathbf{w} \cdot \mathbf{x}$ với $\mathbf{x}, \mathbf{w}$ ở Phương trình (1.54–1.55) — tức là một hàm tính điểm tuyến tính như Phương trình (1.44), với phần tử "1" và $\log \Pr(c)$ đóng vai trò độ lệch. Ngoài ra, trên máy tính, nhân hàng trăm xác suất nhỏ dễ cho kết quả quá gần 0 (tràn dưới), còn cộng các logarit thì an toàn.

Nói chung, Phương trình (1.53) được gọi là phương pháp **Bayes ngây thơ đa thức** (*multinomial naive Bayes*). Tất nhiên, cũng có các biến thể Bayes ngây thơ cho các loại vectơ đặc trưng khác. Ví dụ, với vectơ đặc trưng nhận giá trị nhị phân, ta có thể giả định một phân phối Bernoulli trên mỗi phần tử của vectơ và thiết kế một bộ phân loại **Bayes ngây thơ Bernoulli** (*Bernoulli naive Bayes*); với các vectơ có đặc trưng liên tục, ta có thể giả định một phân phối Gauss trên dữ liệu liên tục và thiết kế một bộ phân loại **Bayes ngây thơ Gauss** (*Gaussian naive Bayes*).

Chú giải của người dịch: Vì sao gọi là "ngây thơ"?

Mô hình giả định rằng, khi đã biết lớp c, mỗi từ xuất hiện *độc lập* với các từ khác: xác suất của cả tài liệu chỉ là tích các xác suất của từng từ. Thực tế điều này không đúng ("bánh" và "mì" hay đi cùng nhau), nên giả định này bị coi là "ngây thơ". Dù vậy, Bayes ngây thơ hoạt động khá tốt trong phân loại văn bản, rất nhanh và dễ cài đặt, chỉ cần đếm. Ví dụ minh họa: nếu trong các tài liệu "Đồ ăn" từ "ngon" chiếm 2% tổng số từ, còn trong tài liệu "Không phải đồ ăn" chỉ chiếm 0.1%, thì mỗi lần "ngon" xuất hiện, điểm $\log p_i(c)$ cộng thêm cho lớp "Đồ ăn" sẽ cao hơn hẳn, kéo quyết định về phía lớp này.

2. Mô hình phân biệt

Mô hình được định nghĩa bởi $\Pr(c|\mathbf{x}) = \frac{\Pr(\mathbf{x},c)}{\Pr(\mathbf{x})} = \frac{\Pr(c)\Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})}$ (xem Phương trình (1.47)) được gọi là mô hình sinh vì nó giả định một quá trình cụ thể để sinh ra dữ liệu $\mathbf{x}$ khi biết nhãn c. Ý tưởng là dùng xác suất đồng thời $\Pr(\mathbf{x}, c)$ làm bước trung gian để tính $\Pr(c|\mathbf{x})$. Ngược lại, **mô hình phân biệt** không cố mô hình hóa phân phối của $\mathbf{x}$. Thay vào đó, chúng ước lượng trực tiếp $\Pr(c|\mathbf{x})$.

Một ví dụ là **hồi quy logistic** (*logistic regression*). Nhắc lại rằng với phân loại văn bản nhị phân, một bộ phân loại Bayes ngây thơ dự đoán nhãn c_a cho một tài liệu $\mathbf{x}$ chỉ khi hàm log-odds (*log-odds*) dương:

$$f_a(\mathbf{x}) \;=\; \log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})}. \tag{1.56}$$

Ta có thể giả định rằng đại lượng này tuân theo một mô hình tuyến tính:

$$\log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})} \;=\; \mathbf{w}\cdot\mathbf{x}. \tag{1.57}$$

Vì $\Pr(c_b|\mathbf{x}) = 1 - \Pr(c_a|\mathbf{x})$, giải theo $\Pr(c_a|\mathbf{x})$ ta được:

$$\Pr(c_a|\mathbf{x}) \;=\; \frac{1}{1+\exp(-\mathbf{w}\cdot\mathbf{x})}. \tag{1.58}$$

Đây là một hàm sigmoid (*sigmoid function*), cụ thể là hàm logistic (*logistic function*). Theo mô hình này, ta dự đoán c_a chỉ khi $\mathbf{w}\cdot\mathbf{x} > 0$. Phương trình (1.58) định nghĩa bộ phân loại hồi quy logistic, đóng vai trò là phiên bản phân biệt tương ứng với bộ phân loại Bayes ngây thơ.

Chú giải của người dịch: Từ log-odds đến hàm sigmoid

Đặt $p = \Pr(c_a|\mathbf{x})$ và $z = \mathbf{w}\cdot\mathbf{x}$. Phương trình (1.57) là $\log \frac{p}{1-p} = z$, suy ra $\frac{p}{1-p} = e^z$, tức $p = e^z(1-p)$, nên $p(1+e^z) = e^z$ và $p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$ — đúng Phương trình (1.58). Hàm $\sigma(z) = \frac{1}{1+e^{-z}}$ có đồ thị hình chữ S, "nén" mọi số thực vào khoảng $(0,1)$: $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$. Vì vậy $p > 0.5$ khi và chỉ khi $z > 0$. "Odds" (tỉ lệ cược) $\frac{p}{1-p}$ là cách nói quen thuộc trong cá cược: $p = 0.75$ ứng với odds 3 ("ba ăn một").

Một ưu điểm của mô hình phân biệt là sự linh hoạt khi phát biểu các bài toán phân loại. Trong khi mô hình sinh cố gắng ước lượng phân phối

dữ liệu của $\mathbf{x}$, mô hình phân biệt tập trung vào việc thiết lập một ranh giới vững chắc giữa các lớp. Do đó, mô hình phân biệt quan tâm nhiều hơn tới việc lớp nào được ưu tiên hơn lớp nào khi biết $\mathbf{x}$, hay theo ngôn ngữ xác suất, lớp nào có khả năng xuất hiện cao hơn, thay vì đưa ra các giả định mạnh về từng điểm dữ liệu riêng lẻ. Điều này cho phép học mô hình bằng cách cực tiểu hóa một độ đo lỗi cụ thể, thay vì chỉ cực đại hóa hàm hợp lý (*likelihood*) của các điểm dữ liệu đầu vào. Cách tiếp cận này thuộc khuôn khổ **học dựa trên lỗi** (*error-driven learning*).

Có nhiều cách định nghĩa các lỗi này. Tương tự như mô hình sinh, ta có thể huấn luyện một mô hình phân biệt bằng cách tối ưu các tham số $\mathbf{w}$ để cực đại hóa hợp lý có điều kiện (*conditional likelihood*) của dữ liệu huấn luyện. Cho $\{(\mathbf{x}^{(1)}, c^{(1)}), \cdots, (\mathbf{x}^{(K)}, c^{(K)})\}$ là một tập các tài liệu có nhãn. Vectơ tham số tối ưu $\hat{\mathbf{w}}$ được cho bởi:

$$\hat{\mathbf{w}} \;=\; \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \log \Pr(c^{(k)} | \mathbf{x}^{(k)}). \tag{1.59}$$

Áp dụng vào Phương trình (1.58), quá trình này tương ứng với việc cực đại hóa log-hợp lý (*log-likelihood*):

$$\begin{aligned}
\hat{\mathbf{w}} &= \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \Big[\delta(c_a, c^{(k)}) \log \Pr(c_a | \mathbf{x}^{(k)}) + \delta(c_b, c^{(k)}) \log \Pr(c_b | \mathbf{x}^{(k)}) \Big] \\
&= \arg\max_{\mathbf{w}} \sum_{k=1}^{K} \Bigg[\delta(c_a, c^{(k)}) \log \Big(\frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x}^{(k)})} \Big) + \\
&\qquad\qquad \delta(c_b, c^{(k)}) \log \Big(1 - \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x}^{(k)})} \Big) \Bigg],
\end{aligned} \tag{1.60}$$

trong đó $\delta(\cdot, \cdot)$ là một **hàm chỉ thị** (*indicator function*), trả về 1 nếu hai đối số bằng nhau và 0 trong trường hợp ngược lại.

Chú giải của người dịch: Đọc Phương trình (1.59–1.60)

Chỉ số trên (k) trong $\mathbf{x}^{(k)}$ là *số thứ tự của mẫu* thứ k trong dữ liệu huấn luyện, không phải lũy thừa. Phương trình (1.59) nói: hãy chọn $\mathbf{w}$ sao cho mô hình gán xác suất càng cao càng tốt cho *nhãn đúng* của mọi mẫu. Lấy log rồi cộng lại tương đương với nhân các xác suất (xem chú giải ở Phương trình (1.53)). Trong Phương trình (1.60), hàm δ đóng vai trò "công tắc": với mỗi mẫu,

đúng một trong hai số hạng được giữ lại — số hạng ứng với nhãn đúng của mẫu đó. Ví dụ, nếu mẫu k có nhãn c_a và mô hình cho $\Pr(c_a|\mathbf{x}^{(k)}) = 0.9$, mẫu đóng góp $\log 0.9 \approx -0.105$; nếu mô hình chỉ cho 0.1 thì đóng góp $\log 0.1 \approx -2.303$, bị "phạt" nặng hơn nhiều. Đổi dấu đại lượng này, ta được đúng *entropy chéo* đã gặp ở Mục 1.1.2.

Một cách khác là huấn luyện mô hình bằng cách cực tiểu hóa hàm mất mát 0-1 (*0-1 loss*), tức là ta tính một lỗi mỗi khi nhãn dự đoán không khớp với nhãn đúng:

$$\hat{\mathbf{w}} \;=\; \arg\min_{\mathbf{w}} \sum_{k=1}^{K} \big(1 - \delta(\hat{c}^{(k)}, c^{(k)})\big), \tag{1.61}$$

trong đó $\hat{c}^{(k)}$ là nhãn dự đoán. Dù chặt chẽ về mặt toán học, việc tối ưu trực tiếp hàm mất mát 0-1 rất khó vì nó không khả vi, dẫn tới việc dùng các hàm mất mát thay thế (*surrogate losses*) như log-hợp lý nói trên.

Chú giải của người dịch: Vì sao "không khả vi" lại gây khó?

Hàm mất mát 0-1 chỉ đếm số mẫu bị đoán sai, nên nó nhận giá trị nguyên và "nhảy bậc": thay đổi $\mathbf{w}$ một chút thường không làm số lỗi thay đổi (đạo hàm bằng 0), còn khi thay đổi thì nhảy đột ngột (không có đạo hàm). Các phương pháp huấn luyện phổ biến (sẽ học ở các chương sau) dựa vào đạo hàm để biết nên chỉnh $\mathbf{w}$ theo hướng nào, nên cần một hàm "trơn" như log-hợp lý, vốn thay đổi liên tục theo $\mathbf{w}$.

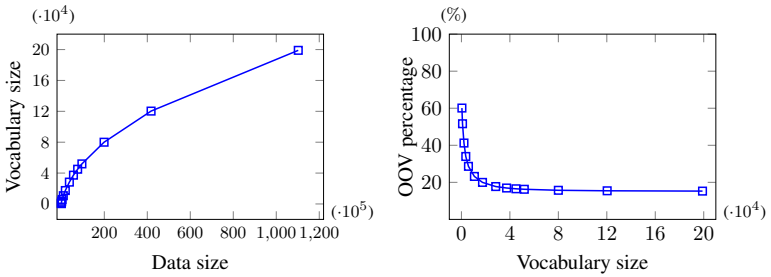
Việc lựa chọn mục tiêu huấn luyện có vai trò then chốt trong các mô hình phân biệt. Chủ đề này rất rộng và sẽ được bàn kỹ hơn ở Mục 1.3.4. Hơn nữa, mô hình phân biệt không đòi hỏi các đặc trưng phải tuân thủ chặt chẽ một quá trình sinh xác suất. $\mathbf{x}$ có thể là bất kỳ vectơ đặc trưng nào được thiết kế cho bài toán. Ví dụ, cho $g(\mathbf{x})$ là điểm số đầu ra của một hệ thống khác. Theo Phương trình (1.58), ta có thể xây dựng một mô hình phân loại nhị phân mới:

$$\Pr(c_a|\mathbf{x}) \;=\; \frac{1}{1 + \exp(-\mathbf{w} \cdot g(\mathbf{x}))}. \tag{1.62}$$

Để học $g(\mathbf{x})$, ta có thể tiền huấn luyện (*pre-train*) nó trên một số dữ liệu bổ sung, hoặc huấn luyện nó đồng thời với $\mathbf{w}$ (tức là các tham số của mô hình cấp trên $\Pr(c_a|\mathbf{x})$).

1.2.5 Từ ngoài bộ từ vựng và làm trơn

Vấn đề **từ ngoài bộ từ vựng** (*out-of-vocabulary*) (**OOV**) xảy ra khi một số từ của tài liệu không có trong bộ từ vựng. Từ OOV rất phổ biến trong NLP vì luôn có từ mới, bất kể ta đã thấy bao nhiêu văn bản. Hình 1.2 đưa ra hai đường cong minh họa vấn đề này. Như thể hiện trong Hình 1.2 (a), các từ mới liên tục xuất hiện khi có thêm dữ liệu. Khi ta cố định dữ liệu dùng để kiểm tra độ phủ của bộ từ vựng, từ OOV vẫn còn tồn tại ngay cả khi ta có một bộ từ vựng cực lớn (Hình 1.2 (b)).



Hình 1.2: Kích thước dữ liệu (tính theo số từ), kích thước bộ từ vựng và tỉ lệ phần trăm OOV. Số liệu thống kê được thu thập trên dữ liệu tiếng Anh của bài toán dịch Trung–Anh (Zh-En) WMT21. Ta dùng càng nhiều dữ liệu thì bộ từ vựng càng lớn. Kích thước bộ từ vựng vẫn tiếp tục tăng ngay cả khi ta xây dựng bộ từ vựng trên dữ liệu hơn 100 triệu từ. Tuy nhiên, độ dốc của đường cong có xu hướng nhỏ dần khi dùng thêm nhiều dữ liệu. Điều thú vị là tỉ lệ OOV hội tụ về một mức nhất định khi kích thước bộ từ vựng tăng, cho thấy luôn có từ mới xuất hiện bất kể ta đã quan sát bao nhiêu từ. (Đồ thị trái là (a), đồ thị phải là (b). Nhãn trục: *Data size* = kích thước dữ liệu, *Vocabulary size* = kích thước bộ từ vựng, *OOV percentage* = tỉ lệ OOV; $(\cdot 10^4)$ nghĩa là giá trị trên trục cần nhân với 10^4.)

Trong các hệ thống thực tế, từ OOV rất phổ biến vì bộ từ vựng thường bị giới hạn ở một số lượng "nhỏ" mục từ. Một phương pháp chuẩn là giữ lại n từ có tần số cao nhất và loại bỏ phần còn lại. Khi đó, các từ OOV được coi là từ *không xác định* (*unknown*). Ví dụ, một ký hiệu mới <unk> được đưa vào bộ từ vựng để mọi từ OOV đều được ký hiệu là <unk>. Một ý tưởng táo bạo hơn là xây dựng một hệ thống **từ vựng mở** (*open-vocabulary*) chấp nhận mọi từ có thể có, nhưng điều này đòi hỏi các thuật toán phức tạp hơn và có lẽ cả việc thiết kế cấu trúc dữ liệu riêng cho từng bài toán. Việc dùng <unk> hiện vẫn là chuẩn mực trên thực tế trong việc

phát triển các hệ thống NLP.

Với những từ đã có trong bộ từ vựng, cũng tồn tại các **từ chưa gặp** (*unseen words*): những từ vắng mặt trong giai đoạn ước lượng tham số nhưng lại xuất hiện trong một tài liệu mới. Một cách cài đặt ngây thơ mô hình mô tả ở mục trước sẽ gặp khó khi xử lý các từ chưa gặp. Ví dụ, Phương trình (1.50) sẽ đơn giản cho xác suất bằng 0 nếu từ V_i không xuất hiện trong bất kỳ mẫu huấn luyện nào có nhãn c. Hệ quả là, với một mẫu mới chứa V_i, Phương trình (1.51) sẽ gán cho nó xác suất bằng 0. Kết quả này rõ ràng là bất hợp lý. Tuy nhiên, ta không nên đơn giản quy lỗi cho bản thân thiết kế mô hình. Thay vào đó, nguyên nhân chính là dữ liệu dùng để ước lượng tham số không đủ. Điều này cũng được giải thích bởi **định luật Zipf** (*Zipf's Law*): một số ít từ xuất hiện rất thường xuyên, trong khi một số lượng lớn từ hiếm khi xuất hiện.

Chú giải của người dịch: Vì sao một số 0 làm hỏng cả tài liệu?

Phương trình (1.51) là một *tích* các $p_i(c)^{x_i}$. Chỉ cần một từ trong tài liệu có $p_i(c) = 0$ (với $x_i \geq 1$) thì cả tích bằng 0, bất kể các từ còn lại ủng hộ lớp c mạnh đến đâu. Ví dụ, một bài viết về ẩm thực dài 200 từ nhưng có một từ "phở cuốn" chưa từng xuất hiện trong dữ liệu huấn luyện lớp "Đồ ăn" sẽ có $\Pr(\mathbf{x}|\text{Đồ ăn}) = 0$ (và $\log 0 = -\infty$ trong Phương trình (1.53)) — rõ ràng là vô lý. Theo định luật Zipf, từ phổ biến thứ r có tần số xấp xỉ tỉ lệ với $1/r$: từ đứng đầu xuất hiện gấp đôi từ thứ hai, gấp ba từ thứ ba… nên luôn có một "cái đuôi" rất dài gồm các từ hiếm.

Tuy nhiên, ta không thể giả định rằng mình luôn có được dữ liệu trong đó mọi từ đều xuất hiện đủ nhiều. Thay vào đó, ta có thể dùng các kỹ thuật làm trơn (*smoothing*) để phân phối lại xác suất trên bộ từ vựng. Xét Phương trình (1.50) làm ví dụ. Ta có thể cộng thêm một số nhỏ cho mỗi V_i, và viết lại phương trình thành:

$$p_i(c) \;=\; \frac{\text{count}(V_i, c) + \alpha}{\sum_{1 \leq i' \leq n} \text{count}(V_{i'}, c) + n \cdot \alpha}, \tag{1.63}$$

trong đó α là số đếm mặc định ta gán cho mỗi từ. Bằng cách này, $p_i(c)$ cho xác suất khác 0 ngay cả khi $\text{count}(V_i, c) = 0$. Phương trình (1.63) chuyển một phần khối lượng xác suất từ các từ tần số cao sang các từ tần số thấp. Phương pháp này được gọi là **làm trơn cộng** (*additive smoothing*) hay **làm trơn cộng-α** (*add-α smoothing*). Đây là một trong những phương

pháp làm trơn đơn giản nhất. Về các phương pháp khác, chúng tôi giới thiệu bạn đọc tới các bài báo về mô hình hóa ngôn ngữ, nơi làm trơn được dùng rất nhiều [Chen và Goodman, 1999].

Chú giải của người dịch: Ví dụ tính tay làm trơn cộng-α

Giả sử bộ từ vựng có $n = 4$ từ, và trong các tài liệu lớp "Đồ ăn" ta đếm được: "ngon" 5 lần, "bếp" 3 lần, "trái cây" 2 lần, "xe buýt" 0 lần (tổng 10). Không làm trơn (Phương trình (1.50)): $p = (0.5; 0.3; 0.2; 0)$. Với $\alpha = 1$ (còn gọi là *làm trơn Laplace*), mẫu số thành $10 + 4 \cdot 1 = 14$: $p = (6/14; 4/14; 3/14; 1/14) \approx (0.43; 0.29; 0.21; 0.07)$. Tổng vẫn bằng 1, từ chưa gặp "xe buýt" nhận xác suất nhỏ nhưng khác 0, còn các từ phổ biến bị "bớt" đi một chút. Hạng tử $n \cdot \alpha$ ở mẫu số chính là để bù cho việc mỗi trong n từ được cộng thêm α.

Cần lưu ý rằng trong NLP hiện đại, vấn đề OOV truyền thống ít được bàn đến hơn. Như chúng tôi sẽ giới thiệu ở các chương sau, các hệ thống NLP gần đây thường dùng các phương pháp tách token theo **từ con (subword)** (*subword*). Trong các hệ thống này, đơn vị xử lý không còn là các từ theo nghĩa ngôn ngữ học truyền thống nữa, mà là các "đơn vị" được học bằng thống kê. Vì tuyệt đại đa số các từ chưa gặp đều có thể được ghép lại từ các đơn vị từ con này (trong trường hợp cực đoan, từ có thể được coi đơn giản là chuỗi các ký tự), việc dùng một bộ từ vựng từ con tương đối gọn có thể giảm đáng kể vấn đề OOV.

Chú giải của người dịch: Từ con giúp gì cho vấn đề OOV?

Ví dụ, từ tiếng Anh *unhappiness* có thể chưa có trong bộ từ vựng, nhưng có thể được tách thành các từ con quen thuộc "un" + "happi" + "ness". Như vậy mô hình vẫn "đọc" được từ mới thông qua các mảnh đã biết, thay vì phải thay cả từ bằng `<unk>`. Cách tách cụ thể (ví dụ thuật toán BPE) sẽ được trình bày ở các chương sau.

1.3 Các vấn đề chung

Xây dựng một bộ phân loại văn bản đơn giản là một khởi đầu tốt nhưng chưa đủ để giải quyết các bài toán thực tế phức tạp và đa dạng. Để có một bức tranh tổng quát hơn về cách các hệ thống học máy hiện đại vận hành,

giờ đây chúng ta sẽ bàn về một số vấn đề quan trọng khi thiết kế những hệ thống như vậy.

1.3.1 Mô hình có giám sát và không giám sát

Học có giám sát (*supervised learning*) làm việc với các mẫu có nhãn, nghĩa là mỗi đầu vào x được gắn với một đầu ra y. Cho một tập các cặp đầu vào–đầu ra $\{(x^{(1)}, y^{(1)}), \cdots, (x^{(K)}, y^{(K)})\}$, nhiệm vụ ở đây là học một hàm $f(\cdot)$ ánh xạ mỗi $x^{(k)}$ thành $y^{(k)}$:

$$y^{(k)} \;=\; f(x^{(k)}). \tag{1.64}$$

Quá trình này được gọi là *có giám sát* vì việc học $f(\cdot)$ được dẫn dắt bởi câu trả lời $y^{(k)}$ do con người gán nhãn thủ công cho $x^{(k)}$. Nói chung, $y^{(k)}$ được gọi là nhãn đúng thực tế (*ground-truth*) hay nhãn chuẩn vàng (*gold-standard*) của $x^{(k)}$. Sau đó, khi có một đầu vào mới x_{new}, ta dùng hàm $f(\cdot)$ đã học để dự đoán đầu ra. Ta nói rằng việc học có giám sát thành công nếu dự đoán $f(x_{\text{new}})$ khớp với nhãn đúng y_{new}.

Tuyệt đại đa số các bài toán NLP có thể được phát biểu thành bài toán học có giám sát. Gán một lớp cho một tài liệu chắc chắn là một trong những trường hợp đơn giản nhất. Các bài toán NLP khác bao gồm (nhưng không giới hạn ở) việc sinh ra một chuỗi nhãn, một đoạn văn bản, một cây cú pháp hay một đồ thị.

Trái với học có giám sát, **học không giám sát** (*unsupervised learning*) làm việc với các mẫu không có nhãn; nói cách khác, với mỗi mẫu, ta có đầu vào x nhưng không có đầu ra đúng y. Trong trường hợp này, ta cần một thuật toán học từ dữ liệu không nhãn $\{x^{(k)}\}$ một hàm ánh xạ từ $x^{(k)}$ sang một đầu ra nào đó. Vì không có sự can thiệp của con người vào đầu ra của hàm, các thuật toán loại này cần tự phát hiện các khuôn mẫu trong $\{x^{(k)}\}$ và tối ưu cách ta biểu diễn $\{x^{(k)}\}$ cùng đầu ra của hàm theo một tiêu chí nào đó. Các tiêu chí này thường xuất phát từ tri thức có trước của con người, để hàm thu được có thể cho ra thứ mà ta mong đợi.

Một ví dụ phổ biến trong học không giám sát là **phân cụm từ** (*word clustering*). Nó gom nhóm một tập các từ bằng cách xếp các từ tương tự vào cùng một cụm[9]. Dựa trên tiêu chí như vậy, ta có thể hướng $f(\cdot)$ tới việc

[9] Việc xác định hai từ có tương tự nhau hay không có thể khó và mơ hồ. Chúng tôi để vấn đề này lại cho Chương 3.

cho ra cùng một cụm với các từ tương tự nhau. Một trường hợp khó hơn là **quy nạp từ điển song ngữ không giám sát** (*unsupervised bilingual dictionary induction*). Nó học một ánh xạ ở cấp độ từ giữa hai ngôn ngữ mà không cần dữ liệu song song (*parallel data*). Bài toán này thường được giải quyết (một phần) bằng cách tận dụng tính đẳng cấu (*isomorphism*) giữa các không gian biểu diễn từ của các ngôn ngữ khác nhau.

Chú giải của người dịch: Phân cụm từ và từ điển song ngữ không giám sát

Phân cụm giống như việc xếp sách vào các kệ mà không ai cho trước tên kệ: máy tự nhận ra rằng "táo, cam, chuối" nên ở cùng một nhóm, còn "ô tô, xe máy, xe đạp" ở một nhóm khác. *Dữ liệu song song* là các cặp câu cùng nghĩa ở hai ngôn ngữ (ví dụ một câu tiếng Việt và bản dịch tiếng Anh của nó). Ý tưởng "đẳng cấu" ở đây là: nếu biểu diễn mỗi từ bằng một điểm trong không gian, thì "hình dạng" của đám điểm tiếng Việt và đám điểm tiếng Anh khá giống nhau ("mèo" gần "chó" giống như *cat* gần *dog*). Vì vậy, ta có thể tìm cách xoay, khớp hai đám điểm vào nhau để suy ra từ nào dịch thành từ nào, mà không cần bất kỳ cặp câu dịch nào.

Nằm giữa học có giám sát và học không giám sát là **học bán giám sát** (*semi-supervised learning*). Nó xử lý những tình huống trong đó chỉ một phần nhỏ dữ liệu đầu vào có nhãn, còn phần lớn vẫn không có nhãn. Như vậy, nó tận dụng được ưu điểm của cả hai mô thức. Ví dụ, trong dịch máy (*machine translation*), ta có thể có một lượng dữ liệu song song (có nhãn) nhất định và lượng dữ liệu đơn ngữ (không nhãn) lớn hơn nhiều bậc độ lớn. Thông thường, một hệ thống cơ sở được huấn luyện trên dữ liệu song song theo cách có giám sát. Sau đó có thể cải thiện hệ thống bằng cách huấn luyện các thành phần của hệ thống cơ sở này trên dữ liệu đơn ngữ quy mô lớn. Điều này được thực hiện hoặc bằng cách kết hợp một mô hình dịch học từ dữ liệu song ngữ với một mô hình ngôn ngữ (*language model*) học từ dữ liệu đơn ngữ của ngôn ngữ đích, hoặc bằng cách tiền huấn luyện (*pre-training*) một số phần của mô hình trên dữ liệu đơn ngữ rồi tinh chỉnh (*fine-tuning*) toàn bộ mô hình trên dữ liệu song ngữ.

Nói rộng ra, mọi thuật toán học đều cần sự giám sát. Điều này nghe có vẻ trái trực giác vì học không giám sát dường như không nhận được tín hiệu nào từ dữ liệu nhãn đúng. Tuy nhiên, từ góc nhìn tổng quát về việc học, tín hiệu giám sát không nên bị giới hạn ở các nhãn do con người cung

cấp. Ngay cả trong một bài toán học không giám sát, quá trình học vẫn được giám sát bởi tri thức có trước và bởi các khuôn mẫu phân phối ẩn chứa trong dữ liệu đầu vào $\{x^{(k)}\}$. Theo nghĩa này, học không giám sát không hẳn là "học mà không có giám sát".

Hiểu "giám sát" theo nghĩa rộng này, ta thấy xuất hiện thêm những mô thức khác cũng là các dạng của học máy, dù chúng không thuộc gọn vào các nhóm có giám sát hay không giám sát truyền thống. Một ví dụ là **học tăng cường** (*reinforcement learning*). Nó mô hình hóa cách một hệ thống đưa ra một chuỗi các quyết định bằng cách vận hành một tác tử (*agent*) trong một môi trường (*environment*). Tác tử nhận được phản hồi (hay một phần thưởng (*reward*)) từ môi trường khi thực hiện một hành động (*action*). Mục tiêu của học tăng cường là học một chính sách (policy) (*policy*) ra quyết định sao cho tối đa hóa phần thưởng tích lũy qua các bước mà tác tử thực hiện. Thường thì phần thưởng thực sự chỉ có được khi tác tử đạt tới một trạng thái kết thúc (*terminal state*), chẳng hạn thắng hoặc thua một ván cờ. Do đó, học tăng cường rất phù hợp để mô tả các bài toán có đặc trưng là **phần thưởng trễ** (*delayed reward*) (hay phần thưởng thưa – *sparse reward*). Điều này phân biệt rõ rệt nó với học có giám sát thông thường, vốn dựa vào các tín hiệu giám sát tức thời, theo từng bước, được mã hóa sẵn từ trước. Đặc điểm phần thưởng trễ này phù hợp với nhiều bài toán NLP. Ví dụ, một hệ thống sinh văn bản đưa ra một chuỗi từ từ trái sang phải, nhưng thường khó xác định một từ đơn lẻ có được chọn đúng hay không cho đến khi toàn bộ văn bản đã được sinh ra và đánh giá.

Chú giải của người dịch: Học tăng cường qua ví dụ chơi cờ

Hãy hình dung bạn học chơi cờ tướng mà không có thầy chỉ từng nước "nước này đúng, nước kia sai". Bạn (tác tử) đi từng nước (hành động) trên bàn cờ (môi trường), và chỉ khi ván cờ kết thúc mới biết thắng (+1) hay thua (-1) — đó là phần thưởng trễ. Sau nhiều ván, bạn dần rút ra một "chiến lược" (chính sách): ở thế cờ nào thì nên đi nước nào để khả năng thắng cao nhất. Khó khăn là phải tự suy ra trong 40 nước đã đi, những nước nào thực sự góp phần vào chiến thắng. Với sinh văn bản cũng vậy: chỉ khi đọc xong cả câu trả lời ta mới đánh giá được nó tốt hay dở.

Một ví dụ khác là **học tự giám sát** (*self-supervised learning*). Nó giải quyết các bài toán học không giám sát theo cách của học có giám sát. Ý tưởng chung là phát biểu bài toán học không giám sát thành một bài toán

tiền đề (*pretext task*) có thể được dùng để giải bài toán ban đầu. Trong bài toán tiền đề, nhãn đúng có thể được tạo ra từ chính dữ liệu đầu vào. Vì vậy, việc học nhận được sự giám sát từ các tín hiệu tự tạo thay vì từ các nhãn thủ công.

Học tự giám sát đã rất thành công trong NLP. **Tiền huấn luyện** có lẽ là ứng dụng nổi bật nhất thúc đẩy những tiến bộ gần đây. Trong tiền huấn luyện, một mô hình nền (chẳng hạn một mô hình ngôn ngữ) được huấn luyện bằng học tự giám sát trên các kho ngữ liệu khổng lồ, rồi các tham số đã học của nó được áp dụng cho một hệ thống hạ nguồn (*downstream*) (chẳng hạn một mô hình phân tích cảm xúc). Cách làm này có hai ưu điểm lớn. Thứ nhất, các mô hình tự giám sát có thể được mở rộng tới lượng văn bản gần như không giới hạn vì chúng không cần gán nhãn thủ công. Thứ hai, các biểu diễn học được trong quá trình tiền huấn luyện có khả năng tổng quát hóa cao và có thể được áp dụng cho nhiều loại bài toán hạ nguồn. Trong Chương 7, chúng ta sẽ tìm hiểu một số ví dụ về việc áp dụng học tự giám sát cho các mô hình NLP hiện đại.

Chú giải của người dịch: Nhãn "tự tạo" trong học tự giám sát

Lấy câu "Hôm nay trời rất đẹp". Ta che đi một từ: "Hôm nay trời rất ___" và yêu cầu mô hình đoán từ bị che. Đáp án đúng ("đẹp") có sẵn ngay trong câu gốc, không cần ai gán nhãn. Với hàng tỉ câu trên Internet, ta tạo được hàng tỉ "bài tập" như vậy miễn phí — đó là bài toán tiền đề. Để làm tốt bài tập này, mô hình buộc phải nắm được ngữ pháp và ý nghĩa của ngôn ngữ, và chính hiểu biết đó sau này được "chuyển giao" cho các bài toán hạ nguồn như phân loại cảm xúc bình luận.

1.3.2 Thiên kiến quy nạp

Một cách không hình thức, ta mô tả học máy (có giám sát) là một quá trình **suy luận quy nạp** (*inductive reasoning*) (hay **suy diễn quy nạp** (*inductive inference*)): ta dùng các quan sát cụ thể (chẳng hạn các tài liệu có nhãn) để xây dựng một mô hình tổng quát (chẳng hạn một bộ phân loại). Ví dụ, nếu ta quan sát thấy từ *cooking* (nấu ăn) thường xuyên xuất hiện trong các tài liệu về đồ ăn, suy luận quy nạp dẫn ta tới kết luận rằng *cooking* là một dấu hiệu mạnh cho mọi tài liệu liên quan đến đồ ăn. Phương pháp "từ cụ thể đến tổng quát" này được gọi đơn giản là **quy nạp** (*induction*). Khi đã

có một mô hình được quy nạp, việc áp dụng nó để dự đoán các quan sát mới, chưa gặp là một quá trình **diễn dịch** (*deduction*).

Quy nạp là nguyên lý được dùng rộng rãi nhất khi thiết kế bộ học của các hệ thống học máy hiện đại[10]. Hãy hình dung có một không gian giả thuyết (*hypothesis space*) (hay không gian mô hình, hay không gian các hàm có thể học) gồm mọi mô hình có thể có mà ta có thể tạo ra. Khi đó, học một mô hình chính là chọn một mô hình trong không gian giả thuyết bằng cách quy nạp từ các mẫu huấn luyện đã cho. Tuy nhiên, ta không thể đơn giản giả định có một mô hình "tiên tri" hoạt động tốt trên mọi mẫu chưa gặp. Một lý do là việc tìm mô hình "tốt nhất" trong một không gian giả thuyết khổng lồ là bất khả thi về mặt tính toán. Nếu không gian giả thuyết không bị ràng buộc, sẽ có vô số chiều mà theo đó ta có thể thiết kế mô hình. Điều này về bản chất liên quan đến **lời nguyền số chiều** (*the curse of dimensionality*)[11]. Một lý do khác là nhiều mô hình trong không gian giả thuyết có thể khớp tốt với các mẫu huấn luyện, nhưng chỉ một số trong đó khớp được với các mẫu chưa gặp. Có nguy cơ ta chọn phải một mô hình ghi nhớ hoàn hảo dữ liệu huấn luyện nhưng không tổng quát hóa (*generalize*) được cho dữ liệu chưa gặp. Điều này liên quan đến **quá khớp** (*overfitting*), một khái niệm mà chúng ta sẽ bàn ở phần sau.

Một giải pháp tự nhiên là định nghĩa các tri thức tiên nghiệm (*priors*) trên không gian giả thuyết sao cho một số mô hình được ưu tiên hơn các mô hình khác. Một ví dụ đơn giản là ta giới hạn các bộ phân loại ở các mô hình tuyến tính (xem Mục 1.2.3). Điều này tương đương với việc áp đặt một tiên nghiệm loại bỏ mọi mô hình phi tuyến khỏi không gian giả thuyết.

Một tiên nghiệm như vậy được gọi là **thiên kiến quy nạp** (*inductive bias*). Tóm lại, thiên kiến quy nạp là một tập các giả định về bài toán học[12]. Chẳng hạn, ta có thể giả định một dạng toán học cụ thể cho mô hình (thiên kiến mô hình (*model bias*)), chọn một thuật toán tối ưu cụ thể (thiên kiến

[10] Không phải phương pháp học máy nào cũng tuân theo quy nạp một cách chặt chẽ. Các mô thức khác bao gồm suy luận diễn dịch (*deductive reasoning*), suy luận giả định (*abductive reasoning*) và suy luận loại suy (*analogical reasoning*, hay *transduction*) [Hurley, 2011].

[11] Lời nguyền số chiều chỉ những vấn đề thường xuất hiện khi số chiều của không gian giả thuyết tăng lên. Ví dụ, tính thưa của dữ liệu là một vấn đề phổ biến nảy sinh khi xử lý dữ liệu nhiều chiều, và do đó là một dạng của lời nguyền số chiều.

[12] Có thể tìm định nghĩa hình thức hơn trong các giáo trình học máy chuẩn [Mitchell, 1997].

thuật toán (*algorithmic bias*)), hoặc đưa ra giả định về cách dữ liệu được sinh ra (thiên kiến mẫu (*sample bias*))[13].

Các thiên kiến quy nạp cố gắng chỉ ra ta nên mô tả một bài toán theo cách nào. Nói chung, kết quả sẽ tốt hơn khi thiên kiến quy nạp phù hợp với phân phối thực sự ẩn sau dữ liệu. Điều này giải thích vì sao lời giải cho một số bài toán ưa chuộng những kiến trúc mô hình (hay thiên kiến mô hình) nhất định. Dĩ nhiên, càng nhiều và càng mạnh các thiên kiến quy nạp thì càng có thể giúp giải bài toán dễ dàng hơn. Tuy nhiên, thiên kiến quy nạp không phải lúc nào cũng có ích, đặc biệt khi chúng không gần với thực tế.

Hãy xét một trò chơi gieo xúc xắc để minh họa điều này. Giả sử bạn có một con xúc xắc 6 mặt. Trước mỗi lần gieo, bạn đoán một số từ 1 đến 6, với mục tiêu thắng càng nhiều lần càng tốt. Theo lẽ thường, đoán ngẫu nhiên là lựa chọn hợp lý duy nhất của bạn vì mọi mặt đều có cơ hội xuất hiện như nhau ($1/6$). Điều này đúng với một con xúc xắc cân đối. Tuy nhiên, hãy tưởng tượng một ngày nọ bạn chơi với một con xúc xắc bị đổ chì, và việc thắng trở nên khó khăn. Bạn nhận thấy kết quả không còn tuân theo phân phối đều nữa. Khi đó, bạn giả định kết quả tuân theo một phân phối đa thức (vì bạn đã có kinh nghiệm xây dựng các bộ phân loại Bayes ngây thơ). Trước khi đặt cược, bạn quan sát 100 lần gieo thử, chọn mặt xuất hiện nhiều nhất cho các ván sau, và lại bắt đầu thắng.

Trong ví dụ này, giả định ban đầu của bạn (tức là cả sáu mặt có khả năng như nhau) là một thiên kiến quy nạp rất mạnh vì mô hình trong đầu bạn có *không* bậc tự do nào. Nó có vẻ hiển nhiên nhưng lại thất bại với con xúc xắc đổ chì. Thiên kiến quy nạp thứ hai của bạn (phân phối đa thức), dù phức tạp hơn về mặt toán học, thực ra lại là một giả định *yếu hơn*. Nó định nghĩa một họ mô hình rộng hơn, cho bạn sự linh hoạt để tìm ra các xác suất cụ thể khớp với con xúc xắc đổ chì.

Chú giải của người dịch: Thiên kiến mạnh và thiên kiến yếu

Với giả định "xúc xắc cân đối", mô hình đã cố định hoàn toàn: $p_1 = \cdots = p_6 = 1/6$, không có gì để học (0 bậc tự do). Với giả định "phân phối đa thức", mô hình có 6 xác suất $p_1, \ldots, p_6$ chỉ cần thỏa $p_i \geq 0$ và tổng bằng 1,

[13] Cần lưu ý rằng thuật ngữ *bias* có nghĩa khác trong các ngữ cảnh khác (ví dụ, độ lệch (*bias*) trong mạng nơ-ron hay thiên kiến nhận thức (*cognitive bias*)). Chúng tôi sẽ nói rõ khi dùng với một nghĩa khác.

tức có 5 bậc tự do (biết 5 số thì suy ra số thứ 6). Ví dụ, nếu trong 100 lần gieo thử mặt 6 xuất hiện 40 lần, còn mỗi mặt khác khoảng 12 lần, ta ước lượng $p_6 \approx 0.4$ và luôn đoán 6: xác suất thắng tăng từ $1/6 \approx 0.17$ lên khoảng 0.4. Giả định "cân đối" là trường hợp riêng của giả định đa thức, nên giả định đa thức yếu hơn (ít ràng buộc hơn) nhưng bao quát hơn.

Suy cho cùng, mọi mô hình học máy đều dựa vào các thiên kiến quy nạp. Nhiều giả định trong số đó là ngầm định; đôi khi ta thậm chí không nhận ra mình đang đưa ra chúng vì chúng có vẻ hoàn toàn "hợp lý". Thế nhưng, nếu một giả định sai từ gốc, nó sẽ gây hại cho toàn bộ quá trình học. Vì vậy, kinh nghiệm thực tế vẫn là yếu tố thiết yếu để nhận ra và tránh những cạm bẫy dễ bị bỏ qua này.

1.3.3 Tính phi tuyến

Hầu hết các bài toán thực tế về bản chất là phi tuyến, khiến chúng khó giải bằng các mô hình tuyến tính đơn giản. Hình 1.3 đưa ra các ví dụ với mức độ khó phân loại khác nhau. Trong Hình 1.3 (a), hai lớp có thể được tách hoàn toàn bằng một đường thẳng (một siêu phẳng). Trong trường hợp này, bài toán là **tách được tuyến tính** (*linearly separable*) vì biên quyết định có thể được biểu diễn bằng một hàm tuyến tính. Ngược lại, trong Hình 1.3 (b), ta không thể vẽ một siêu phẳng duy nhất để tách hoàn toàn các lớp. Thay vào đó, ta cần một biên quyết định phi tuyến, chẳng hạn một siêu cầu (*hypersphere*) (một đường tròn trong không gian 2 chiều). Một trường hợp còn khó hơn được minh họa trong Hình 1.3 (c), trong đó các lớp đan xen vào nhau và biên quyết định rất phức tạp.

Mặc dù việc mô hình hóa các hệ phi tuyến làm tăng độ phức tạp tính toán, đã có một số phương pháp phổ biến để đưa tính phi tuyến vào các hệ thống học máy.

- **Ánh xạ đặc trưng** (*feature mapping*) và **phương pháp hạt nhân** (*kernel methods*). Nhắc lại rằng một bộ phân loại tuyến tính có thể được phát biểu thành hàm $f(\mathbf{w} \cdot \mathbf{x})$, trong đó $\mathbf{w}$ là vectơ trọng số, $\mathbf{x}$ là vectơ đặc trưng, và $f(\cdot)$ là hàm kích hoạt (*activation function*) trả về một lớp (chẳng hạn c_a) nếu đối số của nó lớn hơn 0 và lớp còn lại (chẳng hạn c_b) trong trường hợp ngược lại. Ý tưởng của ánh xạ đặc trưng là ánh xạ vectơ đặc trưng $\mathbf{x}$ vào một không gian có số

chiều cao hơn sao cho bài toán trở nên tách được tuyến tính trong không gian mới. Ví dụ, cho $\phi(\cdot) : \mathbb{R}^2 \to \mathbb{R}^4$ là một hàm ánh xạ và $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$ là một vectơ 2 chiều. Ta giả sử

$$\phi(\begin{bmatrix} x_1 & x_2 \end{bmatrix}) = \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix}. \tag{1.65}$$

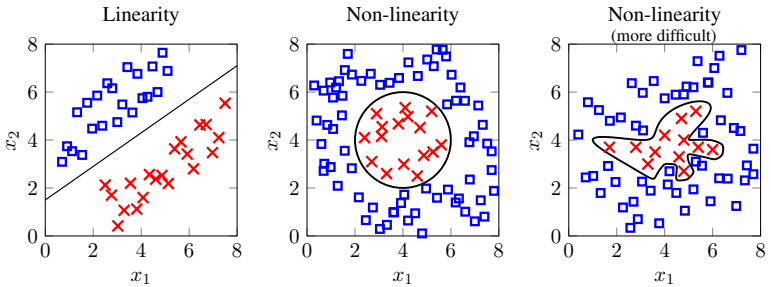
Bằng cách chọn $\mathbf{w} = \begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix}$, ta được một bộ phân loại mới:

$$\begin{aligned} f(\mathbf{w} \cdot \phi(\mathbf{x})) &= f\big(\begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix} \cdot \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix} \big) \\ &= f\big((x_1 - 4)^2 + (x_2 - 4)^2 - 4\big). \end{aligned} \tag{1.66}$$

Hàm này xác định một biên quyết định (tức là siêu cầu $(x_1 - 4)^2 + (x_2 - 4)^2 = 4$) phân loại hoàn hảo các mẫu trong Hình 1.3 (b). Nói cách khác, ta dùng một mô hình tuyến tính trên không gian đặc trưng đã ánh xạ để tạo ra một mô hình phi tuyến. Tuy nhiên, việc tính hàm ánh xạ có thể kém hiệu quả. Vấn đề này thường được giải quyết bằng các phương pháp hạt nhân. Trong các phương pháp hạt nhân, việc tính tích vô hướng của các vectơ trong không gian mới được thực hiện hiệu quả bằng cách dùng một hàm hạt nhân (*kernel function*) trong không gian cũ. Phương pháp này được gọi là **thủ thuật hạt nhân** (*the kernel trick*). Nó đã được áp dụng thành công trong phân loại và các mô hình học máy khác, chẳng hạn **máy vectơ hỗ trợ** (*support vector machines*) [Cortes và Vapnik, 1995].

Chú giải của người dịch: Kiểm tra Phương trình (1.66) bằng tay

Tích vô hướng của hai vectơ 4 phần tử là tổng các tích từng cặp phần tử: $1 \cdot (x_1^2 + x_2^2) + (-8) \cdot x_1 + (-8) \cdot x_2 + 28 \cdot 1 = x_1^2 - 8x_1 + x_2^2 - 8x_2 + 28$. Vì $(x_1 - 4)^2 = x_1^2 - 8x_1 + 16$ và $(x_2 - 4)^2 = x_2^2 - 8x_2 + 16$, ta có biểu thức trên bằng $(x_1 - 4)^2 + (x_2 - 4)^2 - 4$. Đây chính là phương trình đường tròn tâm $(4; 4)$, bán kính 2 quen thuộc ở lớp 10, chuyển vế: điểm nằm *ngoài* đường tròn cho giá trị dương (một lớp), điểm nằm *trong* cho giá trị âm (lớp kia). Ví dụ điểm $(4; 4)$ cho $-4 < 0$, điểm $(0; 0)$ cho $16 + 16 - 4 = 28 > 0$. Điều kỳ diệu là: đối với các "đặc trưng mới" $(x_1^2 + x_2^2,\ x_1,\ x_2,\ 1)$, biểu thức này *tuyến tính*, dù đối với (x_1, x_2) ban đầu nó là một đường cong.



Hình 1.3: Minh họa tính tuyến tính và phi tuyến trong phân loại nhị phân. Bài toán thứ nhất (trái) là tách được tuyến tính vì tồn tại (ít nhất) một siêu phẳng tách hoàn toàn các điểm dữ liệu thuộc hai lớp. Tính tách được tuyến tính không còn đúng ở bài toán thứ hai (giữa). Thay vào đó, ta cần một biên quyết định có dạng giống đường tròn. Biên quyết định sẽ phức tạp hơn nữa nếu có những vùng mà hai lớp điểm dữ liệu trộn lẫn vào nhau và ít nhiều không phân biệt được (phải). (Tiêu đề các đồ thị: *Linearity* = tuyến tính, *Non-linearity* = phi tuyến, *more difficult* = khó hơn.)

- **Hàm kích hoạt phi tuyến** (*non-linear activation functions*). Một cách tiếp cận chuẩn khác để đưa vào tính phi tuyến là thông qua các hàm kích hoạt. Một mô thức phổ biến là áp dụng một hàm kích hoạt phi tuyến lên đầu ra của một phép biến đổi tuyến tính. Chẳng hạn, hàm $f(\cdot)$ dùng trong ví dụ trước đóng vai trò một hàm kích hoạt phi tuyến đơn giản. Tùy theo dạng đầu ra mong muốn, có thể chọn nhiều hàm kích hoạt khác nhau. Trong các mô hình phức tạp, nhiều hàm kích hoạt phi tuyến được lồng vào các bước tính toán trung gian để tăng mạnh năng lực biểu diễn (*expressive power*) của mô hình. Ví dụ, một **mạng nơ-ron sâu** (*deep neural network*) về bản chất là một chồng các mô hình con (thường được gọi là các tầng (*layers*)), trong đó mỗi tầng áp dụng một phép biến đổi tuyến tính rồi đến một hàm kích hoạt phi tuyến.

- **Phương pháp phi tham số** (*non-parametric methods*). Thuật ngữ *phi tham số* bắt nguồn từ thống kê. Trong thống kê phi tham số, các suy luận thống kê được thực hiện mà không đưa ra bất kỳ giả định nào về phân phối ẩn sau dữ liệu. Trong học máy, các phương pháp phi tham số theo cùng ý tưởng đó. Chúng không giả định bất kỳ hàm

ánh xạ nào từ đầu vào sang đầu ra như Phương trình (1.66). Điều này phân biệt chúng với các **phương pháp có tham số** (*parametric methods*), vốn học tường minh một dạng toán học của các biến (hay tham số) để mô tả bài toán. Một ví dụ về phương pháp phi tham số là k **láng giềng gần nhất** (k-*nearest neighbors*), phương pháp phân loại một mẫu mới dựa trên biểu quyết đa số của k mẫu huấn luyện gần nó nhất. Lưu ý rằng phi tham số không có nghĩa là không có tham số. Đúng hơn, nó có nghĩa là số lượng tham số không được cố định trước. Độ phức tạp của mô hình tăng lên một cách linh động khi có thêm dữ liệu huấn luyện. Bù lại, các phương pháp này có thể xử lý những bài toán phi tuyến cao khi có đủ mẫu huấn luyện.

Chú giải của người dịch: Phương pháp k láng giềng gần nhất

Muốn biết một học sinh mới chuyển đến thích đá bóng hay cầu lông, ta xem $k = 5$ bạn ở gần nhà em ấy nhất: nếu 3 bạn thích đá bóng, 2 bạn thích cầu lông, ta đoán em ấy thích đá bóng. Trong Hình 1.3, với một điểm mới, ta tìm k điểm huấn luyện gần nhất (theo khoảng cách trên mặt phẳng) và lấy nhãn chiếm đa số. Phương pháp này không cần công thức biên quyết định nào cả, nên vẫn xử lý được cả hình (c) phức tạp. "Tham số" của nó thực chất là toàn bộ dữ liệu huấn luyện được lưu lại, nên càng có nhiều dữ liệu, "mô hình" càng lớn.

Dù vậy, các phương pháp phi tuyến không hoạt động một mình. Tính tuyến tính là một thành phần quan trọng của hầu hết các hệ thống học máy thực tế. Điều này thể hiện ở hai khía cạnh. Thứ nhất, nhiều tính phi tuyến hơn không phải lúc nào cũng tốt hơn. Nếu một mô hình tuyến tính đã giải quyết bài toán đủ tốt thì việc thêm độ phức tạp phi tuyến là không cần thiết và có nguy cơ gây quá khớp. Phù hợp với nguyên lý **dao cạo Occam** (*Occam's Razor*) — *lời giải đơn giản nhất gần như luôn là lời giải tốt nhất* — hầu hết các mô hình tiên tiến nhất (*state-of-the-art*) đều duy trì sự cân bằng giữa các thành phần tuyến tính và phi tuyến. Thứ hai, các mô hình tuyến tính là những phép xấp xỉ tuyệt vời cho các hành vi phi tuyến. Nếu tính phi tuyến của một bài toán là nhẹ, việc dùng các hàm tuyến tính để xấp xỉ lời giải thường hiệu quả hơn đáng kể và dễ diễn giải hơn cho các mục đích kỹ thuật thực tế.

1.3.4 Huấn luyện và hàm mất mát

Hầu như mọi thuật toán học máy đều có một bước huấn luyện (*training*). Huấn luyện thường chỉ quá trình ước lượng hàm ánh xạ và các tham số đi kèm của nó từ dữ liệu. Ở đây, chúng ta theo một định nghĩa thông dụng của bài toán huấn luyện: cho trước một mô hình hay hàm ánh xạ, ta tối ưu một mục tiêu cụ thể bằng cách đánh giá mô hình qua một *kinh nghiệm huấn luyện* nào đó [Mitchell, 1997]. Ví dụ, huấn luyện một bộ phân loại văn bản Bayes ngây thơ đòi hỏi cực đại hóa một hàm hợp lý (mục tiêu) trên một tập các tài liệu có nhãn (kinh nghiệm huấn luyện).

Thường thì bài toán huấn luyện có thể được phát biểu thành một bài toán **tối ưu hóa** (*optimization*). Theo đó, ta tối ưu một **hàm mục tiêu** (*objective function*) thông qua một thuật toán huấn luyện. Mặc dù mục tiêu lý tưởng là thước đo hiệu năng thực tế trên các mẫu kiểm tra, ta không thể dùng trực tiếp nó để tối ưu, vì các mẫu kiểm tra và nhãn tương ứng của chúng không thể tiếp cận được trong giai đoạn huấn luyện. Thay vào đó, các hàm mục tiêu thực tế được định nghĩa như những đại lượng thay thế tính được cho thước đo dùng lúc kiểm tra. Các hàm mục tiêu này không nhất thiết là thước đo hiệu năng chính xác, mà là những đại lượng được giả định là có tương quan với hiệu năng trên dữ liệu kiểm tra.

Hãy xét một trường hợp tổng quát. Giả sử $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ là một mô hình nhận vectơ đặc trưng $\mathbf{x}$ làm đầu vào và cho ra một vectơ đầu ra n chiều $\mathbf{y}_\theta$. Ví dụ, trong phân loại văn bản, $\mathbf{x}$ là biểu diễn túi từ của một tài liệu, còn $\mathbf{y}_\theta$ là một phân phối xác suất trên một tập các lớp. Ở đây, θ biểu diễn các tham số của mô hình, và chỉ số dưới nhấn mạnh rằng hành vi của mô hình được quyết định bởi θ. Ta giả sử thêm $\mathbf{y}_{\text{gold}}$ là vectơ nhãn đúng (hay nhãn chuẩn vàng). Khi đó ta định nghĩa một **hàm mất mát** (*loss function*), ký hiệu là $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$, để đo mức độ sai khác hay sai số giữa dự đoán $\mathbf{y}_\theta$ và nhãn đúng $\mathbf{y}_{\text{gold}}$. Nó "phạt" mô hình khi dự đoán không chính xác. Với một kiến trúc mô hình cho trước, bài toán huấn luyện được mô tả là tìm các tham số "tốt nhất" $\hat{\theta}$ sao cho mất mát $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ nhỏ nhất:

$$\hat{\theta} \;=\; \arg\min_{\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}). \tag{1.67}$$

Công thức này có thể dễ dàng mở rộng cho trường hợp có K mẫu huấn

luyện:

$$\hat{\theta} \;=\; \arg\min_{\theta} \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}(\mathbf{y}_{\theta}^{(k)}, \mathbf{y}_{\text{gold}}^{(k)}). \tag{1.68}$$

Khi đã có $\hat{\theta}$, ta có thể dùng $f_{\hat{\theta}}(\mathbf{x})$ như một mô hình cố định cho các dự đoán sau này.

Chú giải của người dịch: Đọc Phương trình (1.67–1.68)

$\arg\min_{\theta} g(\theta)$ nghĩa là "giá trị của θ làm cho $g(\theta)$ nhỏ nhất" (khác với $\min_{\theta} g(\theta)$ là chính giá trị nhỏ nhất đó). Ví dụ, với $g(\theta) = (\theta - 3)^2 + 1$ thì $\min g = 1$ còn $\arg\min g = 3$. Phương trình (1.68) lấy *trung bình* mất mát trên cả K mẫu huấn luyện, rồi tìm bộ tham số làm trung bình đó nhỏ nhất — giống như chọn cách học sao cho tổng số lỗi trên toàn bộ đề luyện tập là ít nhất, chứ không chỉ đúng một bài. Dấu mũ trong $\hat{\theta}$ là ký hiệu thông dụng cho "giá trị ước lượng được".

$\mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}})$ thường được gọi là **hàm mất mát** (tính trên từng mẫu), còn giá trị trung bình của nó trên toàn bộ tập dữ liệu, $\frac{1}{K}\sum_{k=1}^{K} \mathcal{L}(\mathbf{y}_{\theta}^{(k)}, \mathbf{y}_{\text{gold}}^{(k)})$, thường được gọi là **hàm chi phí** (*cost function*) hay **rủi ro thực nghiệm** (*empirical risk*). Một hàm mất mát có thể được định nghĩa theo nhiều cách, tùy thuộc vào bài toán ta giải và những tiên nghiệm ta muốn áp đặt lên quá trình huấn luyện. Ở đây, trước hết ta xét trường hợp $\mathbf{y}_{\theta}$ là một phân phối xác suất. Trường hợp này khá phổ biến trong NLP: ví dụ, $\mathbf{y}_{\theta}$ có thể là một phân phối trên bộ từ vựng, trên một danh sách tài liệu, hay trên một tập nhãn cú pháp. Với loại đầu ra mô hình này, các hàm mất mát được dùng phổ biến nhất là các độ đo độ phân kỳ (*divergence*):

- **Mất mát dựa trên độ phân kỳ** (*divergence-based loss*). Các hàm mất mát dựa trên độ phân kỳ đo sự khác biệt giữa hai phân phối $\mathbf{y}_{\theta}$ và $\mathbf{y}_{\text{gold}}$. Ví dụ, entropy chéo (*cross-entropy*) (xem Mục 1.1.2) là một trong những hàm mất mát phổ biến nhất trong NLP. Ta cũng có thể chọn các độ đo dựa trên độ phân kỳ khác, chẳng hạn độ phân kỳ KL và độ phân kỳ Jensen–Shannon (JS), có thể tìm thấy trong hầu hết các giáo trình thống kê. Lưu ý rằng mục tiêu của **ước lượng hợp lý cực đại** (*maximum likelihood estimation*) (**MLE**) gắn liền về bản chất với các mục tiêu dựa trên độ phân kỳ. Cụ thể, cực tiểu hóa log-hợp lý âm (mục tiêu MLE chuẩn) tương đương về mặt toán học với cực tiểu hóa mất mát entropy chéo khi $\mathbf{y}_{\text{gold}}$ là một vectơ one-hot

(*one-hot vector*) (vectơ trong đó phần tử ứng với nhãn đúng bằng 1 và mọi phần tử khác bằng 0).

Chú giải của người dịch: Vì sao entropy chéo với nhãn one-hot chính là log-hợp lý âm?

Giả sử có 3 lớp (Thể thao, Đồ ăn, Âm nhạc), nhãn đúng là "Đồ ăn" nên $\mathbf{y}_{\text{gold}} = [0 \;\; 1 \;\; 0]$, còn mô hình dự đoán $\mathbf{y}_\theta = [0.2 \;\; 0.7 \;\; 0.1]$. Entropy chéo là $-\sum_i y_{\text{gold}}(i) \log y_\theta(i) = -(0 \cdot \log 0.2 + 1 \cdot \log 0.7 + 0 \cdot \log 0.1) = -\log 0.7 \approx 0.36$ (logarit tự nhiên). Mọi số hạng có hệ số 0 đều biến mất, chỉ còn $-\log$ của xác suất mô hình gán cho nhãn đúng — đó đúng là log-hợp lý âm. Nếu mô hình tự tin hơn (0.9) thì mất mát giảm còn $-\log 0.9 \approx 0.11$; nếu chỉ gán 0.1 thì mất mát tăng tới ≈ 2.30.

Tuy nhiên, các hệ thống học máy không phải lúc nào cũng bị giới hạn ở đầu ra dạng phân phối. Thay vào đó, $\mathbf{y}_\theta$ có thể đơn giản là một vectơ trong $\mathbb{R}^n$. Một ví dụ là các hàm phân biệt dùng trong phân loại (xem Mục 1.2.3). Các hàm này gán một điểm số cho mỗi lớp, thể hiện mức độ mô hình tin rằng một lớp cụ thể là câu trả lời đúng. Một cách định nghĩa hàm mất mát cho các vectơ giá trị thực là biến đổi chúng về dạng giống phân phối[14] rồi dùng các mất mát dựa trên độ phân kỳ. Tuy nhiên, việc chuẩn hóa không phải lúc nào cũng cần thiết, đặc biệt khi ta cần các điểm số nằm ngoài khoảng $[0, 1]$. Phổ biến hơn là tính mất mát trực tiếp trên đầu ra thô của các mô hình này. Dưới đây là một số ví dụ.

- **Mất mát dựa trên khoảng cách** (*distance-based loss*). Một cách tự nhiên là phát biểu mất mát như một khoảng cách hình học. Một ví dụ tổng quát là khoảng cách theo chuẩn p (xem Mục 1.1.1):

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \Big(\sum_{i=1}^{n} |y_\theta(i) - y_{\text{gold}}(i)|^p \Big)^{1/p}. \tag{1.69}$$

  Ví dụ, đặt $p = 2$ cho ta khoảng cách Euclid thông thường. Về bản chất, các mất mát dựa trên khoảng cách mô tả một bài toán **khớp đường cong** (*curve fitting*): ta học một hàm $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ để khớp với một tập điểm mục tiêu $\{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}_{\text{gold}})\}$. Cách tiếp cận này là điển hình trong các bài toán **hồi quy** (*regression*)[15]. Một ví dụ cụ thể là

[14] Ví dụ, bằng cách chia các phần tử của vectơ cho tổng của tất cả các phần tử.

[15] Khi đầu ra của mô hình là một vectơ có từ hai chiều trở lên, bài toán được gọi là **hồi quy đa biến** (*multivariate regression*).

ước lượng chất lượng (*quality estimation*) của dịch máy[16]. Mô hình học cách dự đoán một điểm chất lượng bản dịch liên tục (y_θ) cho một cặp câu nguồn–đích ($\mathbf{x}$). Dự đoán được coi là chính xác nếu nó khớp sát với đánh giá của con người. Dựa trên Phương trình (1.69), **sai số bình phương trung bình** (*mean squared error*) (**MSE**) là một hàm mất mát hồi quy rất phổ biến. Nó bỏ đi phép khai căn bậc $1/p$ và chỉ đơn giản tính tổng bình phương khoảng cách Euclid:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_{i=1}^{n} |y_\theta(i) - y_{\text{gold}}(i)|^2. \tag{1.70}$$

Một ví dụ khác là **sai số tuyệt đối trung bình** (*mean absolute error*) (**MAE**). Nó chính xác là dạng của Phương trình (1.69) khi $p = 1$.

Chú giải của người dịch: Ví dụ số cho MSE và MAE

Giả sử mô hình dự đoán điểm chất lượng của 3 bản dịch là $\mathbf{y}_\theta = [7 \;\; 5 \;\; 9]$, còn người chấm cho $\mathbf{y}_{\text{gold}} = [8 \;\; 5 \;\; 6]$. Hiệu từng phần tử: $-1,\ 0,\ 3$. Theo Phương trình (1.70): $1 + 0 + 9 = 10$. Theo dạng $p = 1$: $1+0+3 = 4$. Theo $p = 2$ (khoảng cách Euclid): $\sqrt{10} \approx 3.16$. Để ý rằng bình phương "phạt" sai số lớn (3) nặng hơn nhiều so với sai số nhỏ (1). (Trong thực hành, MSE và MAE thường còn được chia cho số phần tử n để lấy trung bình, như tên gọi "trung bình"; điều này không làm thay đổi tham số tối ưu.)

- **Mất mát 0-1** (*0-1 loss*). Mất mát 0-1 được dùng rộng rãi trong các bài toán phân loại. Nó gán mức phạt 1 cho một dự đoán sai và 0 cho một dự đoán đúng. Gọi $c_\theta = \arg\max_c y_\theta(c)$ là nhãn dự đoán, được chọn bằng cách tìm giá trị lớn nhất trong $\mathbf{y}_\theta$. Tương tự, gọi $c_{\text{gold}} = \arg\max_c y_{\text{gold}}(c)$ là nhãn đúng. Mất mát 0-1 được định nghĩa hình thức như sau:

$$\begin{aligned} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \mathcal{L}_{\text{0-1}}(c_\theta, c_{\text{gold}}) \\ &= \begin{cases} 1 & c_\theta \neq c_{\text{gold}} \\ 0 & c_\theta = c_{\text{gold}} \end{cases}. \end{aligned} \tag{1.71}$$

[16]Trong dịch máy, ước lượng chất lượng gồm nhiều bài toán con. Ở đây, chúng tôi dùng thuật ngữ này để chỉ việc dự đoán trực tiếp một điểm đánh giá.

- **Mất mát dựa trên lề** (*margin-based loss*). Một **lề** (*margin*) biểu diễn hiệu số giữa điểm số dự đoán của nhãn đúng c_{gold} và của một nhãn sai c:

$$\text{margin}(c, c_{\text{gold}}) \;=\; y_\theta(c_{\text{gold}}) - y_\theta(c). \tag{1.72}$$

Nó thể hiện mức độ phân biệt giữa c_{gold} và c. Vì vậy, một ý tưởng tự nhiên là đảm bảo lề đủ lớn, hoặc ít nhất vượt quá một mức tối thiểu. Mô thức này được gọi là **huấn luyện lề lớn** (*large-margin training*). Gọi $\triangle(c, c_{\text{gold}})$ là chi phí phạt được định nghĩa trước khi phân loại nhầm c_{gold} thành c, thỏa mãn $\triangle(c, c_{\text{gold}}) \geq 0$, và $\triangle(c, c_{\text{gold}}) = 0$ khi và chỉ khi $c = c_{\text{gold}}$. Mục tiêu của ta là cực đại hóa biểu thức $\text{margin}(c, c_{\text{gold}}) - \triangle(c, c_{\text{gold}})$; tương đương, hàm mất mát cần phạt những trường hợp giá trị này nhỏ hoặc âm. Mất mát dựa trên lề được cho bởi:

$$\begin{aligned}
\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \max\Big(0, \max_c -\big(\text{margin}(c, c_{\text{gold}}) - \triangle(c, c_{\text{gold}})\big)\Big) \\
&= \max\Big(0, \max_c \big(y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\big)\Big) \\
&= \max_c \Big(0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\Big).
\end{aligned} \tag{1.73}$$

Việc thiết kế $\triangle(c, c_{\text{gold}})$ phụ thuộc vào bài toán đích. Một lựa chọn đơn giản là $\triangle(c, c_{\text{gold}}) = 1$ với $c \neq c_{\text{gold}}$. Cách phát biểu này khiến Phương trình (1.73) trở thành một biến thể của **mất mát bản lề** (*hinge loss*). Một phương án khác là dùng tổng thay cho giá trị lớn nhất trên mọi lớp c:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_c \max\Big(0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \triangle(c, c_{\text{gold}})\Big). \tag{1.74}$$

Chú giải của người dịch: Ví dụ tính mất mát bản lề

Ba lớp với điểm số mô hình y_θ = (Thể thao : 2, Đồ ăn : 3, Âm nhạc : 2.5), nhãn đúng là "Đồ ăn", và $\triangle = 1$ cho mọi lớp sai. Với "Thể thao": $2 - 3 + 1 = 0$; với "Âm nhạc": $2.5 - 3 + 1 = 0.5$; với chính "Đồ ăn": $3 - 3 + 0 = 0$. Theo Phương trình (1.73), mất mát = $\max(0; 0; 0.5) = 0.5$; theo Phương trình (1.74), mất mát = $0 + 0.5 + 0 = 0.5$ (ở đây tình cờ bằng nhau). Dù mô hình đã đoán

đúng ("Đồ ăn" có điểm cao nhất), mất mát vẫn dương vì "Âm nhạc" chỉ kém 0.5 điểm, chưa đạt lề yêu cầu là 1. Nếu điểm "Đồ ăn" tăng lên 3.5 thì mất mát về 0: mô hình không chỉ đúng mà còn đúng một cách "chắc chắn".

- **Mất mát dựa trên xếp hạng** (*ranking-based loss*). Mất mát dựa trên xếp hạng (hay mất mát xếp hạng (*ranking loss*)) được dùng trong nhiều lĩnh vực khác nhau, chẳng hạn truy xuất thông tin (*information retrieval*), phân loại và học độ đo (*metric learning*). Nó giải quyết bài toán trong đó ta muốn sắp thứ tự một tập các mục đã được chấm điểm. Giả sử đầu ra $\mathbf{y}_\theta$ của mô hình đánh giá n mục $\{c_i\}$. Ta định nghĩa $\{\psi_\theta(c_i)\}$ là thứ hạng dự đoán của $\{c_i\}$ khi sắp xếp theo điểm số $\{y_\theta(i)\}$ của chúng. Ví dụ, với điểm số $\{y_\theta(i)\} = \{0.3, -2, 1\}$, các thứ hạng sẽ là $\psi_\theta(c_1) = 2, \psi_\theta(c_2) = 3$ và $\psi_\theta(c_3) = 1$. Tương tự, ta định nghĩa $\{\psi_{\text{gold}}(c_i)\}$ là các thứ hạng chuẩn vàng. Lưu ý rằng nếu một bài toán chỉ đòi hỏi thứ tự tương đối thay vì điểm số tuyệt đối, $\{\psi_{\text{gold}}(c_i)\}$ có thể được định nghĩa mà không cần các giá trị đích $\mathbf{y}_{\text{gold}}$. Ý tưởng là phạt những chỗ đảo thứ hạng trong $\{\psi_\theta(c_i)\}$ so với $\{\psi_{\text{gold}}(c_i)\}$. Một phương pháp đơn giản là phạt các cặp mục bị sắp sai thứ tự, cùng chung ý tưởng với phân loại nhị phân. Gọi Ω là tập các cặp được sắp đúng thứ tự theo nhãn đúng:

$$\Omega \;=\; \{(i,j) \mid \psi_{\text{gold}}(i) < \psi_{\text{gold}}(j)\}. \tag{1.75}$$

Hàm mất mát cộng gộp các mức phạt trên những cặp này:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \sum_{(i,j)\in\Omega} \mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)), \tag{1.76}$$

trong đó $\mathcal{L}_{\text{pair}}$ là một hàm mất mát phân loại theo cặp, chẳng hạn mất mát bản lề được dùng trong [Collobert và Weston, 2008]:

$$\mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)) \;=\; \max\big(0, y_\theta(i) - y_\theta(j) + 1\big). \tag{1.77}$$

Cách tiếp cận này được gọi là **phương pháp theo cặp** (*pairwise method*). Mất mát xếp hạng cũng có thể được định nghĩa theo kiểu theo từng điểm (*pointwise*) hoặc theo danh sách (*listwise*), vốn được dùng rộng rãi trong các hệ thống học xếp hạng (*learning-to-rank*) hiện đại.

Chú giải của người dịch: Thứ hạng và cặp sắp đúng thứ tự

Thứ hạng 1 là mục có điểm cao nhất: với điểm $\{0.3, -2, 1\}$, mục c_3 (điểm 1) đứng đầu, c_1 (0.3) thứ hai, c_2 (-2) cuối. Giả sử thứ hạng đúng là c_1 nhất, c_3 nhì, c_2 ba. Khi đó Ω gồm 3 cặp: $(1, 3)$, $(1, 2)$, $(3, 2)$ — cặp (i, j) nghĩa là "i phải xếp trên j". Mô hình đã xếp đúng cặp $(1, 2)$ và $(3, 2)$ nhưng đảo cặp $(1, 3)$ (vì c_3 được xếp trên c_1). Hàm mất mát theo cặp chỉ cần nhìn từng cặp như một bài toán "ai đứng trên?" nhỏ, giống phân loại nhị phân, rồi cộng các mức phạt lại.

- **Mất mát tương phản** (*contrastive loss*). Mất mát tương phản thường được dùng trong **học tương phản** (*contrastive learning*). Nó giả định rằng, với một mẫu cho trước, có một mẫu tương tự được gán nhãn "dương" (*positive*), và có một số mẫu không tương tự được gán nhãn "âm" (*negative*). Một ý tưởng tự nhiên là cực tiểu hóa khoảng cách giữa các mẫu tương tự, đồng thời cực đại hóa khoảng cách giữa các mẫu không tương tự. Quay lại cách phát biểu của chúng ta, gọi $\mathbf{y}^+$ là đầu ra dương và $\mathbf{Y}^- = \{\mathbf{y}^-\}$ là tập các đầu ra âm. Đồng thời, ta dùng $\mathbf{y}_{\text{gold}}$ để ký hiệu bộ gồm $\mathbf{y}^+$ và $\mathbf{Y}^-$ thay vì một vectơ chuẩn vàng duy nhất. Một dạng của hàm mất mát tương phản được cho bởi phương trình:

$$\begin{aligned} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}^+, \mathbf{Y}^-) \\ &= \log D(\mathbf{y}_\theta, \mathbf{y}^+) - \log \sum_{\mathbf{y}^- \in \mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-) \\ &= \log \frac{D(\mathbf{y}_\theta, \mathbf{y}^+)}{\sum_{\mathbf{y}^- \in \mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-)}, \end{aligned} \tag{1.78}$$

trong đó $D(\alpha, \beta)$ biểu diễn một độ đo khoảng cách (ví dụ khoảng cách Euclid) giữa α và β. Một thách thức trong thiết kế là tạo ra các đầu ra dương và âm này. Trong thiết lập có giám sát, vectơ nhãn đúng đương nhiên đóng vai trò đích dương, còn các mẫu âm có thể được lấy từ các lớp khác. Trong thiết lập không giám sát, $\mathbf{y}^+$ và $\mathbf{Y}^-$ thường được định nghĩa theo kinh nghiệm. Ví dụ, trong một khuôn khổ **tự mã hóa** (*auto-encoding*), $\mathbf{y}^+$ có thể đơn giản là chính đầu vào $\mathbf{x}$ (hoặc một phiên bản tăng cường (*augmented*) của nó), còn $\mathbf{Y}^-$ là một tập các mẫu được sinh ngẫu nhiên.

Chú giải của người dịch: Trực giác của mất mát tương phản

Hãy nghĩ tới việc học nhận mặt: ảnh chân dung của bạn Lan (đầu ra mô hình) cần ở "gần" một ảnh khác của chính Lan (mẫu dương) và ở "xa" ảnh của các bạn khác (mẫu âm). Trong Phương trình (1.78), tử số là khoảng cách tới mẫu dương (muốn nhỏ), mẫu số là tổng khoảng cách tới các mẫu âm (muốn lớn). Tỉ số càng nhỏ thì log của nó càng nhỏ (càng âm), nên cực tiểu hóa mất mát đồng thời kéo mẫu dương lại gần và đẩy các mẫu âm ra xa.

- **Mất mát dựa trên lỗi** (*error-based loss*). Các thước đo đánh giá, vốn thường dùng để định lượng lỗi của hệ thống, cũng có thể được dùng làm hàm mất mát. Ví dụ, trong dịch máy, BLEU là một thước đo thường dùng (càng cao càng tốt, nằm trong khoảng từ 0 đến 1). Ta có thể định nghĩa mục tiêu là cực tiểu hóa $1 - \text{BLEU}$. Gọi $\text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ là lỗi tính theo thước đo. Mất mát dựa trên lỗi đơn giản là giá trị này:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \;=\; \text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}). \tag{1.79}$$

Cho đến đây, chúng ta đã trình bày một số hàm mất mát cho nhiều loại bài toán khác nhau, như phân loại, hồi quy và xếp hạng. Như sẽ thấy trong cuốn sách này, các hàm mất mát khác nhau có ảnh hưởng khác nhau lên hành vi của mô hình. Tuy nhiên, thử nghiệm mọi hàm mất mát có thể có là không khả thi về mặt tính toán. Thay vào đó, người làm thực tế phải cẩn thận lựa chọn hoặc thiết kế hàm mục tiêu phù hợp nhất với bài toán cụ thể của mình. Quá trình này đòi hỏi thời gian và chuyên môn về lĩnh vực nhưng là thiết yếu.

Mặt khác, có những phương pháp chung để cải thiện việc thiết kế hàm mất mát. Ví dụ, thay vì coi đầu ra $\mathbf{y}_\theta$ của mô hình là một vectơ tất định, ta có thể mô hình hóa nó như một **biến ngẫu nhiên** (*random variable*) được rút ra từ một phân phối xác suất. Kết quả là, mất mát riêng lẻ $L(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ cũng trở thành một biến ngẫu nhiên. Trong khuôn khổ này, ta định nghĩa lại mục tiêu là **mất mát kỳ vọng** (*expected loss*) (hay rủi ro kỳ vọng) trên phân phối của $\mathbf{y}_\theta$:

$$\mathcal{L}(\mathbf{x}, \mathbf{y}_{\text{gold}}) = \mathbb{E}_{\mathbf{y}_\theta \sim \Pr(\cdot|\mathbf{x})}\left[\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})\right]$$

$$= \sum_{\mathbf{y}_\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \cdot \Pr(\mathbf{y}_\theta|\mathbf{x}). \tag{1.80}$$

Bằng cách lấy tổng (tích phân) trên toàn bộ không gian các dự đoán có thể có $\mathbf{y}_\theta$, phương pháp này cho một ước lượng mất mát vững hơn. Mô thức này bắt nguồn từ **cách tiếp cận Bayes** (*the Bayesian approach*), và $\mathcal{L}(\cdot)$ (cũng được ký hiệu là $\mathbb{L}(\cdot)$) thường được gọi là **rủi ro Bayes** (*Bayesian risk*) hay đơn giản là **rủi ro kỳ vọng** (*expected risk*).

Chú giải của người dịch: Ví dụ tính mất mát kỳ vọng

Giả sử với một đầu vào $\mathbf{x}$, mô hình có thể cho ra 3 câu trả lời khác nhau với xác suất 0.5, 0.3, 0.2, và mất mát tương ứng của từng câu là 0, 1, 4. Mất mát kỳ vọng là $0 \cdot 0.5 + 1 \cdot 0.3 + 4 \cdot 0.2 = 1.1$ — chính là công thức kỳ vọng "giá trị nhân xác suất rồi cộng lại" quen thuộc trong bài toán trò chơi may rủi ở THPT. Thay vì chỉ chấm một câu trả lời duy nhất, ta xét mọi câu trả lời mà mô hình có thể đưa ra, mỗi câu được tính theo mức độ mô hình "tin" vào nó.

Một cách khác để cải thiện việc huấn luyện là đưa tiên nghiệm vào mục tiêu. Một phương pháp điển hình là thêm một số hạng chính quy hóa (*regularization*) R vào mục tiêu, như sau

$$\hat{\theta} \;=\; \arg\min_{\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) + \alpha \cdot R \tag{1.81}$$

trong đó $R(\theta)$ là một hàm phạt một số khía cạnh của độ phức tạp mô hình, chẳng hạn độ lớn hoặc số lượng tham số khác 0. Hệ số α là một siêu tham số (*hyperparameter*) quy định độ mạnh của chính quy hóa, giúp cân bằng giữa việc khớp dữ liệu huấn luyện và việc giữ cho mô hình đơn giản. Bản thân việc thiết kế R là một lĩnh vực nghiên cứu quan trọng trong các hệ thống học máy thực tế. Chúng ta sẽ tìm hiểu một số kỹ thuật chính quy hóa cụ thể ở các phần sau của cuốn sách.

Chú giải của người dịch: Số hạng chính quy hóa hoạt động thế nào?

Một lựa chọn phổ biến là $R(\theta) = \sum_i \theta_i^2$ (tổng bình phương các tham số). Giả sử hai bộ tham số cho mất mát gần bằng nhau trên dữ liệu huấn luyện: $\theta_A = (1;\ 2)$ với mất mát 0.50 và $\theta_B = (10;\ -9)$ với mất mát 0.48. Với $\alpha = 0.01$: mục tiêu của A là $0.50 + 0.01 \cdot 5 = 0.55$, của B là $0.48 + 0.01 \cdot 181 = 2.29$.

Vậy ta chọn A — mô hình "nhẹ nhàng" hơn, ít có nguy cơ học thuộc lòng dữ liệu. α càng lớn thì càng ưu tiên sự đơn giản; $\alpha = 0$ thì quay lại Phương trình (1.67). Gọi là *siêu* tham số vì α không được học cùng θ mà do con người chọn trước.

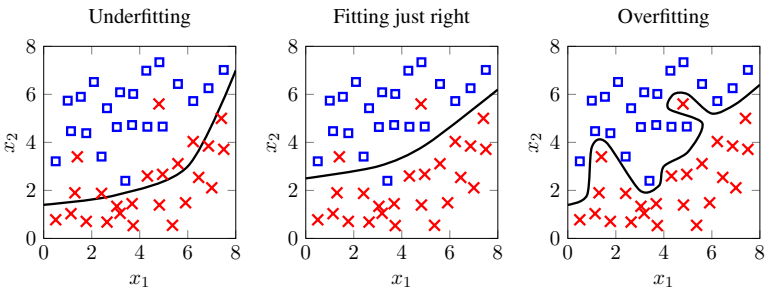
Khi đã định nghĩa được hàm mục tiêu, ta cần một thuật toán tối ưu để tìm các tham số tối ưu. Tối ưu hóa là một lĩnh vực rộng lớn trong học máy. Vì vậy, một thảo luận chi tiết về các thuật toán cụ thể nằm ngoài phạm vi của chương này. Rốt cuộc, không có một thuật toán vạn năng nào có thể giải hiệu quả mọi bài toán huấn luyện (một khái niệm liên quan đến định lý "Không có bữa trưa miễn phí" – *No Free Lunch*). Thay vào đó, các thuật toán cụ thể được lựa chọn dựa trên tính chất toán học của hàm mục tiêu. Ví dụ, ta có thể dùng *hạ gradient (gradient descent)* để huấn luyện một mô hình ngôn ngữ nơ-ron với mất mát entropy chéo [Bengio và cộng sự, 2003a], dùng *quy hoạch toàn phương (quadratic programming)* để huấn luyện một SVM với mất mát bản lề [Cortes và Vapnik, 1995], hoặc dùng **huấn luyện tỉ lệ lỗi tối thiểu** (*minimum error-rate training*) (**MERT**) để tối ưu một mô hình dịch máy thống kê với mất mát $1 - \text{BLEU}$ không khả vi [Och và Ney, 2002].

1.3.5 Quá khớp và chưa khớp

Quy trình chuẩn của học máy (có giám sát) gồm một bước huấn luyện và một bước kiểm tra. Trong khi ta có thể cố gắng cực tiểu hóa mất mát trên các mẫu huấn luyện, mô hình học được lại được dùng để xử lý những mẫu mới chưa từng gặp trước đó. Điều này tương tự với một trải nghiệm quen thuộc trong cuộc sống: một học sinh ôn thi cuối kỳ. Dù *học chăm chỉ = điểm cao* nói chung là đúng, việc học thuộc lòng câu hỏi và đáp án trong sách giáo khoa là một chiến lược tồi cho một kỳ thi có những câu hỏi mới. Câu hỏi thi chắc chắn sẽ khác với những ví dụ cụ thể đã ôn. Do đó, ta cần khả năng tổng quát hóa.

Trong học máy, *tổng quát hóa* được dùng để mô tả mức độ mà một mô hình học từ kinh nghiệm dự đoán tốt trên dữ liệu mới. Một hệ thống được coi là có hiệu năng tổng quát hóa xuất sắc nếu, với sự dẫn dắt của các thiên kiến quy nạp phù hợp, nó trích xuất thành công các khuôn mẫu ẩn sau dữ liệu huấn luyện mà không ghi nhớ nhiễu. Tuy nhiên, đạt được

khả năng tổng quát hóa tốt không có nghĩa là huấn luyện ít đi. Trái lại, người làm thực tế muốn huấn luyện mô hình học máy trên nhiều dữ liệu huấn luyện hơn để ngăn nó ghi nhớ mọi thứ. Tổng quát hóa là một vấn đề rất phức tạp, được quyết định bởi nhiều yếu tố, bao gồm độ phức tạp của bài toán, kiến trúc mô hình, lượng dữ liệu huấn luyện, thuật toán huấn luyện, v.v. Dù không có quy tắc chuẩn nào bảo đảm khả năng tổng quát hóa tốt, các nhà nghiên cứu luôn cố gắng giải quyết nó bằng cách này hay cách khác.



Hình 1.4: Biên quyết định của một bài toán phân loại nhị phân. Trái = chưa khớp, phải = quá khớp, giữa = khớp vừa đủ. Trong trường hợp chưa khớp, có một số lỗi hiển nhiên khi tách hai lớp điểm dữ liệu. Bằng cách dịch biên quyết định lên trên một chút (giữa), ta thu được kết quả tách thỏa đáng, trong đó hầu hết các điểm dữ liệu thuộc cùng một lớp nằm ở cùng một phía của biên quyết định. Ngược lại, để tách hoàn hảo thì cần một biên quyết định rất phức tạp (phải). (Tiêu đề các đồ thị: *Underfitting* = chưa khớp, *Fitting just right* = khớp vừa đủ, *Overfitting* = quá khớp.)

Để mô tả mức độ một mô hình tổng quát hóa tốt cho dữ liệu mới, có hai khái niệm quan trọng: **chưa khớp** (*underfitting*) và quá khớp. Chưa khớp xảy ra khi một mô hình không học được đầy đủ các khuôn mẫu ẩn trong dữ liệu huấn luyện, dẫn đến hiệu năng kém trên cả tập huấn luyện lẫn tập kiểm tra. Điều này thường xảy ra nếu quá trình huấn luyện bị dừng quá sớm, hoặc đơn giản là năng lực (*capacity*) của mô hình quá thấp so với bài toán. Nếu một mô hình bị chưa khớp, ta có thể cải thiện nó bằng cách tăng thời gian huấn luyện, thiết kế các đặc trưng có sức biểu diễn tốt hơn, hoặc dùng một mô hình có kiến trúc phức tạp hơn.

Ngược lại, quá khớp chỉ hiện tượng một mô hình khớp đặc biệt tốt với dữ liệu huấn luyện nhưng tổng quát hóa kém trên dữ liệu kiểm tra (xem

Hình 1.4). Trong NLP, một biểu hiện cơ bản của quá khớp là khi một bộ phân loại văn bản ghi nhớ cứng nhắc chính xác bộ từ vựng đã thấy trong lúc huấn luyện, khiến nó thất bại hoàn toàn khi xuất hiện các từ OOV (xem Mục 1.2.5).

Nguyên nhân của quá khớp rất đa dạng. Một nguyên nhân chính là huấn luyện một mô hình quá phức tạp trên một tập dữ liệu nhỏ. Độ phức tạp của mô hình thường rất quan trọng khi ta thiết kế một mô hình học máy. Nếu mô hình phức tạp và có nhiều tham số, nó sẽ dễ quá khớp với một số ít mẫu hơn nhiều (xem Hình 1.4). Vấn đề sẽ khó hơn nữa nếu có dữ liệu nhiễu, do lỗi kiểu "rác vào, rác ra" (*garbage in, garbage out*) trong huấn luyện. Ngoài ra, huấn luyện quá mức là một nguyên nhân khác của quá khớp. Ví dụ, ta có thể tinh chỉnh mạnh một hệ thống để ép nó mô hình hóa dữ liệu không có lỗi nào. Hệ thống như vậy sẽ rất mong manh trước các mẫu mới, ngay cả khi đầu vào chỉ dao động nhỏ.

Chú giải của người dịch: Chưa khớp và quá khớp qua chuyện ôn thi

Hình dung ba học sinh ôn thi với cùng một đề cương 100 bài. Bạn A chỉ học qua loa vài công thức: làm sai nhiều cả bài trong đề cương lẫn bài trong đề thi — đó là *chưa khớp*. Bạn C học thuộc lòng đáp án của đúng 100 bài, kể cả vài đáp án in sai trong sách (nhiễu): làm đúng 100% đề cương nhưng gặp bài mới thì lúng túng — đó là *quá khớp*, ứng với đường biên ngoằn ngoèo ở Hình 1.4 (phải) cố "ôm" từng điểm. Bạn B hiểu phương pháp giải: có thể sai vài bài trong đề cương nhưng làm tốt đề thi — đó là "khớp vừa đủ". Dấu hiệu nhận biết thường dùng: quá khớp là khi lỗi trên dữ liệu huấn luyện rất nhỏ nhưng lỗi trên dữ liệu mới lớn hơn hẳn; chưa khớp là khi cả hai đều lớn.

Quá khớp có thể được giảm bớt theo nhiều cách. Dưới đây là một số kỹ thuật thường dùng.

- **Dùng nhiều dữ liệu huấn luyện (chất lượng cao) hơn**. Huấn luyện trên quy mô lớn giúp mô hình nắm bắt được các khuôn mẫu thực sự trong dữ liệu. Tuy nhiên, thêm dữ liệu nhiễu sẽ có tác dụng tiêu cực.

- **Dùng dữ liệu kiểm định**. Dữ liệu kiểm định (*validation*) là một tập con rời (không giao) với phần còn lại của dữ liệu có sẵn, được dùng để đánh giá mô hình trong quá trình huấn luyện mà không trực

tiếp cập nhật các tham số. Ví dụ, một tập dữ liệu có thể được chia thành một tập huấn luyện và một **tập kiểm định giữ lại** (*held-out validation set*). Người làm thực tế có thể theo dõi hiệu năng trên tập kiểm định và áp dụng **dừng sớm** (*early stopping*), chẳng hạn dừng quá trình huấn luyện ngay khi hiệu năng trên tập kiểm định bắt đầu giảm.

- **Dùng kiến trúc mô hình đơn giản hơn**. Như đã nói ở trên, dao cạo Occam là một nguyên lý ta có thể tuân theo khi thiết kế mô hình. Nếu một bài toán về bản chất là đơn giản, việc giảm năng lực của mô hình (ví dụ dùng ít tầng hoặc ít tham số hơn) buộc nó tập trung vào các khuôn mẫu chủ đạo, có tính tổng quát trong dữ liệu.

- **Chính quy hóa**. Chính quy hóa là một cách khác để kiểm soát độ phức tạp của mô hình. Thông thường, nó ràng buộc các tham số mô hình bằng các tiên nghiệm. Một ví dụ là làm trơn (xem Mục 1.2.5): nó ước lượng lại phân phối của các từ sau khi huấn luyện. Một phương pháp tổng quát hơn là huấn luyện có chính quy hóa (xem Phương trình (1.81)). Ví dụ, ta có thể định nghĩa thừa số chính quy hóa là chuẩn l_1 của các tham số, và hướng mô hình tới những bộ tham số không có giá trị tuyệt đối lớn.

- **Kết hợp nhiều mô hình**. Một dự đoán tốt hơn cũng có thể đạt được bằng cách kết hợp (*ensembling*) nhiều mô hình. Các mô hình này (gọi là **mô hình thành phần** (*component models*)) nói chung có tham số hoặc kiến trúc khác nhau, và/hoặc được huấn luyện trên các phần khác nhau của dữ liệu. Sự khác biệt giữa các mô hình có thể làm giảm nguy cơ tất cả chúng đều quá khớp dữ liệu theo đúng cùng một cách. Do đó, các mô hình này ít có khả năng mắc cùng những lỗi giống nhau khi dự đoán.

Chú giải của người dịch: Dừng sớm và kết hợp mô hình

Dừng sớm: giống như ôn thi bằng đề cương (tập huấn luyện) nhưng thỉnh thoảng tự làm một đề thử chưa từng xem (tập kiểm định). Khi điểm đề thử bắt đầu *giảm* dù điểm đề cương vẫn tăng, đó là lúc bạn đang "học vẹt" — nên dừng lại. *Kết hợp mô hình*: giống như hỏi ý kiến cả nhóm thay vì một người; ví dụ 5 mô hình bỏ phiếu, 4 nói "Đồ ăn", 1 nói "Không phải đồ ăn" thì chọn "Đồ ăn". Nếu mỗi mô hình mắc lỗi ở những chỗ khác nhau, các lỗi

riêng lẻ có xu hướng bị "át" đi khi bỏ phiếu. Chuẩn l_1 của các tham số là tổng giá trị tuyệt đối $\sum_i |\theta_i|$ (xem Mục 1.1.1).

1.3.6 Dự đoán và suy luận

Mặc dù ở các mục trước chúng ta chỉ giới hạn thảo luận ở bài toán phân loại, học máy (có giám sát) không chỉ là bài toán dự đoán một nhãn cho một đối tượng đầu vào. Có nhiều loại bài toán học máy, tùy thuộc vào dạng của dự đoán được định nghĩa.

- **Phân loại** (*classification*). Phân loại có lẽ là một trong những bài toán học máy phổ biến nhất. Một hệ thống phân loại được yêu cầu gán một hoặc nhiều lớp cho một đối tượng đầu vào.
- **Hồi quy**. Trong thống kê, hồi quy nghiên cứu mối quan hệ giữa một **biến phụ thuộc** (*dependent variable*) (hay một kết quả) và một **biến độc lập** (*independent variable*). Dù hồi quy có nhiều ứng dụng, trong NLP nó thường được phát biểu dưới dạng dự đoán điểm số. Ví dụ, nhận một bài đánh giá phim làm đầu vào (tức là biến độc lập), mô hình hồi quy học cách dự đoán một điểm đề xuất (tức là biến phụ thuộc).
- **Xếp hạng** (*ranking*). Một mô hình xếp hạng dự đoán thứ tự của một tập các đối tượng đầu vào. Ví dụ, một mô hình xếp hạng một số bản dịch theo chất lượng dịch.
- **Dự đoán cấu trúc** (*structure prediction*). Nhiều mô hình học máy được yêu cầu cho ra không chỉ một giá trị thực hay một lớp mà còn một cây hay một chuỗi. Bài toán dự đoán các đầu ra có cấu trúc được gọi là dự đoán cấu trúc. Ví dụ, một bộ phân tích cú pháp (*syntactic parser*) là một hệ thống dự đoán cấu trúc, vì đầu ra của nó là một cấu trúc cây.

Mặc dù là một khía cạnh nền tảng của học máy, việc dự đoán theo thông lệ thường được coi là đơn giản, bởi nhiều mô hình và phương pháp được thử nghiệm trên các bài toán phân loại và hồi quy chuẩn. Mặt khác, dự đoán không hề đơn giản trong các bài toán dự đoán cấu trúc như phân tích cú pháp và dịch máy, vốn rất phổ biến trong NLP. Về bản chất, dự

đoán một cây hay một chuỗi là một bài toán tìm kiếm (*search*). Ví dụ, về lý thuyết có vô số bản dịch hợp lệ cho một câu ở ngôn ngữ nguồn. Ngay cả khi có một mô hình hoàn hảo để đánh giá mọi bản dịch, việc tìm ra bản dịch tối ưu trong không gian tìm kiếm này vẫn là một thách thức tính toán nghiêm trọng.

Trong những trường hợp như vậy, ta phải dùng các cách hiệu quả để thực hiện tìm kiếm. Điều này được thực hiện hoặc bằng cách sử dụng các thuật toán tìm kiếm tổng quát từ lĩnh vực trí tuệ nhân tạo, hoặc bằng cách phát triển các heuristic (*heuristics*) mới cho từng bài toán cụ thể. Hơn nữa, việc phân tích bài toán tìm kiếm này cho ta một góc nhìn quan trọng về các lỗi mà một mô hình học máy mắc phải: lỗi có thể được phân thành lỗi mô hình (*model errors*) (do mô hình hóa xác suất không chính xác) hoặc lỗi tìm kiếm (*search errors*) (do thuật toán tìm kiếm không tìm được điểm tối ưu thực sự). Dù loại bỏ lỗi tìm kiếm là một mục tiêu chính, làm được điều đó thường đòi hỏi một lượng tính toán lớn đến mức không thể chấp nhận. Vì vậy, triển khai một mô hình học máy cho mục đích thực tế luôn đòi hỏi sự đánh đổi giữa hiệu quả và độ chính xác. Chúng ta sẽ thấy các ví dụ về sự đánh đổi này trong Chương 5.

Chú giải của người dịch: Vì sao dự đoán một câu lại là bài toán tìm kiếm?

Với phân loại 2 lớp, ta chỉ cần tính điểm 2 lựa chọn rồi chọn cái lớn hơn. Nhưng một bản dịch dài 20 từ, mỗi vị trí chọn từ một bộ từ vựng 10 000 từ, có tới $10\,000^{20} = 10^{80}$ khả năng — xấp xỉ số nguyên tử trong vũ trụ quan sát được! Không thể thử hết, nên ta dùng các chiến lược tìm kiếm "khôn" (như chọn từng từ tốt nhất một, hay giữ lại vài ứng viên tốt nhất ở mỗi bước) và chấp nhận có thể bỏ lỡ đáp án tối ưu. Khi kết quả sai, ta phân biệt: *lỗi mô hình* là khi mô hình chấm điểm bản dịch sai cao hơn bản dịch đúng; *lỗi tìm kiếm* là khi mô hình chấm đúng nhưng thuật toán tìm kiếm không tìm tới được bản dịch đúng.

1.4 Lựa chọn và đánh giá mô hình

Với hầu hết các bài toán học máy, mục tiêu là tìm được một mô hình hoạt động tốt nhất trên dữ liệu mới. Từ mục tiêu này có thể tách ra hai bài toán [Hastie và cộng sự, 2009]:

- **Lựa chọn mô hình** (*model selection*). Chọn mô hình tốt nhất trên dữ liệu huấn luyện theo một tiêu chí nào đó.
- **Đánh giá mô hình** (*model evaluation*). Ước lượng hiệu năng của một mô hình cho trước trên dữ liệu mới.

Như đã nói ở Mục 1.3.4, hàm mất mát (*loss function*) (hay hàm lỗi – *error function*) là cách thông dụng để đo lỗi của một dự đoán $\mathbf{y}_\theta = f_\theta(\mathbf{x})$ so với một đáp án chuẩn (*gold-standard*) $\mathbf{y}_{\text{gold}}$. Cho K mẫu huấn luyện có nhãn $\{(\mathbf{x}^{(1)}, \mathbf{y}_{\text{gold}}^{(1)}), \cdots, (\mathbf{x}^{(K)}, \mathbf{y}_{\text{gold}}^{(K)})\}$, **lỗi huấn luyện** (*training error*) được cho bởi

$$\text{Err}_{\text{train}} = \mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\}), \tag{1.82}$$

trong đó $\{\mathbf{y}_\theta^{(k)}\}$ là các dự đoán trên tập dữ liệu huấn luyện, còn $\{\mathbf{y}_{\text{gold}}^{(k)}\}$ là các đáp án chuẩn tương ứng. Nói chung, $\mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\})$ được định nghĩa là mất mát trung bình trên toàn bộ các mẫu huấn luyện:

$$\mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\}) = \frac{1}{K}\sum_{k=1}^{K} \mathcal{L}(\mathbf{y}_\theta^{(k)}, \mathbf{y}_{\text{gold}}^{(k)}), \tag{1.83}$$

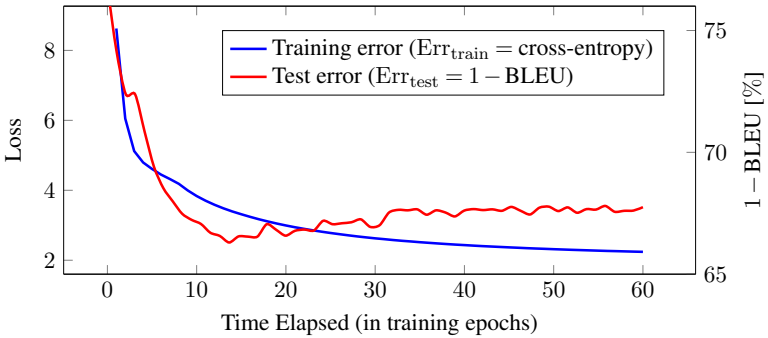
hoặc được định nghĩa là một độ đo duy nhất tính trên toàn bộ tập mẫu huấn luyện. Tương tự, ta có thể định nghĩa **lỗi kiểm tra** (*test error*) trên tập dữ liệu kiểm tra, ký hiệu là Err_{test}. Lỗi kiểm tra đóng vai trò là một ước lượng của **lỗi tổng quát hóa** (*generalization error*), cho biết mô hình được kỳ vọng sẽ hoạt động tốt đến đâu trên dữ liệu mới.

Chú giải của người dịch: Lỗi huấn luyện và lỗi kiểm tra

Hãy hình dung một học sinh ôn thi bằng một bộ đề có đáp án. *Lỗi huấn luyện* giống như số câu sai khi học sinh tự làm lại chính bộ đề đã ôn; *lỗi kiểm tra* là số câu sai trong kỳ thi thật với đề mới. Điều ta thực sự quan tâm là kỳ thi thật (dữ liệu mới), tức *lỗi tổng quát hóa*. Phương trình (1.83) chỉ là phép lấy trung bình cộng quen thuộc: nếu có $K = 4$ mẫu với mất mát lần lượt $0.2,\ 0.5,\ 0.1,\ 0.4$ thì $\text{Err}_{\text{train}} = \frac{1}{4}(0.2 + 0.5 + 0.1 + 0.4) = 0.3$.

Trong các mục trước, ta đã giả định rằng cực tiểu hóa $\text{Err}_{\text{train}}$ là mục tiêu chính của quá trình huấn luyện, tức là $\hat{\theta} = \arg\min_\theta \text{Err}_{\text{train}}$. Tuy nhiên, ta không thể mặc định rằng hàm thu được $f_{\hat{\theta}}(\cdot)$ đồng thời cũng đạt Err_{test} nhỏ nhất. Hình 1.5 minh họa các đường cong học (*learning curves*) của

một hệ thống dịch máy. Rõ ràng Err_{test} không tương quan hoàn toàn với $\text{Err}_{\text{train}}$. Trong khi lỗi huấn luyện giảm liên tục theo tiến trình huấn luyện, lỗi kiểm tra ban đầu cũng giảm theo nhưng rồi bắt đầu tăng lên, cho thấy mô hình bị quá khớp (*overfitting*). Điều này khiến bài toán phức tạp hơn một chút, vì ta không thể lúc nào cũng tin vào $\text{Err}_{\text{train}}$, dù nó là (và nên là) thước đo mức độ tốt của việc huấn luyện. Chắc chắn ta cần một cách nào đó để chọn được mô hình tốt hơn, chứ không chỉ nhìn vào $\text{Err}_{\text{train}}$.



Hình 1.5: Đường cong lỗi huấn luyện và lỗi kiểm tra của một hệ thống học máy. Lỗi huấn luyện được đo bằng mất mát entropy chéo, còn lỗi kiểm tra được đo bằng $1 - \text{BLEU}$. Mọi số liệu được thu thập bằng cách chạy một hệ thống dịch máy nơ-ron trên bộ đánh giá chuẩn IWSLT Đức–Anh (De-En). Lỗi huấn luyện tiếp tục giảm khi số epoch huấn luyện tăng lên. Ngược lại, lỗi kiểm tra có xu hướng giảm trước rồi tăng sau. Khi lỗi kiểm tra bắt đầu tăng, nhiều khả năng mô hình đã quá khớp dữ liệu huấn luyện. (Nhãn trong hình: *Training error* = lỗi huấn luyện, *Test error* = lỗi kiểm tra, *Loss* = mất mát, *Time Elapsed (in training epochs)* = thời gian đã trôi qua, tính bằng số epoch huấn luyện.)

Chú giải của người dịch: BLEU và epoch

BLEU là một điểm số tự động (từ 0 đến 100%) đo mức độ giống nhau giữa bản dịch của máy và bản dịch tham khảo của con người; BLEU càng cao càng tốt, nên $1 - \text{BLEU}$ càng nhỏ càng tốt và có thể dùng làm "lỗi". Một **epoch** là một lượt mô hình duyệt qua toàn bộ dữ liệu huấn luyện. Trong Hình 1.5, sau khoảng 15 epoch, mô hình bắt đầu "học thuộc lòng" dữ liệu huấn luyện thay vì học quy luật chung: lỗi huấn luyện vẫn giảm nhưng lỗi kiểm tra lại tăng — đó chính là quá khớp.

1.4.1 Các chiến lược lựa chọn mô hình

Việc chọn mô hình tối ưu chỉ dựa trên dữ liệu huấn luyện về bản chất là một tham vọng lớn: ta kỳ vọng một mô hình học máy sẽ tổng quát hóa ra toàn bộ tổng thể (*population*) từ một tập mẫu hữu hạn, thường là nhỏ. Từ góc nhìn học thống kê, việc tổng quát hóa chỉ có thể thực hiện được nhờ cách ta phát biểu bài toán học. Một giả định nền tảng trong học máy tiêu chuẩn là giả định i.i.d. (*i.i.d. assumption*) — rằng cả dữ liệu huấn luyện lẫn dữ liệu kiểm tra đều *độc lập và có cùng phân phối* (*independent and identically distributed*), được sinh ra bởi cùng một phân phối sinh dữ liệu (*data-generating distribution*) nền tảng.

Nếu giả định này đúng và tập dữ liệu đủ lớn, một mô hình học từ dữ liệu huấn luyện nhiều khả năng sẽ hoạt động tốt trên dữ liệu kiểm tra. Ngược lại, nếu tập huấn luyện và tập kiểm tra được sinh ra một cách hoàn toàn tùy ý, không có ràng buộc gì (ví dụ, theo một phân phối đều trên toàn bộ không gian của mọi hàm có thể có), thì mô hình sẽ không thể tổng quát hóa. Khi đó, việc quan sát dữ liệu huấn luyện hoàn toàn không cho ta thông tin gì về dữ liệu kiểm tra.

Ý tưởng này được hình thức hóa bởi **Định lý Không có bữa trưa miễn phí** (*No Free Lunch Theorem*) [Wolpert, 1996; Wolpert và Macready, 1997], định lý chứng minh rằng nếu ta lấy trung bình hiệu năng trên mọi bài toán có thể có về mặt toán học, thì mọi thuật toán học đều tốt như nhau (tương đương với đoán ngẫu nhiên)[17]. Nói cách khác, một thuật toán học vạn năng là điều không thể: một thuật toán chỉ có thể thành công nếu nó chứa những thiên kiến quy nạp (*inductive bias*) phù hợp với chính bài toán đang xét. May mắn là, trong các ứng dụng thực tế, dữ liệu huấn luyện và dữ liệu kiểm tra nói chung được giả định là ít nhất cũng tuân theo một phân phối nào đó ở một mức độ nhất định. Vì vậy, thực sự có những cách để nắm bắt phân phối này và cải thiện khả năng tổng quát hóa của mô hình. Khi cải thiện các hệ thống học máy, người ta thường xét hai tình huống:

- Với một kiến trúc mô hình và thuật toán huấn luyện cố định, làm thế nào để xây dựng hoặc tuyển chọn dữ liệu huấn luyện nhằm giảm

[17] Định lý không có bữa trưa miễn phí ban đầu được trình bày trong bối cảnh phân loại [Wolpert, 1996], và sau đó được mở rộng cho các bài toán tìm kiếm và tối ưu hóa [Wolpert và Macready, 1997].

lỗi kiểm tra?

- Với một tập dữ liệu huấn luyện cố định, làm thế nào để thiết kế hoặc chọn kiến trúc mô hình nhằm giảm lỗi kiểm tra?

Tình huống thứ nhất phức tạp và liên quan đến các vấn đề kỹ thuật dữ liệu thực tiễn, như gán nhãn, làm sạch dữ liệu và ước lượng chất lượng dữ liệu. Ta sẽ để các chủ đề này cho các chương sau và ở đây tập trung vào bài toán lựa chọn mô hình trong tình huống thứ hai.

Chú giải của người dịch: i.i.d. và "Không có bữa trưa miễn phí"

i.i.d. nghĩa là mỗi mẫu dữ liệu được "rút thăm" độc lập từ cùng một "chiếc hộp" (cùng phân phối). Ví dụ: nếu đề luyện thi và đề thi thật đều được rút ngẫu nhiên từ cùng một ngân hàng câu hỏi, thì luyện tốt sẽ thi tốt. Nếu đề thi thật lấy từ một môn hoàn toàn khác, việc ôn luyện chẳng giúp gì. Định lý "Không có bữa trưa miễn phí" nói rằng không có phương pháp học nào tốt nhất cho *mọi* bài toán: một phương pháp thắng ở loại bài toán này thì phải thua ở loại khác. Do đó, muốn học tốt, ta phải đưa vào mô hình những giả định (thiên kiến quy nạp) phù hợp với bài toán cụ thể.

1. Độ phức tạp của mô hình

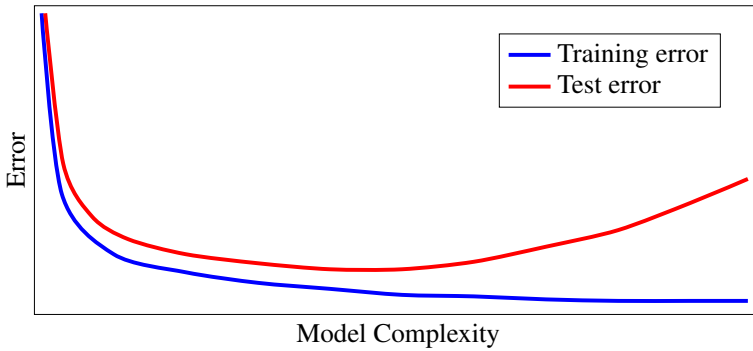
Phương pháp lựa chọn mô hình đơn giản nhất có lẽ là thử các mô hình trên dữ liệu kiểm định (*validation data*). Thông thường, dữ liệu này không trùng lặp với dữ liệu huấn luyện lẫn dữ liệu kiểm tra, nhưng được giả định là được sinh ra theo cùng cách với dữ liệu kiểm tra. Tuy nhiên, không phải lúc nào cũng có sẵn dữ liệu như vậy. Trong một số trường hợp, ta thậm chí không biết gì về dữ liệu kiểm tra. Vì thế, nhiều phương pháp lựa chọn mô hình không cần đến dữ liệu kiểm định.

Một lựa chọn thay thế phổ biến là dựa vào **độ phức tạp của mô hình** (*model complexity*) (hay **năng lực của mô hình** (*model capacity*)) làm chỉ báo định hướng. Trong học máy, độ phức tạp của mô hình có thể được hiểu theo nhiều cách khác nhau. Về mặt trực giác, một mô hình phi tuyến phức tạp hơn một mô hình tuyến tính, và một mô hình có hàng triệu tham số phức tạp hơn một mô hình chỉ có vài tham số. Những định nghĩa hình thức hơn có thể tìm thấy trong lý thuyết học tính toán, chẳng hạn **chiều Vapnik–Chervonenkis (VC)** (*Vapnik-Chervonenkis (VC) dimension*) [Vapnik và Chervonenkis, 1971]. Với mục đích của chúng ta, ta hiểu độ phức tạp của

mô hình một cách rộng rãi là thước đo *năng lực biểu đạt* của mô hình: độ phức tạp càng cao thì không gian giả thuyết (*hypothesis space*) càng lớn.

Dù các mô hình có năng lực biểu đạt cao rất mạnh, độ phức tạp tăng lên không hẳn lúc nào cũng có lợi. Mô hình phức tạp rất dễ bị quá khớp, đặc biệt khi được huấn luyện trên tập dữ liệu nhỏ. Ngược lại, mô hình quá đơn giản dễ bị chưa khớp (*underfitting*). Mục tiêu của lựa chọn mô hình là xác định mức độ phức tạp "tối ưu" của mô hình. Hình 1.6 vẽ lỗi huấn luyện và lỗi kiểm tra theo năng lực của mô hình. Điều quan trọng là độ phức tạp tối ưu không nằm ở chỗ lỗi huấn luyện hội tụ về không, mà ở điểm lỗi kiểm định/kiểm tra đạt cực tiểu toàn cục trước khi bắt đầu tăng lên.

Khi không có lỗi kiểm định tường minh, người ta thường dùng dao cạo Occam (*Occam's Razor*): khi có nhiều mô hình hoạt động tốt tương đương nhau trên dữ liệu huấn luyện, nên ưu tiên mô hình đơn giản nhất. Có nhiều tiêu chí để định lượng độ phức tạp của mô hình. Ví dụ,



Hình 1.6: Đường cong lỗi huấn luyện và lỗi kiểm tra ứng với các mức độ phức tạp khác nhau của mô hình. Mô hình phức tạp giúp giảm lỗi huấn luyện vì chúng có thể tính được các hàm phức tạp để khớp với các điểm dữ liệu. Tuy nhiên, độ phức tạp lớn dễ dẫn đến quá khớp hơn và có hại cho khả năng tổng quát hóa của mô hình. Chẳng hạn, lỗi kiểm tra tăng lên khi độ phức tạp tiếp tục được thêm vào. (Nhãn trong hình: *Error* = lỗi, *Model Complexity* = độ phức tạp của mô hình, *Training error* = lỗi huấn luyện, *Test error* = lỗi kiểm tra.)

- **Số lượng tham số**. Tuy rất đơn giản, việc đếm số tham số tự do (*free parameters*) là một phương pháp kinh nghiệm trực quan và

rất hiệu quả. Trong các mô hình làm trơn tuyến tính, điều này đôi khi được hình thức hóa thành **số bậc tự do hiệu dụng** (*effective degrees of freedom*), được định nghĩa là vết (*trace*) của ma trận chiếu (*projection matrix*) ánh xạ đáp án đúng $\mathbf{y}_{\text{gold}}$ thành các dự đoán $\mathbf{y}_\theta$.

- **Chuẩn p của tham số**. Chuẩn p của một ma trận trọng số là một chỉ báo mạnh về độ phức tạp (xem Mục 1.1.1). Ví dụ, dưới ràng buộc chuẩn L_1 hoặc L_2, một mô hình có giá trị tuyệt đối của tham số lớn hơn sẽ có biên quyết định cong hơn nhiều và do đó được coi là phức tạp hơn.
- **Độ dài mô tả**. Bắt nguồn từ lý thuyết thông tin và nén dữ liệu, tiêu chí này đánh giá số bit cần thiết để mã hóa mô hình cùng các lỗi của nó. Nguyên lý **Độ dài mô tả tối thiểu** (*Minimum Description Length*) (**MDL**) ưu tiên mô hình cho mức nén dữ liệu cao nhất.
- **Chiều VC**. Khái niệm này vốn bắt nguồn từ lý thuyết học tính toán. Chiều VC định lượng một cách hình thức năng lực của mô hình là số điểm dữ liệu lớn nhất mà họ bộ phân loại có thể tách trọn vẹn (*shatter*) (tức là gán được mọi tổ hợp nhãn nhị phân có thể có cho các điểm đó).

Ngoài ra còn những lựa chọn khác để định nghĩa tiêu chí, bao gồm **tiêu chuẩn thông tin Akaike** (*Akaike information criterion*) (**AIC**), **tiêu chuẩn thông tin Bayes** (*Bayesian information criterion*) (**BIC**), **độ dài thông điệp tối thiểu** (*minimum message length*) (**MML**), v.v. Có thể tìm thấy chúng trong nhiều giáo trình về thống kê và/hoặc học thống kê [Burnham và Anderson, 2002; Konishi và Kitagawa, 2007; Hastie và cộng sự, 2009].

Chú giải của người dịch: Chiều VC qua một ví dụ

Xét các bộ phân loại là *đường thẳng* trong mặt phẳng (một bên đường là nhãn $+$, bên kia là nhãn $-$). Với 3 điểm không thẳng hàng, ta luôn kẻ được một đường thẳng tách chúng theo *bất kỳ* cách gán nhãn nào trong $2^3 = 8$ cách — ta nói đường thẳng "tách trọn vẹn" được 3 điểm. Nhưng với 4 điểm là 4 đỉnh của hình vuông, cách gán nhãn "hai đỉnh chéo nhau là $+$, hai đỉnh còn lại là $-$" thì không đường thẳng nào tách được. Người ta chứng minh được rằng không có 4 điểm nào được tách trọn vẹn, nên chiều VC của họ đường thẳng

trong mặt phẳng là 3. Chiều VC càng lớn, họ mô hình càng "linh hoạt". *Dao cạo Occam* là nguyên tắc "nếu hai lời giải thích tốt như nhau, hãy chọn cái đơn giản hơn". Về *vết* của một ma trận vuông: đó là tổng các phần tử trên đường chéo chính, ví dụ vết của $\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$ là $1+4=5$.

2. Sự đánh đổi giữa độ chệch và phương sai

Việc kiểm soát độ phức tạp của mô hình để tránh quá khớp và chưa khớp cũng gắn liền với sự đánh đổi giữa độ chệch và phương sai (*bias-variance tradeoff*). Độ chệch (*bias*) (hay độ chệch dự đoán – *prediction bias*) là mức chênh lệch giữa dự đoán của mô hình và giá trị thực. Trong thống kê, độ chệch là một **sai số hệ thống** (*systematic error*), không thể triệt tiêu dù ta thực hiện rất nhiều lần thí nghiệm lặp lại. Nói chung, sai số do độ chệch xuất phát từ những giả định sai về bài toán, chẳng hạn xấp xỉ một bài toán phi tuyến bằng một mô hình tuyến tính. Điều này rất thú vị! Ta có thể liên hệ sai số do độ chệch ở đây với thiên kiến quy nạp được dùng trong thiết kế mô hình (xem Mục 1.3.2). Ví dụ, với dữ liệu huấn luyện cho trước, một mô hình có độ chệch lớn thường là do có nhiều giả định hơn và mô hình chưa đủ phức tạp. Nói đơn giản, ta có thể nói rằng nhiều thiên kiến quy nạp hơn (hoặc mạnh hơn) có thể dẫn đến độ phức tạp mô hình thấp hơn và sai số do độ chệch trong dự đoán lớn hơn. Đôi khi, thuật ngữ *bias* được dùng như cách gọi tắt cho cả độ chệch trong dự đoán (theo góc nhìn thống kê) lẫn thiên kiến quy nạp (theo góc nhìn thiết kế mô hình), dù chúng được coi là có ý nghĩa khác nhau[18].

Mặt khác, phương sai (*variance*) mô tả mức độ phân tán của các dự đoán khi dữ liệu huấn luyện thay đổi. Sai số do phương sai cũng có tương quan với độ phức tạp của mô hình. Ví dụ, một mô hình phức tạp thường có phương sai cao hơn.

Cả độ chệch và phương sai đều góp phần vào lỗi tổng thể của một hệ thống. Một minh họa kinh điển là phân rã độ chệch–phương sai (*bias-variance decomposition*) của sai số bình phương trung bình (*mean squared error*). Ở đây ta dùng ký hiệu hơi khác so với các mục trước. Gọi $\mathcal{D}$ là một tập gồm K mẫu huấn luyện và $f_{\hat{\theta}(\mathcal{D})}(\cdot)$ là mô hình học được

[18] Trong thống kê, *bias* (độ chệch) thường được dùng để mô tả một khía cạnh nào đó của một bộ ước lượng (*estimator*).

trên $\mathcal{D}$. Ngoài ra, với một mẫu mới $\mathbf{x}$, gọi $\mathbf{y}_{\hat{\theta}(\mathcal{D})} = f_{\hat{\theta}(\mathcal{D})}(\mathbf{x})$ là dự đoán của mô hình và $\mathbf{y}_{\text{gold}}$ là dự đoán "đúng". Độ chệch và phương sai được định nghĩa là:

$$\text{bias} = \mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\text{gold}} \tag{1.84}$$

$$\text{variance} = \mathbb{E}_{\mathcal{D}}\left[\left\|\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\hat{\theta}(\mathcal{D})}\right\|^2\right], \tag{1.85}$$

trong đó $\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}]$ là dự đoán kỳ vọng, lấy trung bình trên mọi tập dữ liệu huấn luyện có thể có với kích thước K. Như vậy, độ chệch biểu diễn độ sai lệch giữa dự đoán kỳ vọng và giá trị thực, còn phương sai biểu diễn bình phương độ lệch kỳ vọng của từng dự đoán riêng lẻ so với giá trị trung bình của chúng[19]. Lấy sai số bình phương trung bình làm thước đo lỗi, ta có thể viết lỗi kỳ vọng như sau:

$$\begin{aligned}\text{error} &= \mathbb{E}_{\mathcal{D}}\left[\left\|\mathbf{y}_{\hat{\theta}(\mathcal{D})} - \mathbf{y}_{\text{gold}}\right\|^2\right] \\ &= \|\text{bias}\|^2 + \text{variance}.\end{aligned} \tag{1.86}$$

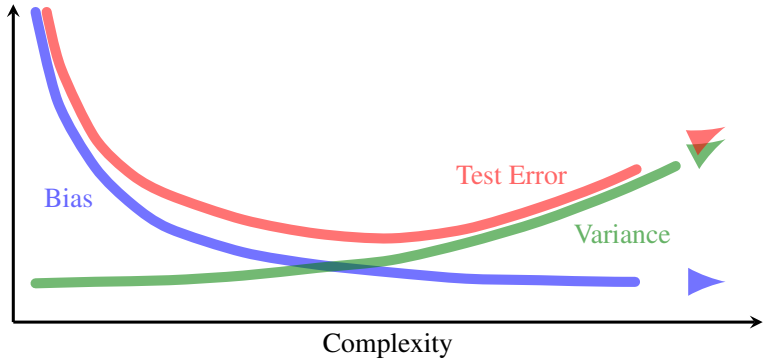
Chú giải của người dịch: Độ chệch và phương sai: ví dụ bắn cung

Hãy hình dung mỗi tập huấn luyện $\mathcal{D}$ khác nhau cho ra một "mũi tên" (một dự đoán), còn $\mathbf{y}_{\text{gold}}$ là hồng tâm. *Độ chệch* là khoảng cách từ *tâm của chùm tên* tới hồng tâm (bắn lệch có hệ thống); *phương sai* cho biết các mũi tên *tản mát* quanh tâm chùm đến mức nào. Ví dụ số (trường hợp một chiều): giả sử giá trị đúng là $y_{\text{gold}} = 5$ và trên 4 tập huấn luyện khác nhau mô hình dự đoán $4,\ 6,\ 7,\ 7$. Trung bình là 6, nên bias $= 6 - 5 = 1$; phương sai là $\frac{1}{4}[(6-4)^2+(6-6)^2+(6-7)^2+(6-7)^2] = \frac{6}{4} = 1.5$. Lỗi bình phương trung bình là $\frac{1}{4}[(4-5)^2+(6-5)^2+(7-5)^2+(7-5)^2] = \frac{10}{4} = 2.5 = 1^2+1.5$, đúng như Phương trình (1.86). Ký hiệu $\mathbb{E}_{\mathcal{D}}[\cdot]$ đọc là "kỳ vọng theo $\mathcal{D}$", hiểu nôm na là trung bình trên mọi tập huấn luyện có thể có; $\|\mathbf{v}\|^2$ là bình phương độ dài của vectơ $\mathbf{v}$.

Để sai số bình phương trung bình thấp hơn, giảm đồng thời cả độ chệch lẫn phương sai rõ ràng là mục tiêu lý tưởng. Tuy nhiên, giảm đại

[19] Nói một cách chặt chẽ, bias như định nghĩa ở đây là một vectơ, còn variance là một số vô hướng. Hơn nữa, vì $\mathbf{y}$ biểu diễn một phân phối xác suất trong chương này, sự khác biệt giữa các phân phối thường được đo bằng các độ đo phân kỳ (như entropy chéo hoặc độ phân kỳ Kullback–Leibler) thay vì khoảng cách Euclid. Công thức độ chệch dựa trên khoảng cách Euclid trình bày ở đây chỉ đóng vai trò minh họa trực quan về mặt toán học. Về mặt khái niệm, có thể thay hiệu số Euclid bằng một độ đo phân kỳ khi đánh giá các phân phối.

lượng này thường làm tăng đại lượng kia (xem Hình 1.7). Người làm thực tế phải tìm mức độ phức tạp tối ưu của mô hình để cân bằng sự đánh đổi này, nhờ đó tránh được quá khớp và chưa khớp. Sự cân bằng này phụ thuộc rất nhiều vào độ phức tạp vốn có của bài toán. Một mô hình tuyến tính đơn giản nhìn chung có phương sai thấp nhưng độ chệch cao, điều này dẫn đến chưa khớp nếu bài toán thực chất có tính phi tuyến cao.



Hình 1.7: Độ chệch và phương sai theo độ phức tạp của mô hình [Goodfellow và cộng sự, 2016]. Các đường cong cho thấy sự xung đột khi giảm đồng thời sai số do độ chệch và sai số do phương sai. Bằng cách thay đổi độ phức tạp của mô hình, ta có thể thu được một mô hình độ chệch thấp – phương sai cao, hoặc một mô hình độ chệch cao – phương sai thấp. Cả hai trường hợp đều có lỗi kiểm tra cao. Ví dụ, mô hình phương sai cao thường có độ phức tạp lớn hơn. Dù mô hình như vậy có thể xử lý các bài toán phức tạp, nó dễ quá khớp dữ liệu hơn. Mặt khác, mô hình độ chệch cao thường là mô hình đơn giản hơn nhưng lại có xu hướng chưa khớp dữ liệu. Để cải thiện khả năng tổng quát hóa trên dữ liệu kiểm tra, ta có thể tìm một điểm cân bằng giữa độ chệch và phương sai. Chẳng hạn, lỗi kiểm tra thấp khi ta chọn một mô hình "phức tạp vừa phải". (Nhãn trong hình: *Bias* = độ chệch, *Variance* = phương sai, *Test Error* = lỗi kiểm tra, *Complexity* = độ phức tạp.)

Quay lại bài toán lựa chọn mô hình, sự đánh đổi giữa độ chệch và phương sai không phải là một quy tắc để chọn mô hình, mà là một nguyên lý định hướng cho việc thiết kế mô hình. Thường thì ta phải chấp nhận những thỏa hiệp để tạo ra một mô hình tổng quát hóa tốt. Cũng cần lưu ý rằng, trong nhiều ứng dụng, mô hình phức tạp thường đi kèm với chi phí tính toán kém hiệu quả. Một cách làm hợp lý có thể là bắt đầu với một mô

hình đơn giản và chỉ tăng thêm độ phức tạp khi cần.

3. Kết hợp mô hình

Chọn một mô hình tối ưu duy nhất không phải là chiến lược duy nhất để giảm lỗi tổng quát hóa. Một cách khác là kết hợp nhiều mô hình thành một bộ dự đoán duy nhất, vững chắc hơn. Mô thức này được gọi là **học kết hợp** (*ensemble learning*) [Seni và cộng sự, 2010; Zhou, 2012a]. Ý tưởng cốt lõi của học kết hợp là huấn luyện một tập các **mô hình thành phần** (*component models*) (thường gọi là bộ học cơ sở (*base learners*)) rồi tổng hợp đầu ra của chúng để tạo ra một dự đoán tốt hơn. Cách đơn giản nhất là **lấy trung bình mô hình** (*model averaging*) (hay bỏ phiếu mềm (*soft voting*)), tức là tính trung bình các điểm số dự đoán từ từng mô hình riêng lẻ. Các phương pháp tinh vi hơn, như hỗn hợp chuyên gia (*mixtures of experts*), gán trọng số động hoặc kết hợp các biểu diễn bên trong của những mô hình con này.

Nói chung, các mô hình thành phần nên có *sự đa dạng trong dự đoán*. Sự đa dạng này thường đạt được bằng cách huấn luyện các mô hình trên những tập con khác nhau của dữ liệu, dùng các tập con đặc trưng khác nhau, hoặc dùng các cách khởi tạo trọng số khác nhau. Từ góc nhìn thống kê, những phương pháp này có thể giảm lỗi tổng quát hóa một cách có hệ thống. Ví dụ, bagging (*bagging*) giúp giảm phương sai [Breiman, 1996], còn boosting (*boosting*) giúp giảm độ chệch [Schapire, 1990]. Điều này gắn trở lại với những gì đã trình bày ở Mục 1.4.1: lỗi tổng quát hóa có thể được giảm bằng cách giảm sai số do độ chệch hoặc giảm sai số do phương sai.

Tuy nhiên, việc bàn về cách kết hợp mô hình nằm ngoài phạm vi chương này. Dù có thể không hoàn toàn chính xác khi xếp kết hợp mô hình vào chủ đề lựa chọn mô hình, nó vẫn có thể được xem là một cách cải thiện khả năng tổng quát hóa. Theo nghĩa này, cả kết hợp mô hình và lựa chọn mô hình đều giải quyết những vấn đề có chung một chủ đề. Thực tế, kết hợp mô hình hiệu quả đáng kể cho nhiều nhiệm vụ NLP. Chẳng hạn, phần lớn các hệ thống tiên tiến nhất trong NLP đều dựa trên sự kết hợp nhiều mô hình.

Chú giải của người dịch: Học kết hợp: "trí tuệ đám đông"

Giống như hỏi ý kiến nhiều giám khảo rồi lấy điểm trung bình thay vì tin một người, học kết hợp lấy trung bình dự đoán của nhiều mô hình. Ví dụ: ba mô hình cho xác suất lớp "Đồ ăn" của một văn bản là 0.9, 0.6 và 0.3; bỏ phiếu mềm cho $(0.9 + 0.6 + 0.3)/3 = 0.6$. Khi các mô hình mắc lỗi *khác nhau* (đa dạng), các lỗi ngẫu nhiên có xu hướng bù trừ nhau, nên phương sai giảm. *Bagging* huấn luyện mỗi mô hình trên một tập dữ liệu được rút ngẫu nhiên có hoàn lại từ tập gốc; *boosting* huấn luyện tuần tự, mô hình sau tập trung sửa những mẫu mà các mô hình trước làm sai.

1.4.2 Dữ liệu huấn luyện, kiểm định và kiểm tra

Bây giờ ta chuyển sang vấn đề dữ liệu. Như đã bàn ở các mục trước, trong giai đoạn huấn luyện, một tập dữ liệu huấn luyện (*training dataset*) được dùng để khớp các tham số của mô hình. Trong giai đoạn kiểm tra, một tập dữ liệu kiểm tra (*test dataset*) được dùng để đánh giá mô hình đã học. Liên quan chặt chẽ nhưng khác với dữ liệu kiểm tra là tập dữ liệu kiểm định (*validation dataset*). Dù tập kiểm định cũng được dùng để đánh giá, nó được sử dụng *trong* giai đoạn huấn luyện. Mục đích chính của nó là định hướng việc lựa chọn mô hình và tinh chỉnh siêu tham số (*hyperparameter*) (chẳng hạn, xác định khi nào áp dụng dừng sớm (*early stopping*)).

Trong tình huống lý tưởng, nhà nghiên cứu có các tập dữ liệu riêng biệt cho huấn luyện, kiểm định và kiểm tra. Tuy nhiên, điều này hiếm khi thực tế trong các ứng dụng thật. Ví dụ, nhà phát triển không thể tiếp cận dữ liệu thật của người dùng trong tương lai trước khi hệ thống được triển khai. Từ góc nhìn khoa học, không có dữ liệu mới "thật sự" để kiểm tra — khi bạn đã thấy dữ liệu mới, nó không còn mới nữa.

Một cách đơn giản, như trong nhiều bài báo nghiên cứu, là kiểm chứng các mô hình học máy trên các nhiệm vụ đánh giá chuẩn. Trong các nhiệm vụ này, toàn bộ dữ liệu đã được chuẩn bị sẵn, và tất cả những gì bạn cần làm là chạy mô hình của mình trên dữ liệu đó. Cách này giúp dễ dàng so sánh trực tiếp các hệ thống khác nhau, vì tất cả chúng được huấn luyện và kiểm tra trên cùng các tập dữ liệu. Đôi khi, ta chỉ được cung cấp một tập dữ liệu duy nhất chưa được chia sẵn. Khi đó, dữ liệu có thể được chia thành nhiều phần, mỗi phần dùng cho một mục đích. Ví dụ, có thể chia 60% để huấn luyện, 20% để kiểm định và 20% để kiểm tra.

Dù việc chia dữ liệu cho ta một cách đánh giá hiệu năng của mô hình, kết quả đánh giá không phải lúc nào cũng ổn định do sai lệch lấy mẫu (*sampling bias*). Đôi khi hiệu năng thay đổi rất nhiều giữa các lần chia dữ liệu khác nhau. Vấn đề này càng rõ khi tập dữ liệu quá nhỏ để huấn luyện hay kiểm tra một cách đầy đủ.

Một cách thông dụng để giảm ảnh hưởng của sai lệch này là **kiểm định chéo** (*cross-validation*). Kiểm định chéo là một phương pháp lấy mẫu lại (*resampling*), trong đó dữ liệu được phân chia lặp đi lặp lại nhiều lần, và kết quả cuối cùng là trung bình các kết quả đánh giá qua mọi vòng. Một biến thể đơn giản là lấy mẫu con ngẫu nhiên (*random subsampling*), lặp lại việc phân chia dữ liệu một cách ngẫu nhiên và lấy trung bình hiệu năng qua các lần chạy. Một phương pháp có hệ thống hơn là kiểm định chéo k-phần (k-*fold cross-validation*). Phương pháp này chia dữ liệu thành k phần (hay *fold*) rời nhau. Trong mỗi vòng kiểm định chéo, $k-1$ phần được gộp lại làm dữ liệu huấn luyện, còn phần duy nhất còn lại được giữ ra làm dữ liệu kiểm tra (hoặc kiểm định). Ví dụ, trong kiểm định chéo 10-phần, mô hình được huấn luyện và đánh giá 10 lần, mỗi lần dùng một phần khác nhau làm tập kiểm tra.

Chú giải của người dịch: Kiểm định chéo 5-phần

Giả sử có 100 mẫu, chia thành 5 phần, mỗi phần 20 mẫu: P1,…,P5. Vòng 1: huấn luyện trên P2–P5, kiểm tra trên P1; vòng 2: huấn luyện trên P1, P3–P5, kiểm tra trên P2;… cho đến vòng 5. Nếu độ chính xác ở 5 vòng là $80\%, 84\%, 78\%, 82\%, 81\%$ thì kết quả báo cáo là trung bình 81%. Mỗi mẫu đều được dùng để kiểm tra đúng một lần, nên kết quả ít phụ thuộc vào "may rủi" của một lần chia.

Thêm một lưu ý về quy mô dữ liệu. Với người làm thực tế, một trong những câu hỏi thường gặp nhất là cần bao nhiêu mẫu thì đủ để học được một mô hình tốt. Hiếm khi có một sự đồng thuận chung. Dù lý thuyết học tính toán cung cấp các cận lỗi hình thức ứng với một kích thước mẫu cụ thể, kinh nghiệm thực tế phổ biến vẫn chỉ đơn giản là "càng nhiều càng tốt". Tuy nhiên, học máy hiện đại ngày càng chú trọng **hiệu quả mẫu** (*sample efficiency*). Một hệ thống hiệu quả về mẫu có thể đạt hiệu năng cao với số ví dụ huấn luyện ít hơn đáng kể hoặc cần ít epoch huấn luyện hơn. Ví dụ, trong các nghiên cứu NLP gần đây, tinh chỉnh (*fine-tuning*) một mô hình tiền huấn luyện (*pre-trained*) rất hiệu quả về mẫu vì nó tận

dụng lượng tri thức có sẵn phong phú, chỉ cần cập nhật ở mức vừa phải thay vì học các biểu diễn từ đầu. Một biểu hiện cực đoan của điều này là **học few-shot** (*few-shot learning*), với mục tiêu tổng quát hóa một cách vững chắc chỉ sau khi quan sát một số ít mẫu của nhiệm vụ cụ thể.

1.4.3 Thước đo hiệu năng

Là một phần thiết yếu của mọi bài toán học máy, thước đo hiệu năng (*performance measure*) mô tả một hệ thống hoạt động tốt đến đâu trên một lượng dữ liệu nào đó. Thông thường, nó được dùng hoặc để thiết kế mục tiêu huấn luyện, hoặc để đánh giá kết quả của hệ thống cuối cùng. Ví dụ, tất cả các hàm mất mát mô tả ở Mục 1.3.4 đều có thể dùng làm thước đo hiệu năng.

Khi đánh giá hiệu năng trên dữ liệu kiểm tra, các độ đo thường được thiết kế để đếm số lỗi thực tế. Vì vậy, dùng lại các hàm mất mát trong huấn luyện có thể không phải là lựa chọn tốt để báo cáo điểm số cuối cùng. Chẳng hạn, các độ đo được dùng rộng rãi cho bài toán phân loại là độ chính xác dự đoán (*precision*), độ phủ (*recall*) và điểm F_1 (F_1 *score*). Chúng được đề xuất để định lượng năng lực của hệ thống phân loại ở một số khía cạnh nhất định: với một lớp c, độ chính xác dự đoán tính tỉ lệ dự đoán đúng trong số tất cả các mẫu được dự đoán là c, còn độ phủ tính tỉ lệ dự đoán đúng trên tất cả các mẫu có nhãn là c. Điểm F_1 là độ đo kết hợp độ chính xác dự đoán và độ phủ.

Chú giải của người dịch: Precision, recall và F_1 qua ví dụ

Có 10 văn bản thật sự thuộc lớp "Đồ ăn". Bộ phân loại gắn nhãn "Đồ ăn" cho 8 văn bản, trong đó 6 văn bản đúng là "Đồ ăn". Khi đó *precision* $= 6/8 = 0.75$ (trong những gì đã "bắt", bao nhiêu phần đúng), *recall* $= 6/10 = 0.6$ (trong những gì cần bắt, bắt được bao nhiêu phần). F_1 là *trung bình điều hòa* của hai số này: $F_1 = \frac{2 \cdot P \cdot R}{P+R} = \frac{2 \cdot 0.75 \cdot 0.6}{0.75+0.6} \approx 0.667$. Trung bình điều hòa "phạt" nặng khi một trong hai số quá thấp.

Lưu ý rằng các thước đo hiệu năng không nhất thiết được thiết kế để tối ưu hóa. Theo nghĩa đó, chúng có thể không đảm bảo một số tính chất toán học, chẳng hạn khả vi hay liên tục. Một ví dụ là độ đo **BLEU** được dùng rộng rãi trong dịch máy [Papineni và cộng sự, 2002]. BLEU tính một điểm độ chính xác n-gram (n-*gram precision*) có hiệu chỉnh, kết hợp

với một hình phạt độ ngắn (*brevity penalty*). Những phép đếm rời rạc này khiến độ đo về bản chất không khả vi. NLP có rất nhiều độ đo đánh giá riêng cho từng nhiệm vụ như vậy. Điều này dẫn tới một sự lệch pha giữa huấn luyện và đánh giá (*training-evaluation discrepancy*) mang tính nền tảng: hàm mất mát được tối ưu trong huấn luyện khác với độ đo dùng để đánh giá mô hình cuối cùng. Đôi khi người làm thực tế cần tính đến khoảng cách này khi thiết kế hệ thống.

Một thách thức khác với các thước đo hiệu năng trong NLP là một đầu vào thường có nhiều đầu ra hợp lệ. Ví dụ, một câu ở ngôn ngữ nguồn thường có nhiều bản dịch tốt. Một giải pháp là tính đến nhiều đáp án chuẩn khi thiết kế thước đo hiệu năng. BLEU là một trường hợp như vậy: nó đếm số đoạn dịch đúng lớn nhất trên tất cả các bản dịch tham khảo (*reference translations*). Giải pháp thứ hai là đánh giá bởi con người (*human evaluation*). Cách đánh giá này chính xác hơn nhưng tất nhiên tốn kém hơn. Khi phát triển hệ thống thực tế, người làm thực tế thường huấn luyện và tinh chỉnh hệ thống bằng các thước đo tự động, và chỉ dùng đến đánh giá của con người cho lần kiểm tra cuối cùng.

Chú giải của người dịch: n-gram và tính khả vi

Một *n-gram* là một dãy n từ liên tiếp; ví dụ câu "tôi thích trái cây" có các 2-gram "tôi thích", "thích trái", "trái cây". BLEU đếm xem bao nhiêu n-gram của bản dịch máy xuất hiện trong bản dịch tham khảo. Vì kết quả đếm là số nguyên, thay đổi rất nhỏ tham số mô hình thường không làm điểm BLEU thay đổi chút nào (hoặc làm nó "nhảy bậc"), nên không thể lấy đạo hàm để biết nên chỉnh tham số theo hướng nào. Đó là lý do người ta huấn luyện bằng entropy chéo (trơn, lấy đạo hàm được) nhưng lại báo cáo kết quả bằng BLEU.

1.4.4 Kiểm định ý nghĩa thống kê

Giả sử bạn đang cải tiến một hệ thống theo cách nào đó, bạn có thể tự hỏi liệu cải tiến đó có đủ đáng kể hay không. Tất cả những gì bạn có là một thước đo hiệu năng. Vì vậy bạn có thể biết chênh lệch hiệu năng giữa hai thời điểm bất kỳ trong quá trình phát triển hệ thống, nhưng không thể biết chênh lệch đó là thật hay chỉ do ngẫu nhiên.

Trong ví dụ này, bạn đang ngầm cố gắng chấp nhận hay bác bỏ khẳng định rằng một hệ thống tốt hơn (hoặc không tốt hơn) một hệ thống khác.

Trong thống kê, **kiểm định ý nghĩa thống kê** (*significance tests*) cung cấp một khuôn khổ hình thức để mô hình hóa bài toán này. Giả sử ta đánh giá hai hệ thống A và B trên nhiều tập dữ liệu hoặc nhiều cách chia dữ liệu kiểm tra, dùng cùng một thước đo hiệu năng. Ta phát biểu hai giả thuyết đối lập:

H_0: Hệ thống A không tốt hơn hệ thống B.
H_1: Hệ thống A tốt hơn hệ thống B.

trong đó H_0 là **giả thuyết không** (*null hypothesis*), còn H_1 là **giả thuyết đối** (*alternative hypothesis*), mâu thuẫn với giả thuyết không. Bằng cách kiểm định các giả thuyết này dựa trên dữ liệu thực nghiệm, ta muốn xác định liệu có đủ bằng chứng để bác bỏ H_0 và chấp nhận H_1, hay không thể bác bỏ H_0. Kiểm định thống kê về bản chất có thể mắc lỗi: ta có thể bác bỏ nhầm giả thuyết không trong khi nó thực ra đúng (**sai lầm loại I** (*Type I error*), hay dương tính giả (*false positive*)), hoặc không bác bỏ giả thuyết không trong khi nó thực ra sai (**sai lầm loại II** (*Type II error*), hay âm tính giả (*false negative*)). Hai loại sai lầm này đánh đổi lẫn nhau. Một cách tiếp cận chuẩn trong thống kê tần suất (*frequentist statistics*) là giới hạn xác suất mắc sai lầm loại I dưới một ngưỡng định trước α, đồng thời cố gắng cực tiểu hóa sai lầm loại II. Ngưỡng α này được gọi là **mức ý nghĩa** (*significance level*) của kiểm định. Thông lệ trong học máy là chọn mức ý nghĩa 1% hoặc 5%.

Khi tiến hành kiểm định, ta tính một **giá trị** p (*p-value*). Lưu ý rằng giá trị p *không phải* là xác suất mắc sai lầm loại I. Thay vào đó, nó là xác suất quan sát được một chênh lệch hiệu năng cực đoan bằng hoặc hơn chênh lệch thực sự quan sát được, *với giả định giả thuyết không là đúng*. Nếu giá trị p này nhỏ hơn mức ý nghĩa α, ta bác bỏ giả thuyết không. Ví dụ, với mức ý nghĩa 5%, một giá trị p quan sát được bằng 3% cho thấy cải thiện hiệu năng là có ý nghĩa thống kê. Để tìm hiểu đầy đủ hơn về kiểm định giả thuyết, bạn đọc có thể tham khảo các giáo trình thống kê thông dụng [McClave và Sincich, 2006; Freedman và cộng sự, 2007; Freedman, 2009].

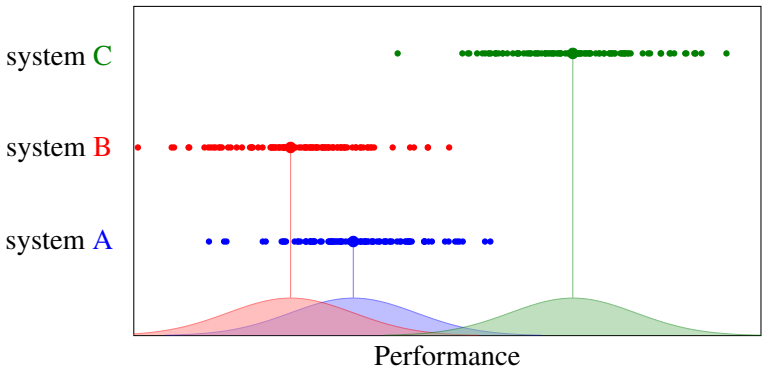
Chú giải của người dịch: Giá trị p qua ví dụ đồng xu

Bạn nghi một đồng xu bị lệch về mặt ngửa. H_0: đồng xu cân đối. Tung 10 lần được 9 lần ngửa. Nếu H_0 đúng, xác suất được 9 hoặc 10 lần ngửa (tức kết quả "cực đoan bằng hoặc hơn") là $\frac{\binom{10}{9}+\binom{10}{10}}{2^{10}} = \frac{11}{1024} \approx 1.1\%$. Đó là giá trị p. Vì $1.1\% < 5\%$, ta bác bỏ H_0 ở mức ý nghĩa 5% (nhưng không bác bỏ được ở mức 1%). Ý tưởng: nếu kết quả quan sát được "quá khó xảy ra" khi H_0 đúng, ta nghi ngờ H_0. So sánh hai hệ thống NLP cũng vậy: "đồng xu cân đối" tương ứng với "hai hệ thống thực ra ngang nhau".

Lưu ý rằng kết luận của kiểm định ý nghĩa phụ thuộc vào nhiều yếu tố, như số lượng thí nghiệm và phương sai trong kết quả thí nghiệm. Một vấn đề khi áp dụng kiểm định ý nghĩa cho các nhiệm vụ NLP là thường có rất ít tập dữ liệu để chạy thí nghiệm [Dror và cộng sự, 2020]. Khi có thể yên tâm giả định rằng thống kê kiểm định (*test statistic*) tuân theo một phân phối xác suất đã biết (chẳng hạn phân phối chuẩn), ta có thể dùng kiểm định tham số (*parametric test*). Tuy nhiên, hiệu năng của các hệ thống NLP trong thực tế hiếm khi thỏa mãn những giả định chặt chẽ này. Do đó, các nhà nghiên cứu thường dựa vào các **kiểm định phi tham số** (*non-parametric tests*) cổ điển (không đặt giả định nào về phân phối nền tảng) hoặc các phương pháp lấy mẫu lại thực nghiệm (như bootstrap (*bootstrapping*) hoặc ngẫu nhiên hóa xấp xỉ (*approximate randomization*)). Những kỹ thuật lấy mẫu lại này xấp xỉ phân phối nền tảng bằng cách lấy mẫu lặp đi lặp lại từ tập dữ liệu sẵn có hoặc xáo trộn ngẫu nhiên đầu ra của các hệ thống.

Kiểm định ý nghĩa rất quan trọng để rút ra những kết luận thuyết phục khi phát triển hệ thống học máy, dù chúng thường vô tình bị bỏ qua. Hình 1.8 cho thấy kết quả đánh giá của ba mô hình. Mỗi mô hình được chạy nhiều lần với các tham số khởi tạo khác nhau. Dù hệ thống A vượt trội hệ thống B về hiệu năng trung bình, kết quả của chúng có phương sai lớn. Kiểm định ý nghĩa cho thấy chênh lệch này không có ý nghĩa thống kê. Ngược lại, chênh lệch giữa hệ thống A và hệ thống C là có ý nghĩa, vì hiệu năng của chúng khác nhau đủ lớn trong hầu hết các trường hợp. Mặt khác, các nhà nghiên cứu đã nhận thấy rằng thực sự có những ngưỡng tăng hiệu năng cho thấy ý nghĩa thống kê trong một số hoàn cảnh nhất định. Ví dụ, ta có thể nói rằng ý nghĩa thống kê có thể được chỉ ra một cách gần đúng bằng một mức tăng nhất định của độ đo nếu ta so sánh các hệ thống

tương tự nhau [Berg-Kirkpatrick và cộng sự, 2012].



Hình 1.8: Hiệu năng của ba hệ thống học máy. Với mỗi hệ thống có nhiều kết quả khác nhau vì ta đưa vào một số yếu tố ngẫu nhiên trong huấn luyện (ví dụ xáo trộn dữ liệu, điểm khởi đầu ngẫu nhiên, v.v.). Dù có vẻ Hệ thống A vượt trội Hệ thống B, thực ra không có khác biệt thật sự giữa chúng, vì phân phối hiệu năng của chúng chồng lấn lên nhau rất nhiều (xem phần dưới cùng của hình). Khi so sánh Hệ thống C với Hệ thống A hoặc B, chênh lệch hiệu năng là có ý nghĩa, vì ta có thể chấp nhận giả thuyết H_1 (tức là Hệ thống C vượt trội Hệ thống A hoặc B) với một số lượng lớn thí nghiệm. (Nhãn trong hình: *system* = hệ thống, *Performance* = hiệu năng.)

1.5 Các nhiệm vụ NLP dưới góc nhìn học máy

Dù có rất nhiều loại nhiệm vụ NLP, phần lớn trong số chúng có thể được phát biểu trong một khuôn khổ học máy chung. Điều này cho phép một mô thức thống nhất để giải quyết những bài toán NLP đa dạng bằng các phương pháp học máy tổng quát. Thông thường, một nhiệm vụ NLP có thể được mô tả là học cách ánh xạ các đơn vị ngôn ngữ sang một đầu ra nào đó. Theo ký hiệu dùng trong chương này, ta dùng $\mathbf{x}$ để chỉ vectơ (hoặc ma trận) đặc trưng đầu vào của một nhiệm vụ NLP, và dùng $f(\mathbf{x})$ để chỉ hàm được học để xử lý $\mathbf{x}$. Dưới đây là một số nhiệm vụ thường gặp trong NLP.

1.5.1 Phân loại

Giả sử có một tập các lớp hay nhãn $\mathcal{C}$. Mỗi lớp được biểu diễn bằng một số nguyên riêng trong $\{1, \cdots, |\mathcal{C}|\}$. Một mô hình phân loại (*classification model*) là một hàm ánh xạ đầu vào $\mathbf{x}$ thành một vectơ $|\mathcal{C}|$ chiều $\mathbf{y}$, tức là $\mathbf{y} = f(\mathbf{x})$. Mỗi phần tử của $\mathbf{y}$ là một điểm số ứng với lớp i, ký hiệu là $y(i)$. Nhiệm vụ ở đây là gán $\mathbf{x}$ vào một hoặc nhiều lớp có điểm số cao nhất. Lấy phân loại đơn nhãn làm ví dụ. Dự đoán được cho bởi phương trình

$$\hat{c} \;=\; \arg\max_{1 \le i \le |\mathcal{C}|} y(i), \tag{1.87}$$

trong đó $\hat{c}$ là lớp "tốt nhất" được gán cho $\mathbf{x}$. Đôi khi ta cần một đầu ra mang dạng xác suất (xem Mục 1.2.1). Gọi $\psi(\cdot)$ là một hàm chuẩn hóa một vectơ thành một phân phối[20]. Khi đó ta thu được một bộ phân loại xác suất:

$$\mathbf{y} \;=\; \psi(f(\mathbf{x})). \tag{1.90}$$

Chú giải của người dịch: Chuẩn hóa thành phân phối: ví dụ số

Giả sử có 3 lớp và $f(\mathbf{x})$ cho điểm $\mathbf{a} = [2 \;\; 1 \;\; 1]$. Theo (1.87), $\hat{c} = 1$ vì $y(1) = 2$ lớn nhất. Chuẩn hóa theo (1.88): $[\frac{2}{4} \;\; \frac{1}{4} \;\; \frac{1}{4}] = [0.5 \;\; 0.25 \;\; 0.25]$. Chuẩn hóa bằng Softmax (1.89): $e^2 \approx 7.39$, $e^1 \approx 2.72$, tổng ≈ 12.83, nên được $\approx [0.576 \;\; 0.212 \;\; 0.212]$. Cả hai cách đều cho các số không âm có tổng bằng 1 (một phân phối xác suất) và giữ nguyên thứ tự lớn–nhỏ. Softmax có ưu điểm là dùng được cả khi điểm số âm (vì $e^a > 0$ với mọi a), còn cách (1.88) đòi hỏi các điểm số không âm.

Phân loại có lẽ là bài toán phổ biến nhất trong NLP. Ngoài việc xếp tài liệu vào các lớp định trước, còn có rất nhiều ứng dụng khác. Trong số đó có việc chọn nghĩa cho một từ [Yarowsky, 1994], xác định cực tính (*polarity*) (tích cực hay tiêu cực) của một câu [Pang và cộng sự, 2002], kiểm tra xem

[20]Một ý tưởng tương tự có thể tìm thấy ở Phương trình (1.46). Cho một vectơ $\mathbf{a} = \begin{bmatrix} a(1) & \cdots & a(n) \end{bmatrix}$, hàm chuẩn hóa có dạng:

$$\psi(\mathbf{a}) \;=\; \left[\frac{a(1)}{\sum_{i=1}^{n} a(i)} \quad \cdots \quad \frac{a(n)}{\sum_{i=1}^{n} a(i)} \right] \tag{1.88}$$

Một cách khác là dùng hàm Softmax:

$$\psi(\mathbf{a}) \;=\; \left[\frac{\exp(a(1))}{\sum_{i=1}^{n} \exp(a(i))} \quad \cdots \quad \frac{\exp(a(n))}{\sum_{i=1}^{n} \exp(a(i))} \right] \tag{1.89}$$

hai thực thể có nên được liên kết với nhau hay không [Krebs và cộng sự, 2018], phân loại cách một tham tố ngữ nghĩa (*semantic argument*) gắn với một động từ [Gildea và Jurafsky, 2002], v.v. Khi áp dụng một mô hình phân loại cho các nhiệm vụ này, tất cả những gì bạn cần làm là thiết kế dạng của $\mathbf{x}$ và tập các lớp.

1.5.2 Gán nhãn chuỗi

Một mở rộng của bài toán phân loại tiêu chuẩn là phân loại đồng thời một tập các mẫu. **Gán nhãn chuỗi** (*sequence labeling*) là một ví dụ về bài toán như vậy. Trong gán nhãn chuỗi, đầu vào là một chuỗi gồm n token, chẳng hạn một chuỗi n từ. Hệ thống gán nhãn chuỗi phải gán cho mỗi token đầu vào $\mathbf{x}(i)$ một nhãn $l(i)$. Ở đây chữ in đậm trong $\mathbf{x}(i)$ dùng để nhấn mạnh rằng token được biểu diễn dưới dạng một vectơ đặc trưng. Để tiện, ta viết $\mathbf{x}(i)$ là $\mathbf{x}_i$ và $l(i)$ là l_i. Hàm $f(\cdot)$ ánh xạ chuỗi $\mathbf{x}_1 \cdots \mathbf{x}_n$ thành một chuỗi khác $\mathbf{y}_1 \cdots \mathbf{y}_n$, trong đó $\mathbf{y}_i$ là vectơ đầu ra tương ứng với $\mathbf{x}_i$. Điều này có thể được phát biểu là:

$$\begin{bmatrix}\mathbf{y}_1 & \cdots & \mathbf{y}_n\end{bmatrix} = f\Big(\begin{bmatrix}\mathbf{x}_1 & \cdots & \mathbf{x}_n\end{bmatrix}\Big). \tag{1.91}$$

Với vectơ $\mathbf{y}_i$, mỗi phần tử $y_i(c)$ tương ứng với điểm dự đoán của một lớp $c \in \mathcal{C}$. Lưu ý rằng $f(\cdot)$ cho phép sử dụng một ngữ cảnh rộng hơn. Ví dụ, có thể đặt điều kiện dự đoán $\mathbf{y}_i$ trên toàn bộ chuỗi đầu vào [Lafferty và cộng sự, 2001]. Đầu ra cuối cùng của hệ thống có thể được định nghĩa là chuỗi nhãn "tối ưu" suy ra từ $\mathbf{y}_1 \cdots \mathbf{y}_n$. Một cách đơn giản là chọn chuỗi nhãn cực đại hóa tổng điểm số trên mọi vị trí

$$\begin{bmatrix}\hat{l}_1 & \cdots & \hat{l}_n\end{bmatrix} = \underset{l_1,\cdots,l_n \in \mathcal{C}}{\arg\max} \sum_{i=1}^{n} y_i(l_i). \tag{1.92}$$

Một ứng dụng trực tiếp của gán nhãn chuỗi trong NLP là gán thẻ cho từng token của chuỗi đầu vào, chẳng hạn **gán nhãn từ loại** (*part-of-speech tagging*) (hay **POS tagging**). Hơn nữa, gán nhãn chuỗi có thể xử lý những bài toán phức tạp hơn bằng cách dùng nhãn một cách khéo léo. Một ví dụ nổi tiếng là dùng định dạng nhãn "IOB" để nhận diện các cụm trải trên nhiều token (gọi là **phân đoạn cụm** (*chunking*)). Trong phương pháp này, "I", "O" và "B" lần lượt chỉ một token nằm trong cụm, một token

Tokens :	Most	are	expected	to	fall	below	previous-month	levels	.
POS tags :	JJS	VBP	VBN	TO	VB	IN	JJ	NNS	·
Chunk tags :	B-NP	B-VP	I-VP	I-VP	I-VP	B-PP	B-NP	I-NP	O
	NP	VP				PP	NP		

Hình 1.9: Một ví dụ gán nhãn chuỗi cho bài toán gán nhãn từ loại và phân đoạn cụm. Ví dụ được lấy từ dữ liệu huấn luyện của nhiệm vụ chung CoNLL 2000. Mỗi token được gán một nhãn từ loại và một nhãn cụm. Nhãn cụm có ký tự đầu được chọn từ $\{B, I, O\}$, trong đó B = bắt đầu một cụm, I = bên trong một cụm, O = bên ngoài cụm. Như vậy, một cụm luôn bắt đầu bằng một nhãn "B", có thể theo sau là các nhãn "I". Ví dụ, cụm VP (cụm động từ) trong ví dụ trải trên chuỗi nhãn cụm "B-VP I-VP I-VP I-VP". (Câu ví dụ giữ nguyên tiếng Anh: "Most are expected to fall below previous-month levels." – "Phần lớn được dự kiến sẽ giảm xuống dưới mức của tháng trước." Nhãn trong hình: *Tokens* = các token, *POS tags* = nhãn từ loại, *Chunk tags* = nhãn cụm.)

nằm ngoài cụm, và token ngoài cùng bên trái của một cụm [Ramshaw và Marcus, 1995]. Như vậy, một cụm luôn bắt đầu bằng "B" và kết thúc ngay trước "B" tiếp theo hoặc một "O" mới. Xem Hình 1.9 để thấy kết quả gán nhãn từ loại và phân đoạn cụm trên một câu ví dụ. Vì gán nhãn chuỗi cho phép gán nhãn cả token lẫn các đoạn (span), nó đã được áp dụng với kết quả tốt cho nhiều nhiệm vụ, bao gồm gán nhãn từ loại [Bahl và Mercer, 1976], phân đoạn cụm [Tjong Kim Sang và Buchholz, 2000], **nhận dạng thực thể có tên** (*named entity recognition*) (**NER**) [Tjong Kim Sang, 2002], v.v.

Chú giải của người dịch: Nhãn từ loại và IOB

Các nhãn trong Hình 1.9 là ký hiệu từ loại tiếng Anh chuẩn: JJS = tính từ so sánh nhất, VBP/VBN/VB = các dạng động từ, TO = từ "to", IN = giới từ, JJ = tính từ, NNS = danh từ số nhiều; NP/VP/PP = cụm danh từ/cụm động từ/cụm giới từ. Ví dụ tiếng Việt với NER theo IOB: "Ông/O Nguyễn/B-PER Văn/I-PER An/I-PER đến/O Hà/B-LOC Nội/I-LOC" — ta đọc ra hai thực thể: tên người "Nguyễn Văn An" và địa danh "Hà Nội". Nhờ mẹo IOB, bài toán tìm *đoạn văn bản* được quy về bài toán gán *một nhãn cho mỗi token*. Phương trình (1.92) nói rằng nếu điểm của mỗi vị trí độc lập với nhau, chuỗi nhãn tốt nhất chính là chọn nhãn có điểm cao nhất ở từng vị trí.

1.5.3 Mô hình hóa ngôn ngữ/Dự đoán từ

Mô hình hóa ngôn ngữ thống kê (*statistical language modeling*) (gọi tắt là **mô hình hóa ngôn ngữ** (*language modeling*)) là nhiệm vụ gán một xác suất $\Pr(w_1, \cdots, w_n)$ cho một chuỗi từ $w_1 \cdots w_n$. Xác suất đồng thời này thường được phân tích thành tích của các xác suất có điều kiện bằng quy tắc chuỗi:

$$\begin{aligned}\Pr(w_1, \cdots, w_n) &= \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_n|w_1, \cdots, w_{n-1}) \\ &= \prod_{i=1}^{n} \Pr(w_i|w_1, \cdots, w_{i-1}).\end{aligned} \tag{1.93}$$

Phương trình (1.93) mô tả một quy trình sinh chuỗi từ từ trái sang phải (gọi là sinh **tự hồi quy** (*auto-regressive*)). Ước lượng $\Pr(w_i|w_1, \cdots, w_{i-1})$ về bản chất là bài toán dự đoán từ bị thiếu: ta che đi từ cuối cùng của một chuỗi và hướng dẫn mô hình ngôn ngữ dự đoán đúng từ ở vị trí đó. Dưới đây là một chuỗi từ bị thiếu từ cuối cùng.

Pride and prejudice is one of the best known ___
("Kiêu hãnh và định kiến là một trong những ___ nổi tiếng nhất")

Ta có thể dùng lại ý tưởng của bài toán phân loại để mô hình hóa phân phối xác suất $\Pr(__ \,|\, \textit{Pride}, \textit{and}, \cdots, \textit{known})$. Gọi $\mathbf{x}_i$ là biểu diễn vectơ của w_i. Ta có thể định nghĩa một hàm đọc $\mathbf{x}_1 \cdots \mathbf{x}_{i-1}$ và tạo ra một vectơ $\mathbf{h}_i$:

$$\mathbf{h}_i \;=\; f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1}), \tag{1.94}$$

trong đó $\mathbf{h}_i$ biểu diễn trạng thái trung gian của mô hình tại vị trí i. Để thu được một phân phối xác suất hợp lệ, ta chuẩn hóa $\mathbf{h}_i$ bằng một hàm chuẩn hóa $\psi(\cdot)$ nào đó. Như vậy, phân phối tại vị trí i là

$$\begin{aligned}\mathbf{y}_i &= \psi(\mathbf{h}_i) \\ &= \psi(f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1})).\end{aligned} \tag{1.95}$$

Rõ ràng $y_i(w_i)$ là xác suất của w_i khi biết các từ đứng trước, tức là $y_i(w_i) = \Pr(w_i|w_1, \cdots, w_{i-1})$. Lưu ý rằng Phương trình (1.93) chỉ xét ngữ cảnh bên trái khi dự đoán một từ. Một mở rộng tự nhiên là đặt điều kiện dự đoán trên toàn bộ ngữ cảnh có sẵn. Chẳng hạn, xét một câu có một từ bị che ở giữa.

Pride and ___ is one of the best-known novels
("Kiêu hãnh và ___ là một trong những cuốn tiểu thuyết nổi tiếng nhất")

Trong ví dụ này, ta có thể dự đoán từ bị che bằng cách dùng cả ngữ cảnh bên trái (*Pride and*) lẫn ngữ cảnh bên phải (*is one of the best known novels*):

$$\mathbf{y}_i = \psi(f(\mathbf{x}_1, \cdots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \cdots, \mathbf{x}_n)). \tag{1.96}$$

Đây là một mô hình *hai chiều*, thường được dùng trong các phương pháp **tự mã hóa** (*auto-encoding*) để học các mô hình biểu diễn chuỗi [Devlin và cộng sự, 2019].

Chú giải của người dịch: Quy tắc chuỗi trong mô hình ngôn ngữ: ví dụ số

Với câu "tôi thích trái cây" (coi mỗi từ là một token), Phương trình (1.93) cho $\Pr(\text{tôi thích trái cây}) = \Pr(\text{tôi}) \cdot \Pr(\text{thích}|\text{tôi}) \cdot \Pr(\text{trái}|\text{tôi thích}) \cdot \Pr(\text{cây}|\text{tôi thích trái})$. Nếu bốn xác suất này lần lượt là $0.1, 0.2, 0.05, 0.9$ thì xác suất cả câu là $0.1 \cdot 0.2 \cdot 0.05 \cdot 0.9 = 0.0009$. Mỗi thừa số chính là một bài toán "đoán từ tiếp theo", tức một bài toán phân loại với số lớp bằng kích thước bộ từ vựng. Mô hình *tự hồi quy* chỉ nhìn sang trái (giống GPT); mô hình *hai chiều* như (1.96) nhìn cả hai phía (giống BERT).

Lưu ý rằng dù mô hình hóa ngôn ngữ trước đây được xem là một nhiệm vụ cơ bản trong NLP, ngày nay nó được công nhận là nền tảng tuyệt đối của các hệ thống NLP hiện đại. Đặc biệt, cùng với sự phát triển của các mô hình ngôn ngữ lớn (*large language models*), dự đoán từ đã trở thành cơ chế chính nhờ đó các mô hình thu nhận tri thức sâu về ngôn ngữ và về thế giới. Như ta sẽ thấy ở các Chương 7–11, mô thức mô hình hóa ngôn ngữ được sử dụng rộng rãi để giải quyết rất nhiều nhiệm vụ hạ nguồn (*downstream tasks*) khác nhau.

1.5.4 Sinh chuỗi

Sinh chuỗi (*sequence generation*) bao gồm nhiều bài toán NLP, như dịch máy, tóm tắt văn bản (*summarization*), hỏi đáp (*question answering*), hệ thống hội thoại (*dialogue systems*), v.v. Thông thường, nó chỉ việc ánh xạ một dữ liệu nào đó thành một chuỗi. Ở đây ta tập trung vào bài toán **chuỗi-sang-chuỗi** (*sequence-to-sequence*), trong đó một chuỗi phía nguồn được

biến đổi thành một chuỗi phía đích, dù sinh chuỗi không chỉ giới hạn ở việc làm việc với cấu trúc dạng chuỗi ở phía nguồn.

Để tiện ký hiệu, từ đây trở đi ta dùng biến in đậm để chỉ các chuỗi. Ví dụ, **a** là một chuỗi có độ dài n. Nó có thể được viết là $\begin{bmatrix} a_1 & \cdots & a_n \end{bmatrix}$ hoặc $a_1 \cdots a_n$. Gọi $\mathbf{s} = s_1 \cdots s_m$ và $\mathbf{t} = t_1 \cdots t_n$ lần lượt là chuỗi nguồn và chuỗi đích. Bài toán chuỗi-sang-chuỗi có thể được mô tả là tìm chuỗi phía đích cực đại hóa $\Pr(\mathbf{t}|\mathbf{s})$:

$$\hat{\mathbf{t}} \;=\; \arg\max_{\mathbf{t}} \Pr(\mathbf{t}|\mathbf{s}). \tag{1.97}$$

Giống như mô hình hóa ngôn ngữ, $\Pr(\mathbf{t}|\mathbf{s})$ có thể được hình thức hóa theo kiểu tự hồi quy:

$$\begin{aligned}
\Pr(\mathbf{t}|\mathbf{s}) &= \Pr(t_1, \cdots, t_n|\mathbf{s}) \\
&= \Pr(t_1|\mathbf{s}) \cdot \Pr(t_2|\mathbf{s}, t_1) \cdots \Pr(t_n|\mathbf{s}, t_1, \cdots, t_{n-1}) \\
&= \prod_{i=1}^{n} \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1}).
\end{aligned} \tag{1.98}$$

Phương trình (1.98) chỉ khác Phương trình (1.93) ở điều kiện bổ sung (tức **s**) được đưa vào các xác suất. Theo nghĩa này, ta có thể dùng Phương trình (1.94–1.95) để mô hình hóa $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$. Tuy nhiên, việc đặt điều kiện trên **s** làm bài toán phức tạp hơn, vì ta phải mô hình hóa mối quan hệ giữa hai chuỗi **s** và t_i. Một cách tiếp cận trực tiếp cho sinh chuỗi là phát biểu $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$ theo mô thức **bộ mã hóa–bộ giải mã** (*encoder-decoder*). Có hai bước: trước tiên, một bộ mã hóa được dùng để biểu diễn **s** dưới một dạng trung gian nào đó (ví dụ một vectơ), sau đó một bộ giải mã được dùng để mô hình hóa cả các từ phía đích lẫn mối tương quan giữa đầu ra của bộ mã hóa và các từ phía đích. Gộp lại, đầu ra của mô hình bộ mã hóa–bộ giải mã có thể được định nghĩa là:

$$\mathbf{y}_i \;=\; \text{Decode}(\text{Encode}(\mathbf{s}), t_1, \cdots, t_{i-1}), \tag{1.99}$$

trong đó $\text{Encode}(\cdot)$ là bộ mã hóa và $\text{Decode}(\cdot)$ là bộ giải mã. $\mathbf{y}_i$ là một phân phối xác suất trên bộ từ vựng phía đích tại vị trí i, tức là $y_i(t_i) = \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$. Chương 5 sẽ mô tả chi tiết mô hình bộ mã hóa–bộ giải mã.

Chú giải của người dịch: Bộ mã hóa–bộ giải mã qua ví dụ dịch máy

Dịch câu nguồn **s** = "I like fruit" sang **t** = "tôi thích trái cây". Bộ mã hóa "đọc hiểu" toàn bộ câu tiếng Anh và tóm nó thành một biểu diễn trung gian. Bộ giải mã sinh từng từ tiếng Việt: lúc đầu đoán "tôi" dựa trên Encode(**s**); sau đó đoán "thích" dựa trên Encode(**s**) và "tôi"; rồi cứ thế tiếp tục. So với mô hình ngôn ngữ ở Phương trình (1.93), điểm khác duy nhất là mọi dự đoán đều được "nhìn" thêm câu nguồn **s**.

1.5.5 Sinh cây

Trong NLP, cây thường được dùng để biểu diễn cấu trúc hoặc ý nghĩa của dữ liệu tuần tự. Ví dụ, một **bộ phân tích cú pháp** (*syntactic parser*) phân tích một câu để tạo thành một **cây cú pháp** (*syntax tree*) hay **cây phân tích cú pháp** (*parse tree*). Một cách hình thức hơn, cho một chuỗi từ $\mathbf{s} = s_1 \cdots s_m$, bài toán phân tích cú pháp có thể được định nghĩa là:

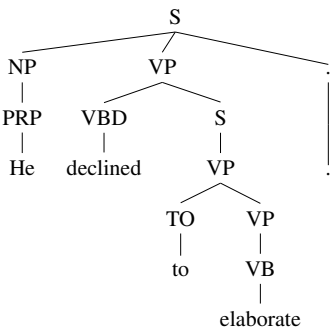
$$\hat{d} \;=\; \arg\max_{d \in \mathcal{D}} \Pr(d|\mathbf{s}), \tag{1.100}$$

trong đó d là một cây phân tích cú pháp, và $\mathcal{D}$ là tập tất cả các cây phân tích cú pháp sinh ra $s_1 \cdots s_m$. Tính $\Pr(d|\mathbf{s})$ là một thách thức, vì độ phức tạp của việc mô hình hóa tăng theo hàm mũ khi chuyển từ chuỗi sang cây. Trong **phân tích cú pháp thống kê** (*statistical parsing*), một giải pháp là mô hình hóa d như một dẫn xuất (*derivation*) của các quy tắc cú pháp. Bằng cách này, $\Pr(d|\mathbf{s})$ có thể được phát biểu là tích các xác suất của các quy tắc. Hình 1.10 trình bày một ví dụ phân tích cú pháp với các quy tắc của văn phạm phi ngữ cảnh (*context-free grammar*) (CFG). Ngoài ra, $\Pr(d|\mathbf{s})$ cũng có thể được mô hình hóa theo kiểu đầu-cuối (*end-to-end*). Ví dụ, một số cách tiếp cận gần đây thực hiện phân tích cú pháp bằng cách định nghĩa một mạng nơ-ron trên cây phân tích cú pháp. Xác suất của một cây con có gốc tại một nút được tính bằng cách xét tương tác giữa nút này và các nút con của nó.

Chú giải của người dịch: Đọc quy tắc văn phạm

Một quy tắc như "S → NP VP ." đọc là: "một câu (S) có thể được tạo thành từ một cụm danh từ, theo sau là một cụm động từ và dấu chấm". Quy tắc

Parse Tree:



CFG Rules:

r_1 : PRP → He
r_2 : VBD → declined
r_3 : TO → to
r_4 : VB → elaborate
r_5 : . → .
r_6 : NP → PRP
r_7 : VP → VB
r_8 : VP → TO VP
r_9 : S → VP
r_{10} : VP → VBD S
r_{11} : S → NP VP .

$$\begin{aligned}\mathrm{P}(d|\mathbf{s}) = &\mathrm{P}(\mathrm{PRP} \to \mathrm{He}) \cdot \mathrm{P}(\mathrm{VBD} \to \mathrm{declined}) \cdot \mathrm{P}(\mathrm{TO} \to \mathrm{to}) \cdot \mathrm{P}(\mathrm{VB} \to \mathrm{elaborate}) \cdot \\ &\mathrm{P}(. \to .) \cdot \mathrm{P}(\mathrm{NP} \to \mathrm{PRP}) \cdot \mathrm{P}(\mathrm{VP} \to \mathrm{VB}) \cdot \mathrm{P}(\mathrm{VP} \to \mathrm{TO\ VP}) \cdot \mathrm{P}(\mathrm{S} \to \mathrm{VP}) \cdot \\ &\mathrm{P}(\mathrm{VP} \to \mathrm{VBDS}) \cdot \mathrm{P}(\mathrm{S} \to \mathrm{NP\ VP\ .}) \\ = &\prod_{i=0}^{11} \mathrm{P}(r_i)\end{aligned}$$

Hình 1.10: Một ví dụ về cây phân tích cú pháp và các quy tắc CFG. Câu ví dụ được lấy từ dữ liệu huấn luyện của nhiệm vụ chung CoNLL 2000. Cây phân tích cú pháp được biểu diễn như một dẫn xuất từ các quy tắc CFG. Xác suất của cây phân tích cú pháp được định nghĩa là tích các xác suất của các quy tắc. (Câu ví dụ: "He declined to elaborate." – "Ông ấy từ chối nói rõ thêm." *Parse Tree* = cây phân tích cú pháp, *CFG Rules* = các quy tắc CFG; S = câu, NP = cụm danh từ, VP = cụm động từ, PRP = đại từ nhân xưng, VBD = động từ quá khứ, TO = từ "to", VB = động từ nguyên thể.)

"PRP $\rightarrow$ He" nói rằng đại từ PRP có thể là từ "He". Bắt đầu từ S và lần lượt áp dụng 11 quy tắc $r_1, \ldots, r_{11}$, ta "mọc" ra đúng cây trong Hình 1.10. Nếu mỗi quy tắc có một xác suất (ví dụ $\mathrm{P}(\mathrm{S} \rightarrow \mathrm{NP\ VP\ .}) = 0.6$), xác suất của cả cây là tích các xác suất đó — tương tự cách mô hình ngôn ngữ nhân các xác suất từng từ ở Phương trình (1.93).

Một cách khác là coi phân tích cú pháp như bài toán sinh chuỗi. Ví dụ, ta có thể tuyến tính hóa (*linearize*) một cây phân tích cú pháp và biểu diễn nó thành một chuỗi gồm các từ và nhãn cú pháp, hoặc biến quá trình sinh cây thành một chuỗi các hành động. Điều này cho phép dùng các kỹ thuật chuỗi-sang-chuỗi để giải quyết bài toán chuỗi-sang-cây.

Trong ngôn ngữ học và NLP, cấu trúc cây được dùng rất nhiều cho phân tích cú pháp. Ngoài phân tích câu, chúng còn được dùng để biểu diễn cấu trúc phân cấp bên trong của từ (hình thái học (*morphology*)), của cụm từ và của toàn bộ diễn ngôn (*discourse*). Tuy nhiên, cây không phải là cách duy nhất để biểu diễn các quan hệ phi tuyến. Một cấu trúc dữ liệu tổng quát hơn là đồ thị (*graph*). Dù cây có thể được xem là một lớp đặc biệt của đồ thị, có những hiện tượng ngôn ngữ mà cây không thể nắm bắt đầy đủ [Fellbaum, 2005; Singhal, 2005; Banarescu và cộng sự, 2013]. Ví dụ, trong biểu diễn ngữ nghĩa của một câu, ta thường cần một đồ thị để nối các vị từ (*predicates*) với các tham tố (*arguments*) của chúng, cho phép một nút có nhiều nút cha. Dù việc sinh đồ thị tổng quát đòi hỏi nhiều tính toán hơn so với phân tích một câu thành cây, ta vẫn có thể điều chỉnh thành công nhiều phương pháp tinh vi vốn được phát triển cho sinh chuỗi và sinh cây.

1.5.6 Mô hình hóa mức độ liên quan

Nói chung, mức độ liên quan (*relevance*) chỉ mức độ mà một thực thể có liên hệ với một thực thể khác. Khái niệm mức độ liên quan được dùng trong nhiều lĩnh vực con khác nhau của NLP và khoa học thông tin. Ví dụ, trong truy xuất thông tin (*information retrieval*), mức độ liên quan được dùng để mô tả một tài liệu được truy xuất đáp ứng truy vấn đến mức nào. Khái niệm này còn là nền tảng của hỏi đáp, hệ thống hội thoại, phân tích ngữ nghĩa và nhiều nhiệm vụ khác cần đến một cơ chế đối sánh hoặc truy xuất.

Hãy xét một mô tả tổng quát hơn. Giả sử ta có một query (truy vấn (*query*)) và một key (khóa (*key*)) biểu diễn thứ mà ta muốn đối sánh với truy vấn. Khi đó, ta định nghĩa vectơ đặc trưng của truy vấn và khóa là

$$\mathbf{q} = Q(\text{query}) \tag{1.101}$$

$$\mathbf{k} = K(\text{key}), \tag{1.102}$$

trong đó $Q(\cdot)$ và $K(\cdot)$ là các bộ trích xuất đặc trưng (*feature extractors*). Mức độ liên quan giữa truy vấn và khóa sau đó được tính bằng một hàm tính điểm (*scoring function*):

$$r = f(\mathbf{q}, \mathbf{k}). \tag{1.103}$$

Hàm $f(\cdot)$ có thể là một độ đo khoảng cách đơn giản (nếu $\mathbf{q}$ và $\mathbf{k}$ nằm trong cùng một không gian vectơ) cho đến một mô hình có tham số phức tạp thực hiện các phép biến đổi phi tuyến. Thực ra, cách định nghĩa mức độ liên quan này có thể được dùng trong nhiều tình huống khác nhau. Đôi khi mức độ liên quan còn được gọi là độ tương đồng (*similarity*) hay độ tương quan (*correlation*). Gọi x và y là hai mẫu (ví dụ hai từ). Độ tương đồng của x và y được cho bởi

$$r = f(g(x), g(y)), \tag{1.104}$$

trong đó $g(\cdot)$ là một bộ trích xuất đặc trưng, và $f(\cdot)$ là một **hàm tương đồng** (*similarity function*). Việc học cả $g(\cdot)$ lẫn $f(\cdot)$ được gọi là **học độ tương đồng** (*similarity learning*). Trong một cách thiết lập học độ tương đồng, ta cố định $f(\cdot)$ và học $g(\cdot)$ sao cho các mẫu tương đồng cho ra đầu ra $g(\cdot)$ tương đồng. Hơn nữa, việc huấn luyện bộ trích xuất đặc trưng không nhất thiết phải gắn liền với hàm tương đồng ở hạ nguồn. Ví dụ, để có được độ tương đồng giữa các từ, ta có thể tiền huấn luyện $g(\cdot)$ bằng một mục tiêu mô hình hóa ngôn ngữ rồi sau đó dùng nó cùng với nhiều hàm tương đồng có sẵn khác nhau (như độ tương đồng cosin (*cosine similarity*)). Điều này phản ánh một triết lý rộng hơn trong học máy hiện đại: tách rời học biểu diễn (*representation learning*) khỏi nhiệm vụ hạ nguồn cụ thể. Ý tưởng này được áp dụng rộng rãi trong tiền huấn luyện, góp phần đẩy các kết quả tiên tiến nhất gần đây trên nhiều nhiệm vụ NLP.

Trong một cách thiết lập khác của học độ tương đồng, ta có thể học trực tiếp $f(\cdot)$. Việc này có thể được thực hiện bằng cách học đồng thời

$f(\cdot)$ và $g(\cdot)$, hoặc học $f(\cdot)$ dựa trên một $g(\cdot)$ cố định. Bài toán này cũng liên quan đến **học độ đo** (*metric learning*). Thông thường, học độ đo được phát biểu như một bài toán có giám sát [Kulis, 2013]. Một hàm tương đồng mong muốn có thể được học với sự giám sát dựa trên một độ tương đồng chuẩn nào đó. Tuy nhiên, trong thực tế NLP thường không có thông tin giám sát như vậy. Khi đó, ta có thể dùng *khoảng cách tương đối* làm tín hiệu giám sát thay thế. Ví dụ, hàm tương đồng có thể được học bằng cách tối ưu một hàm mất mát tương phản (*contrastive loss*) (xem Mục 1.3.4).

Đo độ tương đồng giữa các đối tượng đóng vai trò quan trọng trong nhiều phương pháp học máy, như phân cụm (*clustering*) và phân loại láng giềng gần nhất (*nearest neighbor classification*). Trong NLP, nó hữu ích để khám phá quan hệ giữa các từ, cụm từ, câu và tài liệu; chẳng hạn, độ tương đồng là một cách để kiểm tra xem các vectơ từ tương ứng thế nào với hiểu biết của chúng ta về nghĩa của từ [Mikolov và cộng sự, 2013c; Pennington và cộng sự, 2014].

Chú giải của người dịch: Độ tương đồng cosin

Độ tương đồng cosin của hai vectơ chính là côsin của góc giữa chúng, như ở lớp 10: $\cos(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}\cdot\mathbf{k}}{\|\mathbf{q}\|\,\|\mathbf{k}\|}$. Ví dụ $\mathbf{q} = (1; 2)$, $\mathbf{k} = (2; 4)$ cùng hướng nên cosin bằng $\frac{1\cdot 2 + 2\cdot 4}{\sqrt{5}\cdot\sqrt{20}} = \frac{10}{10} = 1$ (giống nhau nhất); hai vectơ vuông góc cho cosin bằng 0. Nếu $g(\cdot)$ biến các từ "mèo" và "chó" thành hai vectơ gần cùng hướng, còn "mèo" và "ô tô" thành hai vectơ gần vuông góc, ta nói $g(\cdot)$ đã nắm được nghĩa của từ. Ý tưởng truy vấn–khóa ở đây sẽ gặp lại trong cơ chế chú ý của Transformer.

1.5.7 Gióng hàng ngôn ngữ

Gióng hàng ngôn ngữ (*linguistic alignment*) là một nhóm bài toán trong đó ta thiết lập một sự tương ứng nào đó giữa hai tập đơn vị ngôn ngữ. Trong NLP, các bài toán chuỗi-sang-chuỗi và chuỗi-sang-cây về bản chất là các bài toán gióng hàng ngôn ngữ, vì cả hai đều kết nối hai đơn vị ngôn ngữ. Tuy nhiên, theo quy ước, thuật ngữ *gióng hàng* (*alignment*) dùng để chỉ việc gióng hàng nhiều đối tượng cùng một lúc[21].

[21] Khái niệm *alignment* rất rộng. Ở đây chúng tôi dùng thuật ngữ *gióng hàng ngôn ngữ* (*linguistic alignment*) để phân biệt với *căn chỉnh* (*alignment*) của các mô hình ngôn ngữ lớn được bàn ở các chương sau.

Ví dụ, hãy xét nhiệm vụ nổi tiếng **gióng hàng từ** (*word alignment*): ta gióng các từ của một câu với các từ của một câu khác. Ta dùng lại ký hiệu ở Mục 1.5.4 vì cả nhiệm vụ chuỗi-sang-chuỗi lẫn gióng hàng từ đều làm việc trên một cặp chuỗi. Cho chuỗi từ phía nguồn $\mathbf{s} = s_1 \cdots s_m$ và chuỗi từ phía đích $\mathbf{t} = t_1 \cdots t_n$, gióng hàng từ giữa hai chuỗi được ký hiệu là một ma trận $\mathbf{A}$ kích thước $m \times n$. $A(i,j) = 1$ nếu có một **liên kết gióng hàng** (*alignment link*) giữa s_i và t_j, và $A(i,j) = 0$ trong trường hợp ngược lại. Phép gióng hàng tối ưu có thể được định nghĩa là:

$$\hat{\mathbf{A}} \;=\; \arg\max_{\mathbf{A}} \Pr(\mathbf{A}|\mathbf{s},\mathbf{t}), \tag{1.105}$$

trong đó $\Pr(\mathbf{A}|\mathbf{s},\mathbf{t})$ là xác suất gióng hàng từ. Giống như các bài toán học máy khác, ta có thể mô hình hóa $\Pr(\mathbf{A}|\mathbf{s},\mathbf{t})$ theo kiểu sinh hoặc kiểu phân biệt (xem Mục 1.2.4). Chẳng hạn, trong công trình của Brown và cộng sự [1993], mô hình gióng hàng từ được phân tích thành nhiều bước sinh, mỗi bước ứng với một số giả định về bài toán[22].

Một ma trận gióng hàng 0-1 thể hiện một sơ đồ gióng hàng cứng (*hard alignment*). Vấn đề ở đây là mô hình cứng có thể không mô tả tốt những phép gióng hàng từ có tính nhập nhằng cao. Vì vậy, ta có thể biểu diễn $\mathbf{A}$ dưới dạng một ma trận số thực (gọi là **ma trận gióng hàng từ mềm** (*soft word alignment matrix*) hay **ma trận trọng số gióng hàng từ** (*word alignment weight matrix*)). Giả sử các từ phía nguồn được biểu diễn thành một chuỗi các vectơ đặc trưng $\mathbf{x} = \begin{bmatrix}\mathbf{x}_1 & \cdots & \mathbf{x}_m\end{bmatrix}$. Tương tự, các từ phía đích được biểu diễn là $\mathbf{y} = \begin{bmatrix}\mathbf{y}_1 & \cdots & \mathbf{y}_n\end{bmatrix}$. Một mô hình gióng hàng từ mềm được cho bởi:

$$\mathbf{A} \;=\; a(\mathbf{s},\mathbf{t}), \tag{1.106}$$

trong đó $a(\cdot)$ là một hàm gióng hàng từ tính trọng số gióng hàng $A(i,j)$ cho mỗi cặp $\mathbf{x}_i$ và $\mathbf{y}_j$. Thực ra, mọi phương pháp bàn ở Mục 1.5.6 đều có thể áp dụng để thiết kế $a(\cdot)$. Điều này gắn việc mô hình hóa gióng hàng từ với việc mô hình hóa độ tương đồng, và cho phép giải quyết các bài toán NLP khác nhau bằng cùng một cách tiếp cận học máy.

[22] Chính xác hơn, Brown và cộng sự [1993] mô hình hóa $\Pr(\mathbf{A},\mathbf{s}|\mathbf{t})$, một đại lượng thay thế cho $\Pr(\mathbf{A}|\mathbf{s},\mathbf{t})$, vì $\Pr(\mathbf{A}|\mathbf{s},\mathbf{t}) = \frac{\Pr(\mathbf{A},\mathbf{s}|\mathbf{t})}{\Pr(\mathbf{s}|\mathbf{t})} = \frac{\Pr(\mathbf{A},\mathbf{s}|\mathbf{t})}{\sum_{\mathbf{A}'}\Pr(\mathbf{A}',\mathbf{s}|\mathbf{t})}$.

Chú giải của người dịch: Ma trận gióng hàng: ví dụ

Gióng câu "I like fruit" ($m = 3$) với "tôi thích trái cây" ($n = 4$). Gióng hàng cứng là ma trận 3×4 gồm 0 và 1:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad \text{(hàng: I, like, fruit; cột: tôi, thích, trái, cây)},$$

tức "fruit" ứng với cả "trái" và "cây". Ở gióng hàng mềm, các số 0/1 được thay bằng trọng số, chẳng hạn hàng "fruit" có thể là $[0.05 \;\; 0.05 \;\; 0.5 \;\; 0.4]$, diễn tả mức độ "chắc chắn" của từng liên kết — giống các ô vuông to nhỏ trong Hình 1.11(g).

Phương trình (1.106) cung cấp một cách rất tổng quát để khám phá mối liên hệ tiềm ẩn giữa các cặp biến. Ngoài việc gióng hàng các từ trong chuỗi, nó còn hữu ích để gióng hàng các đối tượng không có thứ tự. Ví dụ, trong quy nạp từ điển song ngữ (*bilingual dictionary induction*), ta có thể học một ma trận trọng số như vậy để ước lượng một từ trong ngôn ngữ này tương ứng mạnh đến đâu với một từ trong ngôn ngữ kia.

Lưu ý cuối cùng liên quan đến vai trò của các mô hình gióng hàng ngôn ngữ. Dù gióng hàng ngôn ngữ có thể được xem là một nhiệm vụ NLP độc lập, nó thường được dùng để thiết kế các mô hình con của một số hệ thống hạ nguồn. Nhiều hệ thống mô hình hóa quan hệ ở mức từ có chứa biểu diễn ngầm của gióng hàng ngôn ngữ. Do đó, gióng hàng ngôn ngữ được coi như một số trạng thái ẩn (*latent states*), và là sản phẩm phụ của các hệ thống này. Ví dụ, trong những ngày đầu của dịch máy thống kê (*statistical machine translation*), gióng hàng từ là một biến ẩn được dùng để mô hình hóa ánh xạ giữa các chuỗi. Kết quả gióng hàng từ có thể dễ dàng được suy ra từ một mô hình dịch máy. Gần đây hơn, các mô hình chuỗi-sang-chuỗi nơ-ron — tiêu biểu nhất là các mô hình có cơ chế chú ý (*attention*) [Bahdanau và cộng sự, 2014] — đã cố gắng làm điều tương tự như gióng hàng từ bằng cách tính các trọng số chú ý giữa các từ.

1.5.8 Trích xuất thông tin

Trong NLP, *trích xuất* không phải là một loại nhiệm vụ mà là một kiểu hành vi mà hệ thống thể hiện. Nói một cách không hình thức, nó chỉ quá

trình thu thập và chắt lọc thông tin có cấu trúc từ một số nguồn thông tin. Vì vậy, thuật ngữ trích xuất thường xuất hiện cùng các thuật ngữ khác để tạo thành một nhiệm vụ cụ thể, như **trích xuất từ khóa** (*keyword extraction*), **trích xuất sự kiện** (*event extraction*) và **trích xuất quan hệ** (*relation extraction*). Nhiều nhiệm vụ trong số này có thể được xếp vào một lĩnh vực — **trích xuất thông tin** (*information extraction*). Trích xuất thông tin có lẽ là chủ đề rộng nhất trong NLP. Không có danh sách đầy đủ các nhiệm vụ trích xuất thông tin. Theo cuốn sách của Jurafsky và Martin [2008], nó bao gồm (nhưng không giới hạn ở) nhận dạng thực thể có tên, phân giải đồng tham chiếu (*coreference resolution*), trích xuất quan hệ, trích xuất sự kiện và điền mẫu (*template filling*).

Vì trích xuất thông tin là một tập hợp không đồng nhất của nhiều bài toán đa dạng, nó không thể được phát biểu trong một mô thức học máy duy nhất. May mắn là phần lớn các bài toán này có thể được đưa về các bài toán học máy tiêu chuẩn, như phân loại và gán nhãn chuỗi, và có thể được giải bằng các công cụ có sẵn. Trong một số trường hợp, có thể cần điều chỉnh nhẹ các mô hình hiện có để thích ứng với nhiệm vụ mới. Ví dụ, trích xuất một đoạn cụ thể từ văn bản có thể đòi hỏi hệ thống tạo ra một *đoạn* (span) cho biết vị trí bắt đầu và kết thúc của đoạn được trích xuất (gọi là **dự đoán đoạn** (*span prediction*)).

Về mặt thực tiễn, không phải lúc nào cũng cần học máy để trích xuất thông tin từ văn bản. Nhiều bài toán có thể giải bằng các quy tắc viết tay. Một ví dụ là dùng biểu thức chính quy (*regular expressions*) để nhận diện địa điểm và ngày tháng trong văn bản. Trong thực tế, các hệ thống thật thường là sự kết hợp giữa các phương pháp kinh nghiệm (heuristic) và các phương pháp học máy tự động.

Chú giải của người dịch: Dự đoán đoạn và biểu thức chính quy

Với câu "Hội nghị diễn ra tại Đà Nẵng vào ngày 15/3/2025" (đánh số token từ 1), nếu cần trích xuất địa điểm, hệ thống dự đoán đoạn chỉ cần đưa ra hai số: vị trí bắt đầu và kết thúc của "Đà Nẵng", tức $span = [6, 7]$ (tương tự $span = [2, 4]$ trong Hình 1.11(h)). Còn ngày tháng có thể bắt bằng một quy tắc viết tay dạng "một hoặc hai chữ số, dấu /, một hoặc hai chữ số, dấu /, bốn chữ số" — đó chính là ý tưởng của biểu thức chính quy, không cần học máy.

1.5.9 Nhận xét

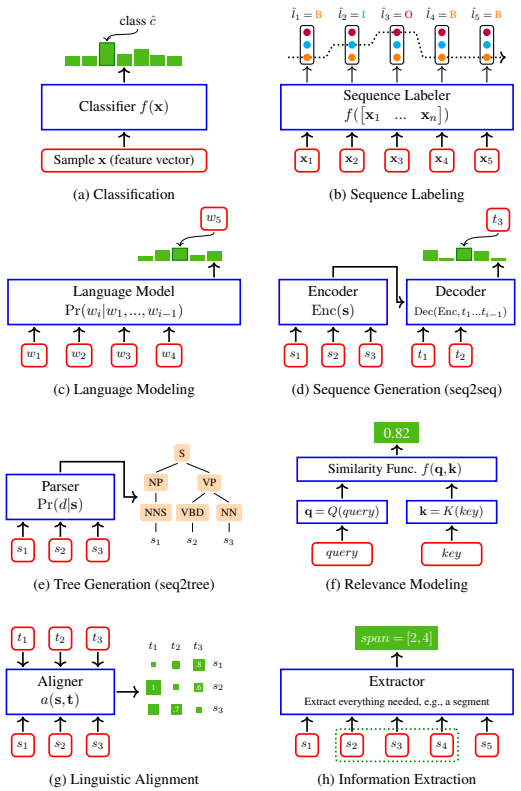
Hình 1.11 minh họa các nhiệm vụ NLP nói trên. Lưu ý rằng ta mới chỉ tập trung vào các bài toán NLP trong mô thức học có giám sát (*supervised learning*). Nhiều nhiệm vụ không giám sát cũng quan trọng đối với nghiên cứu NLP. Ví dụ, việc phân cụm các từ hoặc tài liệu chưa gán nhãn để thuận tiện cho xử lý trong các hệ thống hạ nguồn là điều phổ biến. Có một số phương pháp có thể áp dụng trực tiếp cho nhiệm vụ này [Murphy, 2012]. Một xu hướng gần đây trong NLP là không cần đặt ranh giới chặt chẽ giữa việc dùng học có giám sát và học không giám sát (*unsupervised learning*). Trong nhiều trường hợp, các phương pháp không giám sát giúp ích cho các nhiệm vụ có giám sát, và ngược lại. Một ví dụ đáng chú ý là ta học một bộ trích xuất đặc trưng tiền huấn luyện trên dữ liệu chưa gán nhãn rồi xây dựng một bộ phân loại có giám sát bên trên nó. Cách tiếp cận này được biết đến rộng rãi với tên gọi **học tự giám sát** (*self-supervised learning*): nó vẫn tuân theo khuôn khổ học có giám sát, nhưng tín hiệu giám sát được rút ra từ chính dữ liệu chưa gán nhãn. Điều này đã dẫn tới một sự chuyển dịch mô thức hướng tới việc phát triển các mô hình biểu diễn (tức các bộ trích xuất đặc trưng) một cách độc lập, không phụ thuộc vào các nhiệm vụ có giám sát cụ thể ở hạ nguồn.

Chú giải của người dịch: Học tự giám sát

Ở mô hình ngôn ngữ (Mục 1.5.3), "nhãn" cần dự đoán chính là từ tiếp theo, vốn có sẵn trong văn bản; vì vậy ta có thể tạo ra hàng tỉ "bài tập có đáp án" từ văn bản thô mà không cần ai gán nhãn. Đây là ý nghĩa của "tín hiệu giám sát được rút ra từ chính dữ liệu". Mô hình học được theo cách này sau đó làm bộ trích xuất đặc trưng tốt cho nhiều nhiệm vụ khác — nền tảng của các mô hình như BERT và GPT ở các chương sau.

1.6 Tóm tắt

Chương này đã trình bày các ý tưởng cơ bản của học máy và ứng dụng của nó cho các bài toán NLP. Cụ thể, chúng tôi đã trình bày một bài toán phân loại văn bản đơn giản để bắt đầu làm quen với học máy. Chúng tôi cũng đã bàn về một số vấn đề chung của học máy, như các loại phương pháp học máy, thiên kiến quy nạp, hàm mất mát, quá khớp, v.v. Tiếp theo



Hình 1.11: Các nhiệm vụ xử lý ngôn ngữ tự nhiên dưới góc nhìn học máy. (Tên các hình con: (a) phân loại; (b) gán nhãn chuỗi; (c) mô hình hóa ngôn ngữ; (d) sinh chuỗi (chuỗi-sang-chuỗi); (e) sinh cây (chuỗi-sang-cây); (f) mô hình hóa mức độ liên quan; (g) gióng hàng ngôn ngữ; (h) trích xuất thông tin. Nhãn khác: *Classifier* = bộ phân loại, *Sample (feature vector)* = mẫu (vectơ đặc trưng), *Sequence Labeler* = bộ gán nhãn chuỗi, *Language Model* = mô hình ngôn ngữ, *Encoder/Decoder* = bộ mã hóa/bộ giải mã, *Parser* = bộ phân tích cú pháp, *Similarity Func.* = hàm tương đồng, *Aligner* = bộ gióng hàng, *Extractor* = bộ trích xuất, *Extract everything needed, e.g., a segment* = trích xuất mọi thứ cần thiết, chẳng hạn một đoạn văn bản.)

là phần thảo luận về lựa chọn và đánh giá mô hình. Ngoài ra, chúng tôi đã mô tả cách các bài toán NLP hiện đại được phát biểu thành các bài toán học máy.

Tuy nhiên, học máy là một lĩnh vực nghiên cứu khổng lồ. Một số chủ đề thú vị nằm ngoài phạm vi chương này. Một chủ đề mà chúng tôi hầu như chưa nói đến là học tăng cường (*reinforcement learning*). Nói chung, học tăng cường rất mạnh mẽ. Nó nên được, và đã được, xem là một cách tiếp cận để giải quyết các bài toán NLP, chẳng hạn huấn luyện một mô hình chuỗi-sang-chuỗi bằng một hàm mất mát dựa trên rủi ro (*risk*). Một giáo trình về học tăng cường sẽ cung cấp các ý tưởng chung của học tăng cường [Sutton và Barto, 2018]. Một chủ đề khác chúng tôi bỏ qua ở đây là học Bayes (*Bayesian learning*) [Gelman và cộng sự, 2020; McElreath, 2020; Downey, 2021]. Nó mở ra một hướng nghiên cứu đáng chú ý trong học thống kê và đã được dùng thành công trong các nhiệm vụ NLP. Hơn nữa, còn có nhiều chủ đề khác chuyên sâu về một số khía cạnh nhất định của học máy và được các nhà nghiên cứu, kỹ sư NLP quan tâm. Một vài trong số đó là học máy hiệu quả (*efficient machine learning*) [Tay và cộng sự, 2020b], học đa nhiệm (*multi-task learning*) [Zhang và Yang, 2021], học few-shot/zero-shot [Wang và cộng sự, 2019b; 2020c], v.v.

Một điểm cuối cùng để khép lại chương này. Chúng tôi bỏ qua phần thảo luận chi tiết về một số mô hình và thuật toán học máy, như các mô hình phân loại và hồi quy (*regression*), vì bạn đọc quan tâm có thể tìm thấy nhiều tài liệu nhập môn xuất sắc và đầy đủ [Bishop, 2006; Hastie và cộng sự, 2009; Murphy, 2012; Mohri và cộng sự, 2018]. Trong chương tiếp theo, chúng tôi sẽ bàn thêm một chút về mạng nơ-ron nhân tạo (*artificial neural networks*), nền tảng của học sâu và của các mô hình NLP tiên tiến nhất hiện nay.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.